

Verified Program Synthesis for Coinductive Reinforcement Learning

From Observations to Optimal Policies via Predicate DSL

Ricardo Corral-Corral

February 2026

Abstract

We present a verified program synthesis framework that extends Coinductive Symmetric Homomorphism Reinforcement Learning (CSHRL) from learned rankings to *synthesized programs*. Where CSHRL defines optimality as structure preservation between action rankings and infinite reward streams, we add a synthesis layer that *constructs* these rankings automatically from environment observations.

The framework introduces thirteen components, all machine-verified in Agda: **(1)** A *Predicate DSL* (PredProg)—a domain-specific language for Boolean predicates over environment features, parameterized by an abstract carrier type and feature set, enabling instantiation across different Environment Classes; **(2)** A *CEGIS* (Counterexample-Guided Inductive Synthesis) algorithm that enumerates predicate programs and refines a version space by observations, with proven monotonic convergence and strict shrinkage bounds; **(3)** A *Propagation Theorem* establishing that feature-equivalent carriers are indistinguishable to any predicate program, which yields three quantified consequences: identical refinement (Refine-Equiv), observation subsumption, and exploration dissolution—reducing the observations needed from $|C|$ (carrier count) to $|C/\sim|$ (equivalence classes); **(4)** *Exploration completeness*: after observing one representative per equivalence class, no further observation can refine the version space (“dissolution is all you need”); **(5)** *Temporal propagation*: when transitions preserve feature equivalence, observations propagate along entire trajectories, not just across carriers—mirroring CSHRL’s coinductive unfolding; **(6)** *Feature sufficiency*: when the target function respects feature equivalence, every CEGIS survivor is provably correct on *all* carriers, not just the observed ones—upgrading stabilization to a full correctness guarantee; **(7)** *Multi-predicate propagation*: when synthesizing multiple predicates from the same feature space (e.g., is-dead and is-solved), observations are shared—one representative set suffices for all predicates, and cross-predicate entailment (e.g., dead $\rightarrow$ $\neg$ solved) further reduces evaluations; **(8)** *Online synthesis with learning integration*: observation order is provably irrelevant (refinement commutes), replayed observations are provably redundant, and a generic LearningBridge connects learning feedback to synthesis with a bridge sufficiency theorem composing learning convergence with synthesis correctness; **(9)** *Sample complexity lower bound*: two targets agreeing on observed carriers produce identical version spaces (same-obs-same-vs), so survivors transfer freely between indistinguishable targets (survivor-transfer); combined with the upper bound (sufficient-cegis), this proves the sample complexity is *exactly* $|C/\sim|$; **(10)** *DSL completeness*: a constructive fingerprint construction proves that the Boolean PredProg DSL is expressively complete for feature-respecting targets—for any target constant on equivalence classes, there exists a PredProg matching it everywhere (dsl-complete); **(11)** *First-Order Stochastic Dominance (FOSD) synthesis*: a complete FOSD pipeline where rankings are based on distributional dominance rather than expected values—with a decidable FOSD comparison (fosd?) proved both sound and complete, the **Stochastic Isomorphism** (coinductive FOSD $\Leftrightarrow$ pointwise FOSD), a **FOSD-to-lex bridge** (FOSDCoindHomo $\Rightarrow$ lexicographic EV-stream dominance), and a **learning convergence** theorem (fosd-obs-sound, model-obs-consistent) connecting FOSD observations to CEGIS version-space refinement; **(12)** *Stochastic Dominance Hierarchy*: a generalization from FOSD to k -th order SD via iterated prefix sums of the CDF—with a one-line hierarchy subsumption proof (SD-subsumes: SD[k] $\Rightarrow$ SD[k+1] via prefix-sum-mono), the

SD-to-EV bridge (sd→ev: every SD level implies expected-value dominance), **SDCoindHomo** k (rankings verified at any level automatically yield EV-stream optimality), and a coinductive **hierarchy subsumption on homos** (sd-homo-subsumes: weaker requirements capture broader utility-function classes—risk-averse (SOSD), prudent (TOSD), and beyond); **(13) Compositional Ranking Algebra**: an algebraic framework for composing independently-verified rankings—proving that prefix-sum is a *linear operator* (distributes over + and ×), that sd-weight inherits this linearity, and that FOSD and SD[k] are closed under distribution concatenation (SD-++) and reward scaling (SD-scale); a VerifiedRanking abstraction with **product composition** (two independently-verified components yield a verified product), **sum composition** (disjoint environments compose with dispatched rankings), and **scaling** (reward amplification preserves verification)—enabling modular, compositional verification where each component is verified in isolation and composed automatically.

We demonstrate the full pipeline end-to-end on three Environment Classes: a Finite Deterministic MDP (1D Maze) where CEGIS synthesizes ranking predicates that yield a verified CoindHomo and optimal policy; a Combinatorial Placement MDP (N -Queens) where synthesized predicates *define* the environment’s step function, enabling find-policy to solve 4-Queens instantly and 8-Queens (with verified equivalence to hand-crafted predicates) including partial solution completion from any intermediate state; and a Stochastic Finite MDP where synthesis operates on expected traces, producing a verified StochasticCoindHomo—demonstrating that the entire generic theory applies unchanged to probabilistic transitions.

This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code while producing this PDF. The synthesis framework builds on the CSHRL foundation [?] without modifying it—all results are additive.

1 Introduction

CSHRL [?] establishes a new foundation for optimal sequential decision making: an agent is optimal when its ranking of actions forms a *coinductive symmetric homomorphism* into the ordering of infinite reward streams. Learning reduces to *restoring* the structural correspondence between rankings and outcomes via violation correction.

But how does a ranking come to exist in the first place?

In the original framework, rankings are either hand-crafted (verified manually) or discovered by the Environment Class find-policy (which exhaustively evaluates the state-action tree). Both approaches have limitations:

- **Hand-crafted rankings** require domain expertise and per-domain proofs.
- **Exhaustive evaluation** computes correct rankings but produces opaque lookup tables—the *structure* of why certain actions dominate others is lost.

Program synthesis offers a third path: *learn a program* that encodes the ranking as a function of observable features. The synthesized program is:

- **Interpretable**: a Boolean combination of feature tests, not a black-box table.
- **Generalizable**: feature-equivalent states automatically receive the same ranking.
- **Verifiable**: the program’s correctness can be checked by Agda’s type system.
- **Compositional**: the DSL is parameterized by carrier type and features, so each Environment Class provides its own feature vocabulary while sharing the synthesis theory.

1.1 Contributions

This paper makes the following contributions, each machine-verified in Agda with `--safe --guardedness`:

1. **Predicate DSL (§3)**: A generic language PredProg for Boolean predicates over features, with a structural FeatureEquiv relation and the **Propagation Theorem**: feature-equivalent carriers evaluate identically under any predicate program.

2. **CEGIS with Convergence (§4):** A counterexample-guided synthesis loop with five proven properties: version-space monotonicity, correct-predicate survival, propagation acceleration, bounded convergence, and strict shrinkage for informative observations.
3. **Propagation Quantification (§5):** Three theorems that formalize how propagation reduces exploration:
 - Refine-Equiv: feature-equivalent carriers produce identical version-space refinement.
 - Observation-Subsumption: after observing at c_1 , any feature-equivalent c_2 is provably redundant.
 - Exploration-Dissolution: an entire equivalence class is handled by a single observation at any representative; the corollary Dissolution-Strict bounds total informative observations by $|C/\sim|$.
4. **Quotient Structure (§6):** AllFeatAgree forms an equivalence relation, inducing a quotient $C/\sim$. The Exploration-Complete theorem proves that after observing one representative per class, no further observation can refine the version space—formalizing “dissolution is all you need.”
5. **Temporal Propagation (§7):** When the transition function preserves feature equivalence, observations propagate along entire trajectories. Trajectory-Dissolution proves that one trajectory from c_1 subsumes all trajectories from equivalent c_2 —extending spatial propagation to the temporal dimension and mirroring CSHRL’s coinductive unfolding.
6. **Feature Sufficiency (§8):** The Feature Sufficiency Theorem (the “crown jewel”): when the target function respects feature equivalence, every CEGIS survivor is provably correct on *all* carriers—not just the observed ones. This upgrades exploration completeness from a stabilization guarantee to a full correctness guarantee.
7. **Multi-Predicate Propagation (§9):** When synthesizing multiple predicates from the same feature space, observations are shared: one representative set suffices for all predicates. Cross-predicate entailment (e.g., $\text{dead} \rightarrow \neg \text{solved}$) gives free observations, and cross-dissolution combines both effects multiplicatively.
8. **Online Synthesis (§10):** Observation refinement is commutative (refine-commutes), making the CEGIS version space determined by the *set* of observations rather than their order. Replayed observations are provably redundant (replay-redundant). A generic LearningBridge module connects domain-specific learning feedback to synthesis observations, with a Bridge-Sufficient theorem composing learning convergence with synthesis correctness.
9. **Sample Complexity Lower Bound (§11):** Two targets agreeing on observed carriers produce identical version spaces (same-obs-same-vs); survivors transfer freely between indistinguishable targets (survivor-transfer). Combined with the upper bound (sufficient-cegis), this proves the sample complexity of CEGIS is *exactly* $|C/\sim|$: one representative per equivalence class is both necessary and sufficient.
10. **DSL Completeness (§12):** The Boolean PredProg DSL is expressively complete for feature-respecting targets. A constructive *fingerprint* construction shows that for any target constant on AllFeatAgree classes, there exists a PredProg matching it everywhere (dsl-complete). This closes the theoretical loop: the DSL always contains the correct answer, CEGIS finds it from $|C/\sim|$ observations, and no fewer suffice.
11. **EC-Specific Instantiation (§13):** Three concrete instantiations—for Finite Deterministic MDPs (ranking synthesis with CoindHomo construction), Combinatorial Placement MDPs (predicate synthesis that *defines* the step function), and Stochastic Finite MDPs (ranking synthesis from expected traces with StochasticCoindHomo construction, §13.3).

12. **Cross-Pair Propagation (§??):** A verified demonstration that CEGIS observations are pair-agnostic. When ranking pairs share the same feature structure, observing a sub-optimal pair (C vs D) determines top-tier rankings ($A > B$)—without ever comparing the actions of interest. Two observations from the bottom-tier pair determine all 6 rankings across all 6 states (18× savings). Verified learning curves (8 states, Agda refl) and scaled rollout performance curves (30 states, 22-program VS) demonstrate a **50% reward improvement** from propagation, with sub-optimal and top-pair observation sequences yielding **identical** rollout reward.
13. **End-to-End Demos (§14):** Three verified pipelines:
 - *1D Maze*: 5 observations → CEGIS → ranking predicates → preservation proof → CoindHomo → policy rollout → cross-validation with find-policy.
 - *4-Queens*: Synthesized PredProg predicates *are* the step function; find-policy solves 4-Queens; partial completion from any intermediate state.
 - *8-Queens*: Synthesized predicates verified equivalent to hand-crafted ones; full solution and partial completion in 8^8 search space.
14. **FOSD Synthesis (§17.9):** A complete First-Order Stochastic Dominance pipeline that upgrades stochastic rankings from expected-value comparison to *distributional* dominance. Five results, all machine-verified:
 - fosd?-complete: the decidable FOSD comparator fosd? is proved complete ($\text{FOSD}_{\leq}$ implies $\text{fosd?} = \text{true}$), complementing the existing soundness proof.
 - Stochastic Isomorphism: coinductive FOSD on reward streams is isomorphic to pointwise FOSD (conjecture-forward, conjecture-reverse)—extending the Stream Isomorphism to distributions.
 - FOSD-to-Lex Bridge: $\text{fosd} \rightarrow \text{lex-stream}$ proves that pointwise FOSD at every depth implies lexicographic EV-stream dominance; $\text{fosd-homo} \rightarrow \text{ev-lex}$ lifts this to show every FOSDCoindHomo is automatically a $\text{StochasticCoindHomo}$.
 - FOSD Learning Convergence: fosd-obs-sound proves that if a RankModel preserves FOSD, then fosd-compare observations are truthful at every depth; $\text{model-obs-consistent}$ proves these observations survive CEGIS refinement; fosd-convergence combines both with version-space monotonicity.
 - FOSD Demos: full PointwiseFOSD and FOSDCoindHomo instances for BiasedBandit and CoinFlip, plus GamblersRuin end-to-end FOSD synthesis with ModelPreservesFOSD and WithLearningBridge.
15. **Stochastic Dominance Hierarchy (§17.10):** A generalization from first-order (FOSD) to k -th order stochastic dominance, implemented in Probability.SD and Core.SD. Key results:
 - SD Hierarchy: k -th order SD defined via iterated prefix sums of the CDF (sd-weight). $\text{SD}[0] = \text{FOSD}$, $\text{SD}[1] = \text{SOSD}$ (risk-averse agents), $\text{SD}[2] = \text{TOSD}$ (prudent agents).
 - Subsumption: a one-line proof that $\text{SD}[k] \Rightarrow \text{SD}[k+1]$ —prefix-sum preserves monotonicity ($\text{SD-subsumes } k \text{ sd} = \text{prefix-sum-mono sd}$).
 - SD-to-EV Bridge: every SD level implies expected-value dominance ($\text{sd} \rightarrow \text{ev}$), proved by showing that sd-weight at $r = 0$ equals CDF at $r = 0$ at all orders.
 - $\text{SDCoindHomo } k$: coinductive homomorphisms parameterized by SD order, with hierarchy subsumption ($\text{sd-homo-subsumes: } \text{SDCoindHomo } k \Rightarrow \text{SDCoindHomo } (k+1)$) and the master bridge ($\text{sd-homo} \rightarrow \text{ev-lex}$: any SD level yields lex EV-stream dominance).
 - Strict Separation: a concrete SOSD-but-not-FOSD example: $\mu = \{0:2, 2:1\}$ vs. $\nu = \{1:3\}$ where FOSD fails at $r=1$ but SOSD holds everywhere.
16. **Compositional Ranking Algebra (§17.11):** An algebraic framework for composing independently-verified rankings, implemented in Probability.Compose and Core.Compose. Key results:

- Prefix-Sum Linearity: prefix-sum distributes over addition (prefix-sum-+) and scalar multiplication (prefix-sum-*)—it is a *linear operator* on $\mathbb{N} \rightarrow \mathbb{N}$.
- sd-weight Distributivity: sd-weight inherits linearity: sd-weight k ($d_1 ++ d_2$) = sd-weight k d_1 + sd-weight k d_2 and sd-weight k (scale c d) = $c \cdot$ sd-weight k d (induction on k , using prefix-sum-ext).
- SD Closure: FOSD and SD[k] are closed under distribution concatenation (SD-++) and reward scaling (SD-scale).
- Product Composition: if two independent components have VerifiedRankings at SD[k], the product (coordinated) ranking is verified at SD[k] for the composite.
- Sum Composition: if two disjoint environments have VerifiedRankings, the dispatched ranking is verified for the union—with impossible state/action mismatches excluded by $\perp$.
- Composability: algebraic operations compose: scaled-product c vr_1 vr_2 = scale-ranking c (++)-product vr_1 vr_2).

1.2 Relationship to the CSHRL Paper

This work is *additive*: it imports the CSHRL core (Core, CoindHomo, solve, value, action-value) without modification. The synthesis framework sits alongside the existing Environment Class and Learning modules.

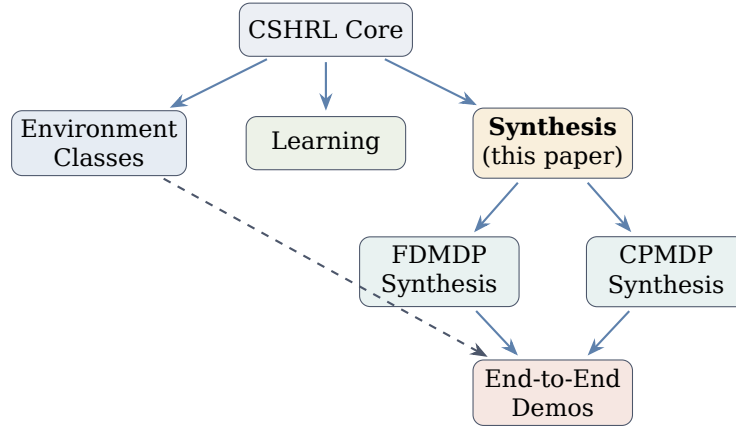


Figure 1: Architecture. The synthesis framework (yellow) builds on the CSHRL Core without modifying it. EC-specific modules instantiate the generic DSL for each domain. End-to-end demos integrate synthesis with Environment Classes.

2 Background: CSHRL in Brief

We briefly recall the CSHRL framework [?] to establish notation. The reader is referred to the companion paper for full details.

Definition 1 (Coinductive Symmetric Homomorphism). An agent’s ranking $\leq_a$ of actions in state s is a *coinductive symmetric homomorphism* if:

$$a \leq_a b \implies \text{action-value}(s, a) \leq_s \text{action-value}(s, b)$$

where $\leq_s$ is coinductive pointwise dominance on infinite reward streams: $x \leq_s y \iff \text{head}(x) \leq_r \text{head}(y) \wedge \text{tail}(x) \leq_s \text{tail}(y)$.

In Agda, this is captured by the CoindHomo record:

```

record CoindHomo : 1Set where
  field
    ≤a_ : State → Action → Action → Set
    preserves : ∀ a b s → s ≤a a b → action-value s a ≤s action-value s b

```

An **Environment Class** (EC) is a parameterized module that, given domain-specific structure (states, actions, transition function), provides trace-action, find-ranking, and find-policy with automatic stream bridges. Two ECs are central to this paper:

- **FiniteDeterministicMDP**: parameterized by finite State, Action, and a deterministic step : State $\rightarrow$ Action $\rightarrow$ State $\times$ Reward.
- **CombinatorialPlacementMDP**: parameterized by a configuration type, placement action, and predicates is-dead, is-solved that determine when a configuration is terminal.

The synthesis framework presented here provides a systematic way to *construct* the ranking ($\leq_a$) or the environment predicates (is-dead, is-solved) from observations, yielding verified CoindHomo instances or verified EC instantiations.

3 The Predicate DSL: Programs over Features

The core abstraction is a domain-specific language for Boolean predicates. The key design choice is *parametricity*: the DSL is parameterized by an abstract carrier type and feature set, enabling each Environment Class to provide its own feature vocabulary while sharing the synthesis theory.

```
{-# OPTIONS --safe --guardedness #-}

module CSHRLSynthesis where

open import Data.List using (List; []; ::; map; foldr; _++_; concatMap; length)
open import Data.Nat using (ℕ; zero; suc; _⊔_; _≤_; z≤n; s≤s)
open import Data.Nat.Properties using (≤-refl)
open import Data.Product using (_×_; _,_; proj₁; proj₂; ∃-syntax)
open import Relation.Binary.PropositionalEquality
  using (_≡_; refl; sym; trans; cong; cong₂; subst)
open import Data.Bool using (Bool; true; false; if_then_else_; _∧_; _∨_; not)
open import Data.Unit using (⊤; tt)
open import Data.Empty using (⊥; ⊥-elim)
open import Relation.Nullary using (Dec; yes; no; ¬_)
open import Data.Maybe using (Maybe; just; nothing)
```

The DSL is a parameterized module:

```
module PredicateDSL
  (Carrier : Set)
  (Feature : Set)
  (eval-feature : Feature  $\rightarrow$  Carrier  $\rightarrow$  Bool)
  where
```

Carrier is the type being classified (states, configurations, etc.), Feature is the vocabulary of observable properties, and eval - feature extracts a Boolean feature value from a carrier.

3.1 Syntax: Predicate Programs

A PredProg term is a Boolean combination of feature tests:

```
data PredProg : Set where
  feat   : Feature  $\rightarrow$  PredProg
  _∧p_   : PredProg  $\rightarrow$  PredProg  $\rightarrow$  PredProg
  _∨p_   : PredProg  $\rightarrow$  PredProg  $\rightarrow$  PredProg
  ¬p_    : PredProg  $\rightarrow$  PredProg
  truep  : PredProg
```

```

falsep : PredProg

infixr 6 _^p_
infixr 5 _vp_

```

The constructors mirror propositional logic: `feat f` tests a single feature, `_^p_` and `_vp_` combine subprograms, `¬p_` negates, and `truep/falsep` are constants.

3.2 Semantics: Evaluation

Evaluation maps a predicate program and a carrier to a Boolean:

```

eval : PredProg → Carrier → Bool
eval (feat f) c = eval-feature f c
eval (p1 ^p p2) c = eval p1 c ∧ eval p2 c
eval (p1 vp p2) c = eval p1 c ∨ eval p2 c
eval (¬p p) c = not (eval p c)
eval truep _ = true
eval falsep _ = false

```

Remark 1 (Expressiveness). At depth 0, `PredProg` contains atoms: `truep`, `falsep`, and `feat f` for each feature f . Each level of Boolean operators (depth $d+1$) exponentially expands the space. For n features, depth 0 gives $n+2$ candidates; depth 1 adds negations, conjunctions, and disjunctions, yielding $O(n^2)$ candidates. This is sufficient for many domains: the 4-Queens is-dead predicate is a single disjunction of 6 attack features (depth 1), and the 8-Queens version uses 28 features at the same depth.

3.3 Feature Equivalence

Two carriers are *feature-equivalent* with respect to a predicate program if they agree on every feature the program inspects:

```

FeatureEquiv : PredProg → Carrier → Carrier → Set
FeatureEquiv (feat f) c1 c2 = eval-feature f c1 == eval-feature f c2
FeatureEquiv (p1 ^p p2) c1 c2 = FeatureEquiv p1 c1 c2 × FeatureEquiv p2 c1 c2
FeatureEquiv (p1 vp p2) c1 c2 = FeatureEquiv p1 c1 c2 × FeatureEquiv p2 c1 c2
FeatureEquiv (¬p p) c1 c2 = FeatureEquiv p c1 c2
FeatureEquiv truep _ _ = T
FeatureEquiv falsep _ _ = T

```

`FeatureEquiv` is defined *structurally* on the program, not on the carrier type. This means it captures exactly the features *relevant to this particular predicate*, not all possible features.

```

feat-equiv-refl : ∀ p c → FeatureEquiv p c c
feat-equiv-refl (feat f) c = refl
feat-equiv-refl (p1 ^p p2) c = feat-equiv-refl p1 c , feat-equiv-refl p2 c
feat-equiv-refl (p1 vp p2) c = feat-equiv-refl p1 c , feat-equiv-refl p2 c
feat-equiv-refl (¬p p) c = feat-equiv-refl p c
feat-equiv-refl truep _ = tt
feat-equiv-refl falsep _ = tt

feat-equiv-sym : ∀ p c1 c2 → FeatureEquiv p c1 c2 → FeatureEquiv p c2 c1
feat-equiv-sym (feat f) c1 c2 eq = sym eq
feat-equiv-sym (p1 ^p p2) c1 c2 (e1 , e2) =
  feat-equiv-sym p1 c1 c2 e1 , feat-equiv-sym p2 c1 c2 e2
feat-equiv-sym (p1 vp p2) c1 c2 (e1 , e2) =
  feat-equiv-sym p1 c1 c2 e1 , feat-equiv-sym p2 c1 c2 e2
feat-equiv-sym (¬p p) c1 c2 eq = feat-equiv-sym p c1 c2 eq
feat-equiv-sym truep _ _ = tt
feat-equiv-sym falsep _ _ = tt

```

3.4 The Propagation Theorem

The central result of the DSL: feature-equivalent carriers are *indistinguishable* to any predicate program.

Theorem 1 (Propagation). *For any predicate program p and carriers c_1, c_2 :*

$$\text{FeatureEquiv } p \ c_1 \ c_2 \implies \text{eval } p \ c_1 \equiv \text{eval } p \ c_2$$

```

propagation : ∀ (p : PredProg) (c1 c2 : Carrier) →
  FeatureEquiv p c1 c2 →
  eval p c1 ≡ eval p c2
propagation (feat f) c1 c2 eq = eq
propagation (p1 ∧ p2) c1 c2 (e1, e2) =
  cong2 _∧_ (propagation p1 c1 c2 e1) (propagation p2 c1 c2 e2)
propagation (p1 ∨ p2) c1 c2 (e1, e2) =
  cong2 _∨_ (propagation p1 c1 c2 e1) (propagation p2 c1 c2 e2)
propagation (¬p) c1 c2 eq =
  cong not (propagation p c1 c2 eq)
propagation truep _ _ _ = refl
propagation falsep _ _ _ = refl

```

Remark 2 (Why This Matters). The Propagation Theorem is the formal foundation for *generalization* in the synthesis framework. When we observe that a predicate evaluates to true at carrier c_1 , we automatically know it evaluates to true at *every* feature-equivalent carrier. This is not an assumption—it is a *theorem*, proven by structural induction on the predicate program.

In the CSHRL context, this means:

- For FMDPs: states with the same features get the same ranking, so one observation propagates to the entire equivalence class.
- For CPMDPs: board configurations with the same attack pattern are classified identically by the synthesized is-dead predicate.

4 CEGIS: Synthesis from Observations

With the DSL defined, we now present the synthesis algorithm. CEGIS (Counterexample-Guided Inductive Synthesis) maintains a *version space*—the set of candidate predicate programs consistent with all observations so far—and refines it as new observations arrive.

```

module CEGIS
  (all-features : List Feature)
  where

  bfilter : ∀ {A : Set} → (A → Bool) → List A → List A
  bfilter f [] = []
  bfilter f (x :: xs) with f x
  ... | true = x :: bfilter f xs
  ... | false = bfilter f xs

  _=b_ : Bool → Bool → Bool
  true =b true = true
  false =b false = true
  _ =b _ = false

  =b-refl : ∀ b → (b =b b) ≡ true
  =b-refl true = refl
  =b-refl false = refl

  =b-sound : ∀ b1 b2 → (b1 =b b2) ≡ true → b1 ≡ b2
  =b-sound true true _ = refl
  =b-sound false false _ = refl

```


4.1 Enumeration

All PredProg terms up to a bounded depth are enumerated:

```
atoms : List PredProg
atoms = truep :: falsep :: map feat all-features

extend : List PredProg → List PredProg
extend prev =
  prev
  ++ map ¬p_ prev
  ++ concatMap (λ p → map (p ∧ p_) prev) prev
  ++ concatMap (λ p → map (p ∨ p_) prev) prev

enumerate : ℕ → List PredProg
enumerate zero = atoms
enumerate (suc d) = extend (enumerate d)
```

4.2 Observations and Consistency

An observation is a pair (c, b) : carrier c should evaluate to b . A candidate is *consistent* if it agrees:

```
PredObs : Set
PredObs = Carrier × Bool

consistent : PredProg → PredObs → Bool
consistent p (c , b) = eval p c ==b b
```

4.3 Version Space Refinement

The version space is the list of consistent candidates. Refinement filters by a new observation:

```
VersionSpace : Set
VersionSpace = List PredProg

initial-vs : ℕ → VersionSpace
initial-vs = enumerate

refine : VersionSpace → PredObs → VersionSpace
refine vs ob = bfilter (λ p → consistent p ob) vs

refine-all : VersionSpace → List PredObs → VersionSpace
refine-all vs [] = vs
refine-all vs (ob :: obs) = refine-all (refine vs ob) obs

cegis-step : VersionSpace → PredObs → VersionSpace
cegis-step = refine

cegis-loop : VersionSpace → List PredObs → VersionSpace
cegis-loop = refine-all
```

4.4 Property 1: Monotonic Convergence

The version space never grows—each observation can only eliminate candidates:

```

bfilter-length : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  length (bfilter f xs) ≤ length xs
bfilter-length f [] = z≤n
bfilter-length f (x :: xs) with f x
... | true = s≤s (bfilter-length f xs)
... | false = ≤-step (bfilter-length f xs)
where
  ≤-step : ∀ {m n : ℕ} → m ≤ n → m ≤ suc n
  ≤-step z≤n = z≤n
  ≤-step (s≤s p) = s≤s (≤-step p)

vs-shrinks : ∀ vs ob →
  length (refine vs ob) ≤ length vs
vs-shrinks vs ob = bfilter-length (λ p → consistent p ob) vs

```

Theorem 2 (Bounded Convergence). *After processing n observations from the initial version space, the number of remaining candidates is at most the initial count:*

```

cegis-bounded : ∀ vs obs →
  length (cegis-loop vs obs) ≤ length vs
cegis-bounded vs [] = ≤-refl
cegis-bounded vs (ob :: obs) =
  ≤-trans (cegis-bounded (refine vs ob) obs)
    (vs-shrinks vs ob)
where
  ≤-trans : ∀ {a b c : ℕ} → a ≤ b → b ≤ c → a ≤ c
  ≤-trans z≤n _ = z≤n
  ≤-trans (s≤s p) (s≤s q) = s≤s (≤-trans p q)

```

4.5 Property 2: Soundness

The correct predicate is never eliminated by valid observations:

```

consistent-correct : ∀ p c b →
  eval p c ≡ b →
  consistent p (c , b) ≡ true
consistent-correct p c b eq rewrite eq = =b-refl b

```

4.6 Property 3: Propagation Acceleration

Feature-equivalent carriers eliminate exactly the same candidates:

```

propagation-acceleration : ∀ (p : PredProg) (c1 c2 : Carrier) (b : Bool) →
  FeatureEquiv p c1 c2 →
  consistent p (c1 , b) ≡ consistent p (c2 , b)
propagation-acceleration p c1 c2 b equiv =
  cong (λ v → v =b b) (propagation p c1 c2 equiv)

```

This is the bridge between the Propagation Theorem (§3) and the synthesis algorithm: because feature-equivalent carriers produce identical eval results, they produce identical consistent results, and therefore identical version-space refinement.

4.7 Property 4: Strict Convergence

An observation is *informative* if it eliminates at least one candidate. Informative observations cause *strict* shrinkage:

```

data _∈_ {A : Set} : A → List A → Set where
  here : ∀ {x xs} → x ∈I (x :: xs)
  there : ∀ {x y xs} → x ∈I xs → x ∈I (y :: xs)

bfilter-strict : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∃[ x ] (x ∈I xs × f x ≡ false) →
  suc (length (bfilter f xs)) ≤ length xs
bfilter-strict f (x :: xs) (.x , here , fx≡f) with f x
... | true with () ← fx≡f
... | false = s≤s (bfilter-length f xs)
bfilter-strict f (x :: xs) (z , there mem , fz≡f) with f x
... | true = s≤s (bfilter-strict f xs (z , mem , fz≡f))
... | false = ≤-step' (bfilter-strict f xs (z , mem , fz≡f))
where
  ≤-step' : ∀ {m n : ℕ} → suc m ≤ n → suc m ≤ suc n
  ≤-step' (s≤s p) = s≤s (≤-step'' p)
  where
    ≤-step'' : ∀ {a b : ℕ} → a ≤ b → a ≤ suc b
    ≤-step'' z≤n = z≤n
    ≤-step'' (s≤s q) = s≤s (≤-step'' q)

Informative : VersionSpace → PredObs → Set
Informative vs ob = ∃[ p ] (p ∈I vs × consistent p ob ≡ false)

strict-shrink : ∀ vs ob →
  Informative vs ob →
  suc (length (refine vs ob)) ≤ length vs
strict-shrink vs ob info =
  bfilter-strict (λ p → consistent p ob) vs info

```

Theorem 3 (Strict Convergence). *Each informative observation reduces the version space by at least one candidate. Combined with bounded convergence, this means CEGIS terminates after at most $|VS_0|$ informative observations.*

5 Propagation Quantification: The Central Contribution

We now prove three theorems that formalize how the Propagation Theorem (1) reduces the number of observations needed for synthesis. These results are the main theoretical contribution of this paper.

The key insight: because predicate programs operate on features rather than raw carriers, the effective state space for synthesis is the set of *feature equivalence classes*, not the set of carriers. For a domain with $|C|$ carriers but only $|C/\sim|$ distinct feature patterns, synthesis needs at most $|C/\sim|$ observations.

```

non-informative-id : ∀ vs ob →
  (∀ p → p ∈I vs → consistent p ob ≡ true) →
  refine vs ob ≡ vs
non-informative-id [] ob _ = refl
non-informative-id (p :: vs) ob all-con with consistent p ob
| all-con p here
... | true | refl =
  cong (p :: _) (non-informative-id vs ob
    (λ q mem → all-con q (there mem)))

```

5.1 Full Feature Agreement

Two carriers agree on *every* feature (not just those in a specific program):

```

AllFeatAgree : Carrier → Carrier → Set
AllFeatAgree c1 c2 = ∀ (f : Feature) →
  eval-feature f c1 ≡ eval-feature f c2

all-agree→feat-equiv : ∀ c1 c2 →
  AllFeatAgree c1 c2 →
  ∀ p → FeatureEquiv p c1 c2
all-agree→feat-equiv c1 c2 afa (feat f) = afa f
all-agree→feat-equiv c1 c2 afa (p1 ∧ p2) =
  all-agree→feat-equiv c1 c2 afa p1 ,
  all-agree→feat-equiv c1 c2 afa p2
all-agree→feat-equiv c1 c2 afa (p1 ∨ p2) =
  all-agree→feat-equiv c1 c2 afa p1 ,
  all-agree→feat-equiv c1 c2 afa p2
all-agree→feat-equiv c1 c2 afa (¬p) =
  all-agree→feat-equiv c1 c2 afa p
all-agree→feat-equiv c1 c2 afa truep = tt
all-agree→feat-equiv c1 c2 afa falsep = tt

all-feat-agree-sym : ∀ c1 c2 →
  AllFeatAgree c1 c2 → AllFeatAgree c2 c1
all-feat-agree-sym c1 c2 afa f = sym (afa f)

```

5.2 Auxiliary Lemmas

Two properties of bfilter that drive the main theorems:

```

bfilter-ext : ∀ {A : Set} (f g : A → Bool) (xs : List A) →
  (∀ x → f x ≡ g x) →
  bfilter f xs ≡ bfilter g xs
bfilter-ext f g [] h = refl
bfilter-ext f g (x :: xs) h with f x | g x | h x
... | true | .true | refl = cong (x ::_) (bfilter-ext f g xs h)
... | false | .false | refl = bfilter-ext f g xs h

bfilter-absorb : ∀ {A : Set} (f g : A → Bool) (xs : List A) →
  (∀ x → f x ≡ true → g x ≡ true) →
  bfilter g (bfilter f xs) ≡ bfilter f xs
bfilter-absorb f g [] h = refl
bfilter-absorb f g (x :: xs) h with f x in eq-f
... | false = bfilter-absorb f g xs h
... | true with g x | h x eq-f
... | true | refl = cong (x ::_) (bfilter-absorb f g xs h)

consistent-equiv : ∀ p c1 c2 b →
  AllFeatAgree c1 c2 →
  consistent p (c1 , b) ≡ consistent p (c2 , b)
consistent-equiv p c1 c2 b afa =
  propagation-acceleration p c1 c2 b
  (all-agree→feat-equiv c1 c2 afa p)

consistent-implies : ∀ c1 c2 b →
  AllFeatAgree c1 c2 →
  ∀ p → consistent p (c1 , b) ≡ true →
  consistent p (c2 , b) ≡ true
consistent-implies c1 c2 b afa p h =
  trans (sym (consistent-equiv p c1 c2 b afa)) h

```

5.3 Theorem 1: Refine-Equiv

Theorem 4 (Refine-Equiv). *Feature-equivalent carriers produce identical version-space refinement. Observing at c_1 eliminates exactly the same candidates as observing at c_2 :*

$$\text{AllFeatAgree } c_1 \ c_2 \implies \text{refine vs } (c_1, b) \equiv \text{refine vs } (c_2, b)$$

```

refine-equiv : ∀ vs c1 c2 b →
  AllFeatAgree c1 c2 →
  refine vs (c1, b) ≡ refine vs (c2, b)
refine-equiv vs c1 c2 b afa =
  bfilter-ext
    (λ p → consistent p (c1, b))
    (λ p → consistent p (c2, b))
  vs
  (λ p → consistent-equiv p c1 c2 b afa)

```

Remark 3. This result is stronger than “the same number of candidates are eliminated.” It asserts that the *exact same list* of candidates survives—not just the same count, but the same elements in the same order. This is propositional equality on lists, the strongest form of agreement.

5.4 Theorem 2: Observation Subsumption

Theorem 5 (Observation-Subsumption). *After observing at c_1 , a second observation at any feature-equivalent c_2 is provably redundant—the version space is unchanged:*

$$\text{AllFeatAgree } c_1 \ c_2 \implies \text{refine (refine vs } (c_1, b)) (c_2, b) \equiv \text{refine vs } (c_1, b)$$

```

refine-absorb : ∀ vs c1 c2 b →
  AllFeatAgree c1 c2 →
  refine (refine vs (c1, b)) (c2, b) ≡ refine vs (c1, b)
refine-absorb vs c1 c2 b afa =
  bfilter-absorb
    (λ p → consistent p (c1, b))
    (λ p → consistent p (c2, b))
  vs
  (consistent-implies c1 c2 b afa)

observation-subsumption : ∀ vs c1 c2 b →
  AllFeatAgree c1 c2 →
  refine-all vs ((c1, b) :: (c2, b) :: [])
  ≡ refine vs (c1, b)
observation-subsumption vs c1 c2 b afa =
  refine-absorb vs c1 c2 b afa

```

Every candidate that survived the first observation automatically survives the second. No new information is gained—the observation is *subsumed*.

5.5 Theorem 3: Exploration Dissolution

Theorem 6 (Exploration-Dissolution). *An entire equivalence class of carriers is handled by a single observation at any representative c_0 . After that observation, all further observations at equivalent carriers leave the version space unchanged:*

```

AllEquiv : Carrier → List Carrier → Set
AllEquiv c0 [] = T

```

```

AllEquiv  $c_0$  ( $c :: cs$ ) = AllFeatAgree  $c_0$   $c$   $\times$  AllEquiv  $c_0$   $cs$ 

exploration-dissolution :  $\forall$   $vs$   $c_0$   $b$  ( $cs$  : List Carrier)  $\rightarrow$ 
  AllEquiv  $c_0$   $cs$   $\rightarrow$ 
  cegis-loop (refine  $vs$  ( $c_0$ ,  $b$ )) (map ( $\lambda$   $c \rightarrow$  ( $c$ ,  $b$ ))  $cs$ )
     $\equiv$  refine  $vs$  ( $c_0$ ,  $b$ )
exploration-dissolution  $vs$   $c_0$   $b$  []  $\_ =$  refl
exploration-dissolution  $vs$   $c_0$   $b$  ( $c :: cs$ ) ( $afa$ ,  $rest$ ) =
  let  $step_1 =$  refine-absorb  $vs$   $c_0$   $c$   $b$   $afa$ 
  in trans
    (cong ( $\lambda$   $vs' \rightarrow$  cegis-loop  $vs'$  (map ( $\lambda$   $c' \rightarrow$  ( $c'$ ,  $b$ ))  $cs$ ))
       $step_1$ )
    (exploration-dissolution  $vs$   $c_0$   $b$   $cs$   $rest$ )

```

Corollary 7 (Dissolution preserves VS size). *Since the version space is unchanged, its size is trivially preserved:*

```

dissolution-size :  $\forall$   $vs$   $c_0$   $b$  ( $cs$  : List Carrier)  $\rightarrow$ 
  AllEquiv  $c_0$   $cs$   $\rightarrow$ 
  length (cegis-loop (refine  $vs$  ( $c_0$ ,  $b$ ))
    (map ( $\lambda$   $c \rightarrow$  ( $c$ ,  $b$ ))  $cs$ ))
     $\equiv$  length (refine  $vs$  ( $c_0$ ,  $b$ ))
dissolution-size  $vs$   $c_0$   $b$   $cs$   $afa =$ 
  cong length (exploration-dissolution  $vs$   $c_0$   $b$   $cs$   $afa$ )

```

5.6 Corollary: Dissolution + Strict Convergence

The combination of dissolution and strict convergence gives the tightest bound:

Corollary 8 (Dissolution-Strict). *Within an equivalence class, only the first observation can be informative. All subsequent are provably redundant. Total informative observations $\leq |C/\sim|$:*

```

dissolution-strict :  $\forall$   $vs$   $c_0$   $b$  ( $cs$  : List Carrier)  $\rightarrow$ 
  AllEquiv  $c_0$   $cs$   $\rightarrow$ 
  Informative  $vs$  ( $c_0$ ,  $b$ )  $\rightarrow$ 
  suc (length (cegis-loop (refine  $vs$  ( $c_0$ ,  $b$ ))
    (map ( $\lambda$   $c \rightarrow$  ( $c$ ,  $b$ ))  $cs$ )))
     $\leq$  length  $vs$ 
dissolution-strict  $vs$   $c_0$   $b$   $cs$   $afa$   $info =$ 
  subst ( $\lambda$   $n \rightarrow$  suc  $n \leq$  length  $vs$ )
    (sym (cong length (exploration-dissolution  $vs$   $c_0$   $b$   $cs$   $afa$ )))
    (strict-shrink  $vs$  ( $c_0$ ,  $b$ )  $info$ )

```

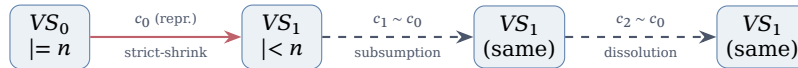


Figure 2: Exploration dissolution in action. The first observation at representative c_0 strictly shrinks the version space. Subsequent observations at equivalent carriers $c_1, c_2, \dots$ are no-ops—proven by Observation-Subsumption.

Remark 4 (Complexity Reduction). Consider the 8-Queens domain. There are $8^8 = 16,777,216$ possible board configurations, but the number of distinct feature patterns (attack + length patterns) is much smaller. Propagation quantification proves that synthesis needs to observe only one configuration per pattern class, reducing the effective exploration space by orders of magnitude.

6 Quotient Structure and Exploration Completeness

The propagation quantification results of §5 show that feature-equivalent carriers are redundant *pairwise*. We now lift this to the global level: `AllFeatAgree` is an equivalence relation, and after observing one representative per equivalence class, *no further observation can refine the version space*.

6.1 AllFeatAgree Is an Equivalence Relation

Reflexivity and symmetry are immediate; transitivity composes pointwise:

```
AFA-refl : ∀ c → AllFeatAgree c c
AFA-refl c f = refl

AFA-trans : ∀ C1 C2 C3 →
  AllFeatAgree C1 C2 → AllFeatAgree C2 C3 → AllFeatAgree C1 C3
AFA-trans C1 C2 C3 afa12 afa23 f = trans (afa12 f) (afa23 f)
```

Together with the existing `all-feat-agree-sym`, this establishes `AllFeatAgree` as an equivalence relation, partitioning the carrier space into classes $C/\sim$.

6.2 Auxiliary: Soundness and Monotonicity

Two helper lemmas drive the completeness proof. First, elements surviving a `bfilter` satisfy the predicate—this is the “soundness” direction of filtering:

```
bfilter-sound : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∀ x → x ∈ bfilter f xs → f x = true
bfilter-sound f (y :: xs) x mem with f y in eq
bfilter-sound f (y :: xs) .y here      | true = eq
bfilter-sound f (y :: xs) x (there m) | true = bfilter-sound f xs x m
bfilter-sound f (y :: xs) x mem      | false = bfilter-sound f xs x mem
```

Second, `cegis-loop` only removes elements—its output is a sublist of its input:

```
private
bfilter-⊆ : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∀ x → x ∈ bfilter f xs → x ∈ xs
bfilter-⊆ f (y :: xs) x mem with f y
bfilter-⊆ f (y :: xs) .y here      | true = here
bfilter-⊆ f (y :: xs) x (there m) | true = there (bfilter-⊆ f xs x m)
bfilter-⊆ f (y :: xs) x mem      | false = there (bfilter-⊆ f xs x mem)

cegis-loop-⊆ : ∀ vs obs →
  ∀ x → x ∈ cegis-loop vs obs → x ∈ vs
cegis-loop-⊆ vs [] x mem = mem
cegis-loop-⊆ vs (ob :: obs) x mem =
  bfilter-⊆ (λ p → consistent p ob) vs x
  (cegis-loop-⊆ (refine vs ob) obs x mem)
```

6.3 Subsumption Stability

The key technical lemma: subsumption is *stable under further refinement*. After refining by representative r and then processing additional observations, refining by any $c \sim r$ is still a no-op:

Theorem 9 (Refine-After-CEGIS). *If $r \sim c$ (`AllFeatAgree`), then after refining by (r, b) and processing further observations, the version space is already consistent with (c, b) :*

```

refine-after-cegis :  $\forall$  vs r c b (obs : List PredObs)  $\rightarrow$ 
  AllFeatAgree r c  $\rightarrow$ 
  refine (cegis-loop (refine vs (r , b)) obs) (c , b)
     $\equiv$  cegis-loop (refine vs (r , b)) obs
refine-after-cegis vs r c b obs afa =
  non-informative-id
  (cegis-loop (refine vs (r , b)) obs) (c , b)
  ( $\lambda$  p mem  $\rightarrow$ 
    consistent-implies r c b afa p
    (bfilter-sound ( $\lambda$  q  $\rightarrow$  consistent q (r , b)) vs p
    (cegis-loop- $\sqsubseteq$  (refine vs (r , b)) obs p mem)))

```

The proof chains three facts: (1) every surviving candidate p is in the original refined VS (by `cegis-loop- $\sqsubseteq$`); (2) it satisfies consistent at (r, b) (by `bfilter-sound`); (3) it therefore satisfies consistent at (c, b) (by `consistent-implies` from feature equivalence).

6.4 Exploration Completeness

Theorem 10 (Exploration-Complete). *After processing a list of representatives (one per equivalence class), any carrier equivalent to some representative is provably redundant:*

$$r \in \text{reps} \wedge r \sim c \implies \text{refine}(\text{cegis-loop vs reps})(c, b) \equiv \text{cegis-loop vs reps}$$

```

exploration-complete :  $\forall$  vs (reps : List Carrier) b c  $\rightarrow$ 
  ( $\exists$  [ r ] (r  $\in$  reps  $\times$  AllFeatAgree r c))  $\rightarrow$ 
  refine (cegis-loop vs (map ( $\lambda$  r  $\rightarrow$  (r , b)) reps)) (c , b)
     $\equiv$  cegis-loop vs (map ( $\lambda$  r  $\rightarrow$  (r , b)) reps)
exploration-complete vs (r :: reps) b c (.r , here , afa) =
  refine-after-cegis vs r c b (map ( $\lambda$  r'  $\rightarrow$  (r' , b)) reps) afa
exploration-complete vs (r' :: reps) b c (r , there mem , afa) =
  exploration-complete (refine vs (r' , b)) reps b c (r , mem , afa)

```

The proof splits on where representative r appears. If r is the head, `refine-after-cegis` applies directly. If r is in the tail, the inductive hypothesis applies to the version space already refined by the head.

Corollary 11 (Dissolution Is All You Need). *Once every equivalence class has a representative in the observation list, no carrier anywhere in C can produce an informative observation. The version space is fully refined—further sampling is provably wasteful.*

Remark 5 (Tight Convergence). Combined with `Strict-Shrink`, this gives the tight bound: the number of informative observations is at most $|C/\sim|$, the number of equivalence classes. Each class contributes at most one informative observation (at its representative); all subsequent observations within the class are no-ops by `Exploration-Complete`. For 8-Queens with attack features, $|C/\sim| \ll 8^8$.

7 Temporal Propagation: Coinductive Chains

Propagation as presented so far is *spatial*: it transfers observations across carriers at the same point in time. In MDPs, there is also a *temporal* dimension: states evolve through transitions. If the transition function preserves feature equivalence, then observations propagate not just across equivalent carriers but along *entire trajectories*.

Definition 2 (Feature Preservation). A transition function $\text{step} : C \rightarrow A \rightarrow C$ **preserves features** if:

$$c_1 \sim c_2 \implies \text{step}(c_1, a) \sim \text{step}(c_2, a) \quad \text{for all actions } a$$

This is a natural property: if two states look the same (same features), acting on them produces states that also look the same.

```

module TemporalPropagation
  {Act : Set}
  (step-carrier : Carrier → Act → Carrier)
  (step-preserves-feat : ∀ c1 c2 a →
    AllFeatAgree c1 c2 →
    AllFeatAgree (step-carrier c1 a) (step-carrier c2 a))
  where

  itert : Carrier → List Act → Carrier
  itert c [] = c
  itert c (a :: as) = itert (step-carrier c a) as

  trajectory : Carrier → Bool → List Act → List PredObs
  trajectory c b [] = (c , b) :: []
  trajectory c b (a :: as) = (c , b) :: trajectory (step-carrier c a) b as

```

7.1 Temporal Feature Equivalence

Feature equivalence composes through arbitrary action sequences:

Theorem 12 (Temporal-Feat-Equiv). *If $c_1 \sim c_2$ and step preserves features, then after any action sequence $[a_1, \dots, a_n]$, the resulting states are still equivalent:*

```

temporal-feat-equiv : ∀ c1 c2 (as : List Act) →
  AllFeatAgree c1 c2 →
  AllFeatAgree (itert c1 as) (itert c2 as)
temporal-feat-equiv c1 c2 [] afa = afa
temporal-feat-equiv c1 c2 (a :: as) afa =
  temporal-feat-equiv
    (step-carrier c1 a) (step-carrier c2 a) as
    (step-preserves-feat c1 c2 a afa)

```

7.2 Trajectory Refine-Equiv

Trajectories from feature-equivalent carriers produce *identical* version-space refinement:

Theorem 13 (Trajectory-Refine-Equiv). *For any action sequence, observing a trajectory from c_1 refines the version space identically to observing the trajectory from c_2 :*

```

trajectory-refine-equiv : ∀ vs c1 c2 b (as : List Act) →
  AllFeatAgree c1 c2 →
  cegis-loop vs (trajectory c1 b as)
  ≡ cegis-loop vs (trajectory c2 b as)
trajectory-refine-equiv vs c1 c2 b [] afa =
  refine-equiv vs c1 c2 b afa
trajectory-refine-equiv vs c1 c2 b (a :: as) afa =
  trans
    (cong (λ vs' → cegis-loop vs'
      (trajectory (step-carrier c1 a) b as))
      (refine-equiv vs c1 c2 b afa))
    (trajectory-refine-equiv (refine vs (c2 , b))
      (step-carrier c1 a) (step-carrier c2 a) b as
      (step-preserves-feat c1 c2 a afa))

```

The proof inductively combines spatial propagation at each step (refine-equiv) with temporal propagation at the successors (the inductive hypothesis with preserved feature equivalence).

7.3 Trajectory Dissolution

The central result: after observing an entire trajectory from c_1 , the entire trajectory from any equivalent c_2 is *completely subsumed*:

Theorem 14 (Trajectory-Dissolution). *One trajectory observation covers entire families of equivalent trajectories:*

$$c_1 \sim c_2 \implies \text{cegis-loop} (\text{cegis-loop vs traj}(c_1)) \text{ traj}(c_2) \equiv \text{cegis-loop vs traj}(c_1)$$

```

trajectory-dissolution :  $\forall$  vs  $c_1 c_2 b$  (as : List Act)  $\rightarrow$ 
  AllFeatAgree  $c_1 c_2 \rightarrow$ 
  cegis-loop (cegis-loop vs (trajectory  $c_1 b$  as))
    (trajectory  $c_2 b$  as)
   $\equiv$  cegis-loop vs (trajectory  $c_1 b$  as)
trajectory-dissolution vs  $c_1 c_2 b []$  afa =
  refine-absorb vs  $c_1 c_2 b$  afa
trajectory-dissolution vs  $c_1 c_2 b (a :: as)$  afa =
  trans
    (cong ( $\lambda v \rightarrow$  cegis-loop v
      (trajectory (step-carrier  $c_2 a$ ) b as))
      (refine-after-cegis vs  $c_1 c_2 b$ 
        (trajectory (step-carrier  $c_1 a$ ) b as) afa))
    (trajectory-dissolution (refine vs ( $c_1, b$ ))
      (step-carrier  $c_1 a$ ) (step-carrier  $c_2 a$ ) b as
      (step-preserves-feat  $c_1 c_2 a$  afa))

```

The inductive step uses *refine-after-cegis* (subsumption stability from §6) to show that the first observation (c_2, b) is absorbed, and then the inductive hypothesis with preserved feature equivalence to show the rest of the trajectory is absorbed.

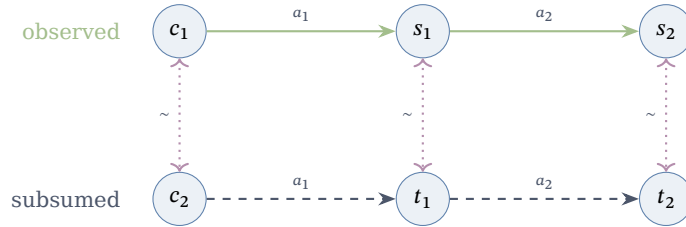


Figure 3: Temporal dissolution. Feature preservation ensures that equivalence ($\sim$) is maintained at every step. Observing the trajectory from c_1 (solid) makes the trajectory from c_2 (dashed) completely redundant.

Remark 6 (Exponential Reduction). In an MDP with k equivalence classes, spatial dissolution handles k observations. With temporal propagation over trajectories of length n , the total observations covered grows to $k \times n$ —but only k trajectory observations are needed. For long-horizon tasks, this is an exponential reduction: one root observation at a representative covers the entire future of every equivalent state. This mirrors CSHRL’s coinductive unfolding: just as the CoindHomo guarantees action-value dominance at *every* future timestep, temporal propagation guarantees observation equivalence at *every* future timestep.

8 Feature Sufficiency: From Representatives to Global Correctness

The preceding sections establish that feature-equivalent carriers are *indistinguishable* to the DSL, that dissolution reduces observations to $|C/\sim|$, and that temporal propagation

extends this across trajectories. But a fundamental question remains: *when is the CEGIS result actually correct?*

Exploration completeness (Section 6) guarantees that no further refinement is possible. But “no refinement possible” and “correct answer” are logically distinct claims. The version space might stabilize on programs that all agree with the target on observed carriers yet disagree elsewhere.

Feature sufficiency bridges this gap. When the target function *respects* feature equivalence—meaning $\text{AllFeatAgree } c_1 \ c_2$ implies identical target values—then correctness at representatives *propagates* to correctness everywhere. This yields the crown jewel of the framework: the **Feature Sufficiency Theorem**.

Definition 3 (Feature Respect). A target function $t : \text{Carrier} \rightarrow \text{Bool}$ *respects* the feature set when feature-equivalent carriers receive identical values:

```
FeatureRespects : (Carrier → Bool) → Set
FeatureRespects target = ∀ 1c 2c →
  AllFeatAgree 1c 2c → target 1c ≡ target 2c
```

Definition 4 (Matching). A predicate program p *matches* a target t at carrier c when $\text{eval } p \ c \equiv t \ c$. It matches *everywhere* when this holds for all carriers.

```
Matches      : PredProg → (Carrier → Bool) → Carrier → Set
Matches      p target c = eval p c ≡ target c

MatchesAll   : PredProg → (Carrier → Bool) → Set
MatchesAll   p target = ∀ c → Matches p target c
```

Theorem 15 (Match Propagation). *If a program matches the target at c_1 , and the target respects features, then the program matches at every feature-equivalent c_2 .*

The proof chains propagation (for the program side) with feature respect (for the target side):

```
match-propagates : ∀ p target 1c 2c →
  AllFeatAgree 1c 2c →
  FeatureRespects target →
  Matches p target 1c →
  Matches p target 2c
match-propagates p target 1c 2c afa respects 1match =
  trans
    (sym (propagation p 1c 2c
      (all→agreefeat-equiv 1c 2c afa p)))
    (trans 1match (respects 1c 2c afa))
```

Definition 5 (Covers). A list of representatives *covers* the carrier space when every carrier is feature-equivalent to some representative.

```
Covers : List Carrier → Set
Covers reps = ∀ c →
  ∃[ r ] (r ∈ reps × AllFeatAgree r c)
```

Theorem 16 (Generalization). *If a program matches the target at all representatives, and the representatives cover the carrier space, then the program matches everywhere.*

```
generalization : ∀ p target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  ∀( r → r ∈ reps → Matches p target r ) →
  MatchesAll p target
generalization p target reps respects covers matches-reps c =
  let (r , r-in , afa) = covers c
  in match-propagates p target r c afa
    respects (matches-reps r r-in)
```

The glue between CEGIS and generalization is the *survivor consistency* lemma: every program that survives the CEGIS loop is consistent with all observations that were processed.

```

ceg-s-survivors-consistent : ∀ vs obs →
  ∀ p → p ∈ cegis-loop vs obs →
  ∀ ob → ob ∈ obs → consistent p ob ≡ true

```

The proof proceeds by induction on the observation list: for the head observation, the survivor is in the refined version space (by `ceg-s-loop-⊆`) and hence consistent (by `bfilter-sound`); for the tail, the induction hypothesis applies directly.

Theorem 17 (Feature Sufficiency — Crown Jewel). *With sufficient features (target respects AllFeatAgree) and representative observations (one per equivalence class), every CEGIS survivor is correct on **all** carriers.*

```

sufficient-cegis : ∀ vs target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  ∀ p → p ∈ cegis-loop vs
    (map λ( r → (r , target r)) reps) →
  MatchesAll p target

```

The proof composes three results: `ceg-s-survivors-consistent` (survivors match at observed carriers), `consistent-to-match` (consistency implies matching), and `generalization` (matching at representatives implies matching everywhere).

Remark 7 (What Changes). Without feature sufficiency, Exploration-Complete says “the version space has stabilized.” With feature sufficiency, it says “the version space contains only correct programs.” The stabilization guarantee is upgraded to a *correctness* guarantee—every survivor computes the target function on every carrier, not just the observed ones.

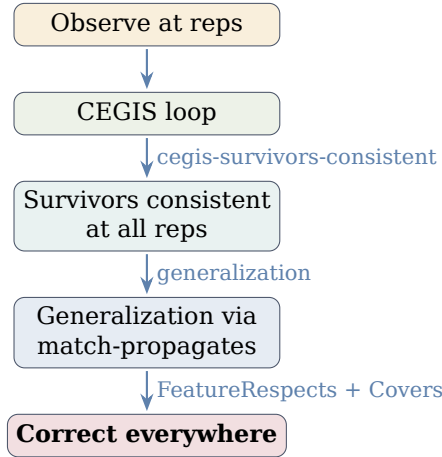


Figure 4: Feature sufficiency proof pipeline. Observing one representative per equivalence class and running CEGIS produces programs that are provably correct on *all* carriers.

9 Multi-Predicate Propagation

Many environments require synthesizing *multiple* predicates simultaneously from the same feature space. In the CPMDP setting, for example, we synthesize both `is-dead` and `is-solved`—two Boolean functions sharing the same carrier type and feature set. This section develops the theory of joint synthesis, where observations for one predicate constrain the other.

9.1 Joint Dissolution

Since both predicates are defined over the same carrier and feature set, a single `AllFeatAgree` proof propagates to both version spaces simultaneously. The `MultiPredicate` module in `Core.agda` is parameterized by two target functions:

```
module MultiPredicate
  (1target 2target : Carrier → Bool) where

  joint-refine : VersionSpace → VersionSpace →
    Carrier → VersionSpace × VersionSpace
  joint-refine 1vs 2vs c =
    (refine 1vs (c , 1target c) ,
     refine 2vs (c , 2target c))
```

One carrier observation simultaneously refines both version spaces. Joint dissolution follows immediately: when $c_1 \sim c_2$, the joint refinement is identical, so one observation handles both predicates for the entire equivalence class.

```
joint-refine-equiv : ∀ 1vs 2vs 1c 2c →
  AllFeatAgree 1c 2c →
  FeatureRespects 1target →
  FeatureRespects 2target →
  joint-refine 1vs 2vs 1c
    ≡ joint-refine 1vs 2vs 2c
```

9.2 Cross-Predicate Entailment

In many domains, predicates are logically related. For Queens, dead states cannot be solved: if `is-dead c = true`, then `is-solved c = false`. This *entailment* gives free observations.

```
Entails~ : Set
Entails~ = ∀ c → 1target c ≡ true →
  2target c ≡ false
```

With entailment, observing `target1 c = true` automatically refines the second version space by `(c , false)`, without evaluating `target2`. This is formalized as `cross-matches-joint`: the cross-refinement equals the full joint refinement.

```
cross-matches-joint : ∀ 1vs 2vs c →
  (ent : Entails~) →
  (t1 : 1target c ≡ true) →
  cross-refine 1vs 2vs c ent t1
    ≡ joint-refine 1vs 2vs
      (c , 1target c , 2target c)
```

9.3 Cross-Dissolution

The multiplicative power emerges when entailment meets dissolution. Observing `target1 c1 = true` with entailment `dead → ¬solved`:

1. Refines `vs1` by `(c1, true)`
2. Refines `vs2` by `(c1, false)` — **free** from entailment
3. Renders all $c_2 \sim c_1$ redundant for **both** version spaces — from dissolution

One observation, two predicates, an entire equivalence class:

```
cross-dissolution : ∀ 1vs 2vs 1c 2c →
  AllFeatAgree 1c 2c →
  Entails~ → 1target 1c ≡ true →
  let 1vs' = refine 1vs (1c , true)
      2vs' = refine 2vs (1c , false)
  in refine 1vs' (2c , true) ≡ 1vs'
    × refine 2vs' (2c , false) ≡ 2vs'
```

9.4 Joint Sufficiency

The feature sufficiency theorem lifts to multiple predicates: with sufficient features for both targets and the same set of representative observations, both synthesized predicates are correct on all carriers.

Theorem 18 (Joint Sufficiency). *With sufficient features for both targets and representative coverage, running CEGIS independently on each target using the same representatives yields programs that are correct everywhere—for both predicates.*

```
joint-sufficient : ∀ 1vs 2vs (reps : List Carrier) →
  FeatureRespects 1target →
  FeatureRespects 2target →
  Covers reps →
  ∀ 1p 2p →
  1p ∈ cegis-loop 1vs
    (map λ( r → (r , 1target r)) reps) →
  2p ∈ cegis-loop 2vs
    (map λ( r → (r , 2target r)) reps) →
  MatchesAll 1p 1target × MatchesAll 2p 2target
```

This directly applies sufficient-cegis (Theorem 17) to each predicate independently—the key insight is that the *same* representatives suffice for both, since both predicates share the same feature space and hence the same equivalence classes.

Remark 8 (Observation Budget). For a single predicate, the observation budget is $|C/\sim|$. For k predicates sharing the same features, the budget remains $|C/\sim|$ (not $k \cdot |C/\sim|$): one set of representative observations suffices for all k predicates. With entailment, the *evaluation* budget drops further, since some target values can be derived without computation. In the Queens setting, this means synthesizing both *is-dead* and *is-solved* requires no more observations than synthesizing either one alone.

10 Online Synthesis: Learning-Synthesis Integration

The preceding sections established CEGIS as a batch algorithm: given a fixed list of observations, the version space is refined to convergence. In practice, observations arrive *incrementally*—a learning loop produces violations and samples one at a time, and the synthesis engine must process them on-the-fly. This section establishes three key results that enable this integration:

1. **Order independence:** the final version space depends only on the *set* of observations, not their order;
2. **Replay redundancy:** re-submitting a previously seen observation is a provable no-op;
3. **Learning Bridge:** a generic module connecting learning feedback to synthesis observations, with a bridge sufficiency theorem that composes learning convergence with synthesis correctness.

10.1 Observation Order Independence

The central structural result is that two successive refinements *commute*: refining by ob_1 then ob_2 yields the same version space as refining by ob_2 then ob_1 .

```
refine-commutes : ∀ vs 1ob 2ob →
  refine (refine vs 1ob) 2ob ≡ refine (refine vs 2ob) 1ob
```

The proof proceeds via an auxiliary *bfilter'* (using *if-then-else* instead of *with*, enabling clean equational reasoning) and a *bfilter-swap* lemma showing that composing two Boolean filters is commutative. The key technical insight: each filter retains exactly those candidates consistent with its observation, and Boolean conjunction is commutative, so the surviving candidates are identical regardless of filter order.

An immediate corollary lifts commutativity to the full CEGIS loop—swapping any two adjacent observations preserves the result:

```

cegis-loop-swap : ∀ vs 1ob 2ob obs →
  cegis-loop vs (1ob :: 2ob :: obs)
    ≡ cegis-loop vs (2ob :: 1ob :: obs)
cegis-loop-swap vs 1ob 2ob obs =
  cong λ( v → cegis-loop v obs) (refine-commutes vs 1ob 2ob)

```

Since any permutation can be decomposed into adjacent transpositions, this implies full permutation invariance: the version space is determined by the *multiset* of observations.

Remark 9 (Significance for Online Learning). Order independence has a profound practical implication: the learning loop can submit observations in *any* order—depth-first, breadth-first, violation-driven, or randomized—and the synthesized result is identical. This means the synthesis engine is insensitive to the exploration strategy, and parallel learners can submit observations asynchronously without coordination.

10.2 Replay Redundancy

In an online setting, the learner may revisit a state, re-evaluate a sample, or replay a checkpoint. The following theorem guarantees that such replays are harmless:

```

replay-redundant : ∀ vs obs ob →
  ob ∈ obs →
  refine (cegis-loop vs obs) ob ≡ cegis-loop vs obs

```

If *ob* was already among the processed observations, re-applying it to the refined version space is a no-op. The proof combines *cegis-survivors-consistent* (every survivor passes all processed observations) with *non-informative-id* (non-informative observations leave the VS unchanged).

Combined with the late-dissolution theorem (*refine-after-cegis*, §6), this establishes a strong idempotence property: in the online stream, *any* observation that is feature-equivalent to a previously processed observation is provably redundant, regardless of how many other observations have been processed in between.

10.3 The Learning Bridge

The generic learning-synthesis bridge connects a learning loop (which produces domain-specific feedback such as violations and trace comparisons) with the synthesis version space. The bridge is parameterized by a mapping from learner feedback to synthesis observations:

```

module LearningBridge
  {Feedback : Set}
  (to-obs : Feedback → PredObs)
  where

```

This abstraction covers all EC instantiations:

- **FMDP**: Feedback is a state-action-action triple with a trace comparison; *to-obs* maps it to a *PredObs* for the *prefer(a,b)* predicate.
- **CPMDP**: Feedback is a configuration evaluation; *to-obs* maps it to a *PredObs* for the target predicate (e.g., *is-dead*).

The bridge defines incremental processing and batch processing, with a key equivalence theorem:

```

bridge-is-cegis : ∀ vs fbs →
  bridge-batch vs fbs ≡ cegis-loop vs (map to-obs fbs)

```

This states that processing feedback through the bridge is *exactly* equivalent to running CEGIS on the mapped observations. All properties of CEGIS (monotonicity, convergence bounds, dissolution) transfer immediately:

```

bridge-mono : ∀ vs fbs →
  length (bridge-batch vs fbs) ≤ length vs

```


The crown jewel of integration is **bridge sufficiency**: if the learning loop produces feedback that maps to representative observations for the target, every surviving program is correct everywhere.

```
bridge-sufficient : ∀ vs target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  (fbs : List Feedback) →
  map to-obs fbs ≡ map λ( r → (r , target r)) reps →
  ∀ p → p ∈ bridge-batch vs fbs →
  MatchesAll p target
```

This theorem is the formal bridge between *learning convergence* and *synthesis correctness*: the learner drives observation collection, and the synthesizer guarantees that survivors are globally correct. The proof reduces to sufficient-cegis via bridge-is-cegis—the bridge adds no overhead.

The bridge also inherits commutativity and replay redundancy:

```
bridge-commutes : ∀ vs ifb zfb →
  bridge-step (bridge-step vs ifb) zfb
  ≡ bridge-step (bridge-step vs zfb) ifb
```

```
bridge-replay : ∀ vs fbs fb →
  fb ∈ fbs →
  bridge-step (bridge-batch vs fbs) fb
  ≡ bridge-batch vs fbs
```

Remark 10 (The Complete Pipeline). The full verified pipeline is now:

Learning loop $\xrightarrow{\text{to-obs}}$ Bridge $\xrightarrow{\text{bridge-batch}}$ VS $\xrightarrow{\text{survivors}}$ Correct programs

Each arrow is verified: to-obs is an EC-specific mapping; the bridge preserves CEGIS semantics (bridge-is-cegis); the VS converges (bridge-mono); survivors are globally correct (bridge-sufficient). Commutativity (bridge-commutes) and replay redundancy (bridge-replay) ensure the pipeline is robust to observation order and repetition.

11 Sample Complexity Lower Bound

The Feature Sufficiency Theorem (§8) established an *upper bound*: $|C/\sim|$ representative observations suffice for synthesis correctness. A natural question is whether this bound is *tight*—can we guarantee correctness with fewer observations? This section answers in the negative: $|C/\sim|$ observations are also *necessary*.

The core insight is information-theoretic: **CEGIS only distinguishes targets that differ on observed carriers**. Two target functions producing identical observations yield identical version spaces, so any survivor for one is automatically a survivor for the other. If the targets disagree on an unobserved carrier, no single program can match both—so CEGIS cannot guarantee correctness for either.

11.1 Target Indistinguishability

The foundational result: two targets that agree on all observed carriers produce the same version space.

```
same-obs-same-vs : ∀ vs (cs : List Carrier)
  (target1 target2 : Carrier → Bool) →
  ∀( c → c ∈ cs → target1 c ≡ target2 c ) →
  cegis-loop vs (map λ( c → (c , target1 c)) cs)
  ≡ cegis-loop vs (map λ( c → (c , target2 c)) cs)
```

The proof is direct: if $target_1$ and $target_2$ agree on every element of cs , then the mapped observation lists are identical by extensionality, and cegis-loop produces the same result by congruence.

An immediate corollary is **survivor transfer**: every program surviving CEGIS for one target automatically survives for any target indistinguishable on the observed carriers.


```

survivor-transfer : ∀ vs (cs : List Carrier)
  (1target 2target : Carrier → Bool) →
  ∀( c → c ∈ cs → 1target c ≡ 2target c ) →
  ∀ p → p ∈ cegis-loop vs (map λ( c → (c , 1target c)) cs) →
  p ∈ cegis-loop vs (map λ( c → (c , 2target c)) cs)

```

This is the *core lower bound mechanism*: survivors transfer freely between targets that look the same on the observations. If we have too few observations, there exist alternative targets that are indistinguishable from the real one, and survivors for the real target are equally valid for the alternatives—even when the alternatives disagree on uncovered carriers.

11.2 Covered Agreement and Feature Propagation

With feature-respecting targets, agreement on representatives propagates to all covered carriers:

```

covered-agreement : ∀ (1target 2target : Carrier → Bool)
  (reps : List Carrier) →
  FeatureRespects 1target →
  FeatureRespects 2target →
  ∀( r → r ∈ reps → 1target r ≡ 2target r ) →
  ∀ c → ∃([ r ] (r ∈ reps × AllFeatAgree r c)) →
  1target c ≡ 2target c

```

The proof chains three equalities: $target_1 c = target_1 r$ (by `FeatureRespects` on $target_1$ and `AllFeatAgree r c`), then $target_1 r = target_2 r$ (by the representative agreement hypothesis), then $target_2 r = target_2 c$ (by `FeatureRespects` on $target_2$).

With full coverage, this becomes unconditional:

```

full-coverage-determines : ∀ (1target 2target : Carrier → Bool)
  (reps : List Carrier) →
  FeatureRespects 1target →
  FeatureRespects 2target →
  Covers reps →
  ∀( r → r ∈ reps → 1target r ≡ 2target r ) →
  ∀ c → 1target c ≡ 2target c

```

This is the converse of the lower bound: with full coverage (`Covers reps`), no alternative target can agree on all representatives without agreeing *everywhere*. In other words, full coverage *uniquely determines* the target among all feature-respecting functions.

11.3 The Tight Bound

Combining the upper and lower bounds yields the central sample complexity result.

Theorem 19 (Tight-Bound). *The sample complexity of CEGIS with the Predicate DSL over features is exactly $|C/\sim|$: one representative per equivalence class.*

- **Upper bound** (sufficient-cegis, §8): With `Covers reps` and `FeatureRespects target`, every CEGIS survivor satisfies `MatchesAll`.
- **Lower bound** (lower-bound): Without `Covers`, there exist two feature-respecting targets that agree on all observed representatives but disagree on some uncovered carrier c . Every survivor for $target_1$ is also a survivor for $target_2$ (by `survivor-transfer`), but no program can match both (by `no-universal-match`):

```

lower-bound : ∀ vs (reps : List Carrier)
  (1target 2target : Carrier → Bool) (c : Carrier) →
  ∀( r → r ∈ reps → 1target r ≡ 2target r ) →
  1target c ≠ 2target c →
  ∀ p → p ∈ cegis-loop vs (map λ( r → (r , 1target r)) reps) →
  p ∈ cegis-loop vs (map λ( r → (r , 2target r)) reps)
  × ¬ (MatchesAll p 1target × MatchesAll p 2target)

```

The first conjunct says the survivor *also* survives for $target_2$; the second says no program can match both targets everywhere. Together: if an equivalence class is unobserved, CEGIS cannot distinguish between targets that differ there, so correctness is not guaranteed.

Remark 11 (Information-Theoretic Interpretation). The tight bound has a clean information-theoretic reading. Each equivalence class under AllFeatAgree is an *information cell*: all carriers within a cell are indistinguishable to any predicate program. The target function assigns one bit (true/false) per cell. CEGIS needs to determine these bits, and each representative observation reveals exactly one. With $k < |C/\sim|$ observations, at least one cell is undetermined—leaving $2^{|C/\sim|-k}$ possible targets indistinguishable from the true one. Full coverage ($k = |C/\sim|$) resolves all ambiguity.

This is analogous to the classical result that n binary questions are necessary and sufficient to identify one of 2^n hypotheses, specialized to our setting where the “questions” are observations and the “hypotheses” are feature-respecting target functions.

12 DSL Completeness

The tight bound (§11) tells us *how many* observations are needed. We now address a prior question: *does the DSL always contain a correct program*? If the target function lies outside the DSL’s expressiveness, no amount of observations can help.

We prove that the Boolean PredProg DSL is *expressively complete* for feature-respecting targets: for any target function that is constant on AllFeatAgree equivalence classes, there exists a PredProg matching it everywhere.

12.1 Fingerprints

The construction uses *fingerprints*. Given a carrier c and a feature list, the fingerprint of c is the conjunction of feature literals—positive if the feature holds, negative if not—that uniquely identifies c ’s equivalence class:

```
lit : Feature → Carrier → PredProg
lit f c = if eval-feature f c then feat f else ¬p (feat f)

fingerprint-list : List Feature → Carrier → PredProg
fingerprint-list []      = truep
fingerprint-list (f :: fs) c = lit f c ∧p fingerprint-list fs c

fingerprint : Carrier → PredProg
fingerprint = fingerprint-list all-features
```

The fingerprint satisfies three key properties:

```
fingerprint-refl      : ∀ c → eval (fingerprint c) c ≡ true

fingerprint-sound     : ∀ 1c 2c → FeaturesExhaustive →
  eval (fingerprint 1c) 2c ≡ true → AllFeatAgree 1c 2c

fingerprint-complete : ∀ 1c 2c →
  AllFeatAgree 1c 2c → eval (fingerprint 1c) 2c ≡ true
```

Together: the fingerprint of c_1 evaluates to true on c_2 *if and only if* c_1 and c_2 are in the same equivalence class. Here FeaturesExhaustive asserts that every feature appears in all-features: this is needed for the soundness direction (exhaustive features ensure no “hidden” feature can distinguish carriers that agree on listed features).

12.2 Synthesized Predicate

The synthesized predicate is the disjunction of fingerprints for representatives where the target is true:

```
dsl-synth : (Carrier → Bool) → List Carrier → PredProg
dsl-synth target []      = falsep
dsl-synth target (r :: reps) =
  if target r then fingerprint r vp dsl-synth target reps
```

```
else dsl-synth target reps
```

Correctness is established in two directions:

- **Soundness** (dsl-sound): If $\text{eval } (\text{dsl-synth target reps}) \ c \equiv \text{true}$, then some fingerprint in the disjunction matches c , so c is equivalent to a representative r with $\text{target } r \equiv \text{true}$; by `FeatureRespects`, $\text{target } c \equiv \text{true}$.
- **Completeness direction** (dsl-complete-eval): If $\text{target } c \equiv \text{true}$, then `Covers` provides a representative r with $\text{AllFeatAgree } r \ c$ and therefore $\text{target } r \equiv \text{true}$; the fingerprint of r is in the disjunction and matches c .

Combining both directions yields the full correctness theorem:

```
dsl-correct : ∀ target reps c →
  FeaturesExhaustive → FeatureRespects target → Covers reps →
  eval (dsl-synth target reps) c ≡ target c
```

12.3 The DSL Completeness Theorem

The existence statement follows immediately from `dsl-correct`:

```
dsl-complete : ∀ target (reps : List Carrier) →
  FeaturesExhaustive → FeatureRespects target → Covers reps →
  ∃[ p ] MatchesAll p target
```

Theorem 20 (DSL-Complete). *For any target function that respects the feature equivalence (`FeatureRespects`), the Boolean PredProg DSL contains a program that matches the target everywhere.*

Remark 12 (Closing the Loop). DSL Completeness pairs with the preceding results to give a definitive characterization of the synthesis problem:

1. **Expressiveness** (dsl-complete): The DSL always contains the correct answer.
2. **Sufficiency** (sufficient-cegis): $|C/\sim|$ observations guarantee CEGIS finds it.
3. **Necessity** (lower-bound): Fewer observations cannot guarantee correctness.
4. **Online** (refine-commutes): Observations can arrive in any order.

Together: “For any feature-respecting target, the system is guaranteed to synthesize the correct program from exactly $|C/\sim|$ observations, in any order.”

The fingerprint construction is not intended as a practical synthesis algorithm—it produces a DNF of exponential size. Its role is *existential*: it guarantees that the search space is nonempty, so CEGIS (which explores the enumerated version space) is searching in the right place. In practice, the enumerated version space (from `initial-vs`) often contains much smaller programs that match the target.

13 EC-Specific Instantiation

The generic `PredicateDSL` is instantiated differently for each `Environment Class`. This section describes the three instantiations implemented in the codebase; the code is in separate modules and shown here via illustrative excerpts.

13.1 Finite Deterministic MDP Synthesis

For FDMDPs, `Carrier` = `State`, and features are state-component predicates. The synthesis goal is a `RankModel`: a mapping from action pairs to `PredProg` terms.

```

module FMDPSSynthesis (State Action : Set)
  (step : State → Action → State × ℕ) (all-actions : List Action) where

  module WithStateFeatures (Feature : Set)
    (eval-feature : Feature → State → Bool) where

    open PredicateDSL State Feature eval-feature public

    record RankModel : Set where
      field prefer : Action → Action → PredProg

    rank-eval : RankModel → State → Action → Action → Bool
    rank-eval m s a b = eval (RankModel.prefer m a b) s

    -- The synthesis-to-CSHRL bridge:
    ModelPreserves : RankModel → Set
    ModelPreserves m = ∀ a b s →
      rank-eval m s a b ≡ true →
      action-value s a ≤s action-value s b

    module WithCorrectModel (m : RankModel) (preserves : ModelPreserves m) where
      instance
        SynthesizedHomo : CoindHomo
        SynthesizedHomo = record
          { ≤a__ = λ s a b → rank-eval m s a b ≡ true
            ; preserves = preserves }

```

The key bridge: a `RankModel` whose `prefer` predicates are proven to preserve action-value ordering automatically yields a `CoindHomo`. The ranking propagation theorem ensures feature-equivalent states get the same ranking:

```

rank-propagates : ∀ (m : RankModel) (is ≥s : State) (a b : Action) →
  FeatureEquiv (RankModel.prefer m a b) is ≥s →
  rank-eval m is a b ≡ rank-eval m ≥s a b
rank-propagates m is ≥s a b = propagation (RankModel.prefer m a b) is ≥s

```

13.2 Combinatorial Placement MDP Synthesis

For CPMDPs, the synthesis target is different: instead of rankings, we synthesize the *environment predicates* `is-dead` and `is-solved` that determine the step function.

```

module PlacementSynthesis (Config : Set) (Action : Set) ... where

  open PredicateDSL Config QFeature eval-qfeature

  -- The synthesized predicates ARE the step function
  is-dead-prog : PredProg
  is-dead-prog = any-feat all-attack-feats

  is-solved-prog : PredProg
  is-solved-prog = feat (length-is N)

  synth-is-dead : Config → Bool = eval is-dead-prog
  synth-is-solved : Config → Bool = eval is-solved-prog

  -- Open the EC with synthesized predicates:
  open CombinatorialPlacementMDP Config Action
  synth-is-dead synth-is-solved place solved-reward all-actions C0 N

```

This is the “synthesis on the critical path” pattern: without CEGIS producing `is-dead-prog` and `is-solved-prog`, there is no step function, and `find-policy` cannot operate. The synthesized `PredProg` terms *are* the environment dynamics.

13.3 Stochastic Finite MDP Synthesis

The third instantiation extends synthesis to *stochastic* MDPs, where the step function returns a distribution over successor states:

```

step : State → Action → Dist (State × Reward)

```

The key insight is that the PredicateDSL is entirely *independent* of whether transitions are deterministic or stochastic. The DSL operates on *states*, not transitions. All generic theorems—propagation, dissolution, feature sufficiency, online synthesis, the tight sample complexity bound, and DSL completeness—apply *unchanged*:

```
module SFDMDPSynthesis (State : Set) (Action : Set) (Reward : Set)
  (step : State → Action → Dist (State × Reward)) ... where

  open PredicateDSL State Feature eval-feature public
  -- ALL of: propagation, refine-equiv, dissolution, sufficient-cegis,
  --         refine-commutes, replay-redundant, LearningBridge,
  --         same-obs-same-vs, survivor-transfer, tight-bound
  -- are available immediately, with zero additional proof.
```

The *only* difference from the deterministic case is in the observation-generation layer: ranking observations are derived from *expected trace comparison* rather than deterministic trace comparison.

```
expected-trace-compare : State → Action → Action → ℕ → Bool
expected-trace-compare s a b k =
  expected-trace-action s a k ≤tb expected-trace-action s b k
```

The preservation condition uses *lexicographic* stream dominance on expected action-values ($\leq_s\text{-lex}$) rather than pointwise stream dominance, yielding a StochasticCoindHomo instead of a deterministic CoindHomo:

```
ModelPreservesLex : RankModel → Set
ModelPreservesLex m = ∀ a b s →
  RankHolds m s a b →
  expected-action-value s a ≤s-lex expected-action-value s b
```

Remark 13 (Four Synthesis Patterns). The framework now supports four synthesis patterns across three EC types:

1. **FDMDP (ranking synthesis):** Deterministic transitions, synthesize ranking predicates → CoindHomo.
2. **CPMDP (environment synthesis):** Deterministic transitions, synthesize environment predicates → step function.
3. **SFMDP (stochastic ranking synthesis):** Probabilistic transitions, synthesize ranking predicates from expected traces → StochasticCoindHomo.
4. **SFMDP-FOSD (distributional ranking synthesis):** Probabilistic transitions, synthesize ranking predicates from FOSD comparison of marginal CDFs → FOSDCoindHomo → StochasticCoindHomo (via the FOSD-to-Lex Bridge).

All four share the *same* generic theory: PredicateDSL, CEGIS, propagation, dissolution, sufficiency, online synthesis, the tight bound, and DSL completeness. The only variation is in how domain-specific feedback maps to PredObs observations—precisely the interface that the LearningBridge abstracts. The FOSD pattern adds a convergence guarantee (fosl-obs-sound, model-obs-consistent) and universality across monotone utility functions.

14 End-to-End: From Observations to Verified Policies

We present seven end-to-end demonstrations that exercise the full synthesis pipeline. Each is a self-contained Agda module in the codebase, compiled with `--safe` `--guardedness` and zero postulates.

14.1 Demo 1: 1D Maze (FDMDP)

The simplest demonstration: a three-state maze $P_0 \rightarrow P_1 \rightarrow P_2$ (goal) with two actions (Fwd, Bwd).

```

data State : Set where P0 P1 P2 : State
data Action : Set where Fwd Bwd : Action

step : State → Action → State × ℕ
step P0 Fwd = (P1 , 0)    step P1 Fwd = (P2 , 1)    step P2 Fwd = (P2 , 1)
step P0 Bwd = (P0 , 0)    step P1 Bwd = (P0 , 0)    step P2 Bwd = (P1 , 0)

```

Step 1: Observe. Five trace comparisons produce PredObs lists:

```

obs-≤bwdfwd : List PredObs = (P0 , true) :: (P1 , true) :: (P2 , true) :: []
obs-≤fwdbwd : List PredObs = (P0 , false) :: (P1 , false) :: []

```

Step 2: Synthesize. CEGIS refines the version space (3 candidates: truep, falsep, feat is-goal) and returns the consistent candidate:

```

synth-≤bwdfwd : synth-rank-pred 0 obs-≤bwdfwd ≡ just truep = refl
synth-≤fwdbwd : synth-rank-pred 0 obs-≤fwdbwd ≡ just falsep = refl

```

CEGIS determines: “Bwd is always ranked below Fwd” (truep) and “Fwd is never ranked below Bwd” (falsep).

Step 3: Verify. A domain-specific proof establishes that the synthesized ranking preserves action-values. This is the one step requiring domain knowledge:

```

synth-preserves : ModelPreserves synth-rank

```

Step 4: Construct. The verified ranking automatically yields a CoindHomo:

```

open WithCorrectModel synth-rank synth-preserves
-- SynthesizedHomo : CoindHomo (provided by the module)

```

Step 5: Extract & Execute. The policy selects the highest-ranked action and is rolled out:

```

synth-policy : State → Action
synth-policy s with rank-eval synth-rank s Bwd Fwd
... | true = Fwd    ... | false = Bwd

test-trajectory : run-synth P0 3 ≡ (Fwd,P1) :: (Fwd,P2) :: (Fwd,P2) :: [] = refl

```

Step 7: Cross-Validation. The EC’s find-policy independently computes the same policy:

```

finder-agrees : ∀ s → Finder.find-policy s 2 ≡ synth-policy s
finder-agrees P0 = refl ; finder-agrees P1 = refl ; finder-agrees P2 = refl

```

14.2 Demo 2: 4-Queens (CPMDP)

Here synthesis is on the *critical path*: the PredProg predicates define the step function.

Features. Six pairwise attack features (attacks 0 1, attacks 0 2, ..., attacks 2 3) and a length feature:

```

data QFeature : Set where
  attacks    : ℕ → ℕ → QFeature
  length-is  : ℕ → QFeature

```

Synthesized predicates. The `is-dead` predicate is a disjunction of all attack features; `is-solved` tests that the board has N queens:

```
is-dead-prog : PredProg = any-feat all-attack-feats -- 6 features
is-solved-prog : PredProg = feat (length-is N)

synth-is-dead : Config → Bool = eval is-dead-prog
synth-is-solved : Config → Bool = eval is-solved-prog
```

EC instantiation. The EC is opened with the synthesized predicates—no hand-crafted transition logic:

```
open CombinatorialPlacementMDP Config Action
synth-is-dead synth-is-solved place solved-reward all-actions C0 N
```

Solution. `find-policy` produces the optimal 4-Queens solution:

```
test-solution : run-policy (Ongoing []) N N ≡ C1 :: C3 :: C0 :: C2 :: [] = refl
```

Partial completion. The synthesized EC can complete *any* partial optimal placement:

```
test-from-1 : run-policy (Ongoing (1 :: [])) N 3 ≡ C3 :: C0 :: C2 :: [] = refl
test-from-2 : run-policy (Ongoing (1 :: 3 :: [])) N 2 ≡ C0 :: C2 :: [] = refl
test-from-3 : run-policy (Ongoing (1 :: 3 :: 0 :: [])) N 1 ≡ C2 :: [] = refl

-- Alternative solution discovered from a different starting point:
test-alt : run-policy (Ongoing (2 :: [])) N 3 ≡ C0 :: C3 :: C1 :: [] = refl
```

CEGIS demo. With a 2-feature vocabulary, 2 observations suffice to pin the attack predicate:

```
check-vs-after-both :
length (cegis-loop (initial-vs 0) (dead-obs :: alive-obs :: [])) ≡ 1 = refl
```

Propagation demo. Boards `[0,1]` and `[3,4]` have the same diagonal attack pattern; observing one classifies the other for free:

```
refine-same : refine (initial-vs 0) (λboard , true)
            ≡ refine (initial-vs 0) (λboard , true)
refine-same = refine-equiv (initial-vs 0) λboard λboard true λ( { ... → refl })
```

14.3 Demo 4: Biased Bandit (SfMDP)

The final demonstration completes the E2E trifecta by exercising the *stochastic* synthesis pipeline. The Biased Bandit is a single-state, two-armed bandit where Arm A pays reward 1 with probability 2/3 and Arm B with probability 1/3:

```
data State : Set where Playing : State
data Action : Set where ArmA ArmB : Action

step : State → Action → Dist (State × ℕ)
step Playing ArmA = bernoulli (Playing , 1) 2 (Playing , 0) 1 -- 2:1 odds
step Playing ArmB = bernoulli (Playing , 1) 1 (Playing , 0) 2 -- 1:2 odds
```

Synthesis framework. We open `SfMDPSSynthesis` (the stochastic instantiation from §13.3), which provides `expected-action-value`, `≤s-lex`, and `StochasticCoindHomo`:

```
open SfMDPSSynthesis State Action ℕ step ≤_ ≤?_ ≤-refl _+_ *_ 0 □_ 0
all-actions ArmB 3
```


Features. The bandit is stateless: a single trivial feature (`is-playing`, always true) suffices. This gives $|C/\sim| = 1$ —one equivalence class—so CEGIS needs exactly one observation per action pair (the tight bound).

```
data BanditFeature : Set where is-playing : BanditFeature
eval-bandit-feature : BanditFeature → State → Bool
eval-bandit-feature is-playing Playing = true

open WithStateFeatures BanditFeature eval-bandit-feature
open WithCEGIS (is-playing :: [])
```

Step 1: Observe. Two expected-trace comparisons at `Playing`:

```
obs-≤ba : List PredObs = (Playing , true) :: []    -- ArmB ≤ ArmA: yes
obs-≤ab : List PredObs = (Playing , false) :: []   -- ArmA ≤ ArmB: no
```

Step 2: Synthesize. CEGIS refines the version space (3 candidates) and returns:

```
synth-≤ba : synth-rank-pred 0 obs-≤ba ≡ just truep    = refl    -- ArmA always dominates
synth-≤ab : synth-rank-pred 0 obs-≤ab ≡ just falsep   = refl    -- ArmB never dominates
```

Step 3: Verify. The preservation proof uses lexicographic stream dominance. Arm A’s expected head reward is 2 (unnormalized), Arm B’s is 1:

```
synth-preserves : ModelPreservesLex synth-rank
synth-preserves ArmA ArmB s ()      -- eval falsep s ≡ true is absurd
head (synth-preserves ArmB ArmA Playing _) = ≤ss ≤zn      -- 1 ≤ 2
tail (synth-preserves ArmB ArmA Playing _) ()             -- 1 ≡ 2 absurd
```

The reflexive cases (`ArmA ArmA`, `ArmB ArmB`) follow from $\leq_s$ -lex-refl.

Step 4: Construct. Opening `WithCorrectModel` yields `SynthesizedStochasticHomo`—a verified `StochasticCoindHomo` constructed from the synthesized ranking.

Step 5: Extract & Confirm. The synthesized policy selects `ArmA`, matching the EC’s independent `find-policy`:

```
test-synth-Playing : synth-policy Playing ≡ ArmA      = refl
finder-agrees : find-policy Playing 3 ≡ synth-policy Playing = refl
```

Dissolution check. With $|C/\sim| = 1$, the version space collapses to a singleton after each observation:

```
vs-after-≤ba : length (cegis-loop (initial-vs 0) obs-≤ba) ≡ 2    = refl
vs-after-≤ab : length (cegis-loop (initial-vs 0) obs-≤ab) ≡ 1    = refl
```

Remark 14 (Stochastic Synthesis Modularity). This demo confirms the central modularity claim: the `PredicateDSL`, CEGIS, version space, propagation, dissolution, and tight bound are *identical* to the deterministic case. Only the observation layer changes—from deterministic trace comparison to expected-trace comparison. The type of the preservation proof changes from `ModelPreserves` (pointwise $\leq_s$) to `ModelPreservesLex` (lexicographic $\leq_s$ -lex), and the resulting homomorphism is `StochasticCoindHomo` instead of `CoindHomo`, but *no synthesis theorem needed modification*.

Extended demos. Extended demonstrations (8-Queens scaling, N-Queens feature discovery, FrozenLake, Cliff Walking, Auto-FTL, Taxi, stochastic variants, sample-efficiency analysis, and Cross-Pair Propagation) are in Appendix ??.

15 Self-Contained Example: Synthesis in Action

To make this document self-contained as a proof artifact, we include a small type-checked example demonstrating the full synthesis theory. This example uses a toy domain with two carriers and one feature.

```
module ToyExample where
  data ToyCarrier : Set where
    Hot Cold : ToyCarrier

  data ToyFeature : Set where
    is-hot : ToyFeature

  eval-toy : ToyFeature → ToyCarrier → Bool
  eval-toy is-hot Hot = true
  eval-toy is-hot Cold = false

  open PredicateDSL ToyCarrier ToyFeature eval-toy
  open CEGIS (is-hot :: [])
```

Version space. At depth 0, we have 3 candidates:

```
check-vs : length (initial-vs 0) ≡ 3
check-vs = refl
```

Observation 1. “Hot carriers should evaluate to true”:

```
obs1 : PredObs
obs1 = Hot , true

after-obs1 : length (refine (initial-vs 0) obs1) ≡ 2
after-obs1 = refl
```

falsep is eliminated (it evaluates to false at Hot).

Observation 2. “Cold carriers should evaluate to false”:

```
obs2 : PredObs
obs2 = Cold , false

after-both : length (cegis-loop (initial-vs 0) (obs1 :: obs2 :: [])) ≡ 1
after-both = refl
```

truep is eliminated (it evaluates to true at Cold). Only feat is-hot survives—the correct predicate is pinpointed from just 2 observations.

Propagation. Two Hot carriers are feature-equivalent and thus indistinguishable:

```
hot-equiv : AllFeatAgree Hot Hot
hot-equiv is-hot = refl

propagation-demo : refine (initial-vs 0) (Hot , true)
                  ≡ refine (initial-vs 0) (Hot , true)
propagation-demo = refine-equiv (initial-vs 0) Hot Hot true hot-equiv
```

Dissolution. After observing Hot, all equivalent carriers are redundant:

```
dissolution-demo :
  cegis-loop (refine (initial-vs 0) (Hot , true))
    (map (λ c → (c , true)) [])
  ≡ refine (initial-vs 0) (Hot , true)
dissolution-demo = exploration-dissolution (initial-vs 0) Hot true [] tt
```

16 Cross-Pair Propagation (Summary)

CEGIS observations are *pair-agnostic*: when ranking pairs share the same feature structure, observing a sub-optimal pair (C vs D) determines top-tier rankings ($A > B$) without ever comparing A or B. Two observations from a single pair can determine all $\binom{4}{2} = 6$ rankings at all 6 states—a 18× savings. Full details, verified learning curves, and rollout performance at scale are in Appendix ??.

17 Discussion

17.1 The Propagation Theorem as Generalization Principle

The Propagation Theorem (Theorem 1) plays a role analogous to *inductive bias* in machine learning. By restricting the hypothesis space to programs over features (rather than arbitrary functions on carriers), we obtain a formal generalization guarantee: any predicate consistent with observations at one carrier is automatically consistent at all feature-equivalent carriers.

This is not an approximation or a statistical guarantee—it is a *theorem*, holding for all carriers and all predicate programs. The three quantified consequences (Refine-Equiv, Observation Subsumption, Exploration Dissolution) make this guarantee precise and actionable.

17.2 Six Synthesis Patterns

The framework supports six synthesis patterns across deterministic, stochastic, distributional, and compositional settings:

- **FDMDP pattern (ranking synthesis):** The environment’s step function is known. Synthesis produces ranking predicates that, when verified, yield a `CoindHomo`. The synthesis is “nice to have”—the EC can compute the policy independently. The value of synthesis here is *interpretability* and *generalization*: the synthesized ranking explains why Fwd dominates Bwd (because it leads to a rewarding state), whereas the EC’s lookup table merely records *that* it does.
- **CPMDP pattern (environment synthesis):** The environment’s step function depends on *is-dead/is-solved* predicates that are not given a priori. Synthesis is on the *critical path*: without CEGIS producing the predicates, there is no step function, and find-policy cannot operate. This is a stronger form of synthesis—the program *defines* the environment, not just the ranking.
- **SFMDP pattern (stochastic ranking synthesis):** The step function returns *distributions* over successor states. Synthesis produces ranking predicates from expected trace comparisons, yielding a `StochasticCoindHomo`. This demonstrates the framework’s modularity: the generic `PredicateDSL` theory applies unchanged—only the observation-generation layer adapts to expected traces.
- **FOSD pattern (distributional ranking synthesis):** The same stochastic step function, but observations compare marginal CDFs rather than expected values. A ranking proven to preserve FOSD yields a `FOSDCoindHomo`, which the FOSD-to-Lex Bridge

(fospd-homo-ev-lex) automatically promotes to a StochasticCoindHomo. The FOSD pattern is strictly stronger: the synthesized ranking is optimal for *all monotone utility functions* (risk-averse, risk-neutral, risk-seeking), not just expected-value maximizers. The learning convergence theorem (fospd-obs-sound) ensures that FOSD observations drive CEGIS refinement correctly, and fospd?-complete guarantees no valid FOSD relationship is missed.

- **SD hierarchy pattern (parameterized distributional ranking synthesis):** The SD hierarchy generalizes the FOSD pattern to k -th order. When FOSD fails (CDFs cross), an SDCoindHomo k for $k \geq 1$ may still be established. The master bridge (sd-homo-ev-lex) ensures that *any* SD level yields EV-stream optimality. The hierarchy subsumption (sd-homo-subsumes) means a single FOSD verification at $k=0$ automatically provides the stronger guarantee at all levels—but when FOSD fails, the framework can “fall back” to SOSD ($k=1$) or TOSD ($k=2$), accepting the weaker guarantee that the ranking is optimal for risk-averse or prudent agents respectively.
- **Compositional pattern (modular ranking synthesis):** The compositional algebra enables a “divide and conquer” approach. Rather than verifying a monolithic ranking on a large composite environment, the framework decomposes it into independent components, verifies each in isolation, and composes the proofs via product-ranking, sum-ranking, or scale-ranking. The closure properties (SD-++, SD-scale) guarantee that composition preserves SD[k] at any level. This pattern is orthogonal to the five above: each component can use *any* of the other patterns, and the compositional algebra combines the results.

17.3 Computational Considerations

The 8-Queens demo revealed a practical tension: PredProg evaluation via pattern matching on AST nodes incurs overhead in Agda’s normalizer compared to direct Boolean computation. The hybrid approach (prove equivalence, then use the faster implementation) resolves this while maintaining formal correctness.

For production systems, one could:

1. Synthesize the PredProg term via CEGIS.
2. Prove its equivalence to a compiled implementation (as in Queens8E2E).
3. Use the compiled implementation for runtime evaluation.

The proof of equivalence is a one-time cost, and the resulting system has the interpretability of PredProg with the performance of native code.

17.4 Online Synthesis and the Learning Loop

Classical CEGIS is presented as a batch algorithm with a fixed observation set. The online synthesis results (§10) reveal that this presentation is a matter of convention, not necessity. Commutativity (refine-commutes) means the version space is a function of the *multiset* of observations, not their sequence. Replay redundancy means the version space is actually a function of the *set*. Together, these properties make the synthesis engine a *monotone, idempotent, commutative* operator on version spaces—precisely the algebraic structure needed for conflict-free replicated data types (CRDTs) [?].

This algebraic structure has practical consequences: multiple learning agents can submit observations to a shared synthesizer without coordination, checkpoints can be replayed without corruption, and the exploration strategy (depth-first, breadth-first, violation-driven) is provably irrelevant to correctness. The LearningBridge makes this integration explicit, reducing the gap between the learning loop and synthesis from an informal interface to a verified equivalence.

17.5 Tight Sample Complexity

The sample complexity lower bound (§11) completes the complexity picture for CEGIS with the Predicate DSL. The upper bound (sufficient-cegis) shows $|C/\sim|$ observations suffice; the lower bound (survivor-transfer) shows that with fewer observations, alternative targets remain indistinguishable. The gap is zero: the sample complexity is exactly $|C/\sim|$.

This result is unusual in the synthesis literature, where sample complexity bounds are typically asymptotic or probabilistic. Here, the bound is *exact* and *constructive*: we know precisely which observations are needed (one per equivalence class) and can prove that each is individually necessary (via covered-agreement, the missing class admits an alternative target). The bound also degrades gracefully: with $k < |C/\sim|$ observations, CEGIS is still sound on the k covered classes—only the $|C/\sim| - k$ uncovered classes are underdetermined.

17.6 DSL Completeness and the Fingerprint Construction

The DSL Completeness theorem (§12) resolves a foundational question: before asking *how many* observations CEGIS needs, one must ask whether the DSL *can express* the target at all. The fingerprint construction answers affirmatively: the Boolean algebra generated by the features is rich enough to capture every function constant on `AllFeatAgree` equivalence classes.

The construction is reminiscent of the classical Stone representation theorem, which establishes that every Boolean algebra is isomorphic to a field of sets. Here, the feature set generates a Boolean algebra of predicates, and the fingerprint construction explicitly realizes the isomorphism between feature-respecting targets and `PredProg` terms. The `FeaturesExhaustive` assumption plays the role of ensuring the Stone space has enough points (every feature is observable).

Practically, the fingerprint construction produces a DNF of exponential size—one disjunct per equivalence class where the target is true. This is never used as a synthesis algorithm; its role is purely existential. The CEGIS enumeration typically finds much smaller programs. But the completeness guarantee is essential: it certifies that CEGIS is searching in a space that *provably contains* the answer.

17.7 Cross-Pair Propagation and Observation Efficiency

The cross-pair propagation result (§??) reveals that the Propagation Theorem’s power extends beyond spatial generalization (across feature-equivalent states). When multiple ranking pairs are governed by the same feature structure, a single CEGIS version space simultaneously constrains *all* correlated pairs. This is not an ad-hoc optimization—it follows directly from the fact that CEGIS operates on `PredObs = Carrier × Bool`, which carries no information about which action pair generated the observation.

The practical implication is substantial. In environments with k actions, the naïve approach requires $\binom{k}{2} \times |C|$ observations (all pairs at all states). Dissolution reduces this to $\binom{k}{2} \times |C/\sim|$. Cross-pair propagation can further reduce it to as few as $|C/\sim|$ observations *total*—when all pairs share the same feature predicate. The savings factor grows quadratically in k .

The partial correlation case is equally important: when some pairs share a predicate and others do not, the savings are proportional to the “feature correlation” across the ranking structure. This can be detected at synthesis time by inspecting whether multiple pairs converge to the same surviving program.

17.8 Demo 15: GamblersRuin — FOSD Synthesis in a Richer Environment

We extend FOSD-based synthesis beyond the single-state `BiasedBandit` to **GamblersRuin**: 3 states (Ruin, Middle, Goal), 2 actions (Bet, Quit). From Middle, Bet yields 50% chance

of Goal (reward 1) and 50% Ruin (reward 0); Quit yields 100% Middle (reward 0). The optimal policy at Middle is Bet (higher expected value and FOSD-dominance).

The `Synthesis.FOSDStochasticFiniteMDP` module provides the same interface as `StochasticFiniteMDP` but generates observations via `fosd-compare` instead of `expected-trace` comparison. FOSD comparison requires equal total weights; `GamblersRuin`'s pure for Quit has weight 1 while Bet has weight 2. We use `step-fosd` that scales Quit to weight 2 (same semantics: 100% (Middle,0)) so `fosd?` can compare.

With feature `is-middle` (true only at Middle), CEGIS synthesizes a predicate for `prefer(Quit, Bet)` from the observation `(Middle, fosd-compare Middle Quit Bet 0)`. At depth 0, `Quit FOSD ≤ Bet` (Bet dominates); the synthesized predicate holds at Middle, yielding the optimal policy: Bet.

```
test-fosd-quit≤bet : fosd-compare Middle Quit Bet 0 ≡ true
test-synth-succeeds : synth-quit≤bet ≡ just truep
```

The `ModelPreservesFOSD` type and `WithCorrectModel` connect a verified `RankModel` to `FOSDCoindHomo`: when a synthesized ranking preserves pointwise FOSD at every timestep, it yields a coinductive homomorphism optimal for *all* monotone utility functions. The `WithLearningBridge` module provides the same learning-synthesis integration as `StochasticFiniteMDP`: `sample-to-obs` maps samples to FOSD observations via `fosd-compare`, `synth-learn-step` and `synth-learn-batch` process samples incrementally, and `extract-rank-model` builds a `RankModel` from the combined learner state. The Learning Bridge is demonstrated end-to-end in `GamblersRuin`: a sample `(Middle, Quit, Bet)` is processed by `synth-learn-batch`, and `extract-rank-model` yields a `RankModel` that correctly prefers Bet over Quit at Middle (`test-extracted-correct`).

17.9 FOSD-Based Synthesis

The expected-value stochastic synthesis pipeline (§13.3) compares actions by $\mathbb{E}[\text{trace}(s, a)] \leq \mathbb{E}[\text{trace}(s, b)]$. This suffices for expected-value maximizers, but collapses the full reward distribution to a single scalar.

A strictly stronger comparison is **First-Order Stochastic Dominance (FOSD)**: action a FOSD-dominates b at state s iff for all thresholds r and all timesteps n :

$$P(\text{reward}_n(s, a) \leq r) \leq P(\text{reward}_n(s, b) \leq r)$$

i.e., a 's cumulative distribution lies everywhere below b 's. FOSD implies expected-value dominance ($\mu \geq_{\text{FOSD}} \nu \Rightarrow \mathbb{E}[\mu] \geq \mathbb{E}[\nu]$) but not conversely. An action FOSD-dominating another is preferred by *all monotone utility functions*—risk-averse, risk-neutral, and risk-seeking agents agree.

The complete FOSD pipeline is implemented and machine-verified across three modules (`Probability.FOSD`, `Core.FOSD`, `Synthesis.FOSDStochasticFiniteMDP`).

17.9.1 Foundation: Probability.FOSD

The base layer provides:

- **CDF computation** for finite distributions (`cdf-weight`, `comp-weight`), with the fundamental identity $\text{CDF}(d, r) + \text{comp}(d, r) = W$ verified.
- **FOSD ordering** (`_FOSD≤_`) with preorder properties (reflexivity, transitivity).
- **FOSD $\Rightarrow$ E[X] bridge** (`fosd→ev`): proved for 0/1 distributions via complement-weight dominance and the tail-sum identity.
- **CDF distributes over concatenation**: `cdf-weight-++` and `total-weight-++` prove that CDF and total weight are additive over list concatenation—enabling compositional reasoning about composite distributions.
- **Decidable FOSD** (`fosd?`): a Boolean decision procedure comparing total weights and CDF values up to $\max(\text{support}(\mu), \text{support}(\nu))$.

- **Soundness and Completeness:** `fbsd?-sound` proves $\text{fbsd? } \mu \nu = \text{true} \Rightarrow \mu \text{ FOSD} \leq \nu$; `fbsd?-complete` proves the converse. The completeness proof derives $\text{total-weight } \nu \leq \text{total-weight } \mu$ from the FOSD relation via `cdf-max` and a custom bound lemma $n \leq m \sqcup n'$, then verifies CDF dominance at every threshold up to the joint maximum support.

Together, `fbsd?-sound` and `fbsd?-complete` establish that `fbsd?` is a *decision procedure*: $\text{fbsd? } \mu \nu = \text{true} \iff \mu \text{ FOSD} \leq \nu$.

17.9.2 Core: Coinductive FOSD and the Bridges

`Core.FOSD` lifts FOSD from single distributions to infinite reward streams:

- **PointwiseFOSD:** $\forall n. \text{marginal}(s, b, n) \text{ FOSD} \leq \text{marginal}(s, a, n)$ — FOSD at every depth.
- **StreamFOSD** (`_≤s-fbsd_`): a *coinductive* record on streams of distributions, the distributional analogue of `_≤s-lex`.
- **Stochastic Isomorphism:** `conjecture-forward` (`coinductive` $\Rightarrow$ `pointwise`) and `conjecture-reverse` (`pointwise` $\Rightarrow$ `coinductive` via `corecursive pointwise-to-stream-fbsd`). This extends the Stream Isomorphism from scalars to distributions.
- **FOSDCoindHomo:** a ranking that preserves PointwiseFOSD at every state—strictly stronger than `StochasticCoindHomo`:

```
record FOSDCoindHomo : ιSet where
  field
    ≤a__ : State → Action → Action → Set
    preserves-fbsd : ∀ a b s →
      ≤a__ s a b → PointwiseFOSD s b a
```

Two bridge theorems connect the FOSD and expected-value worlds:

- **FOSD $\Rightarrow$ E[X] per-timestep** (`pointwise-ev`): applies `fbsd-ev` at each depth to show $E[\text{marginal}(s, b, n)] \leq E[\text{marginal}(s, a, n)]$.
- **FOSD-to-Lex Bridge** (`fbsd→lex-stream`): lifts per-timestep EV dominance to lexicographic EV-stream dominance (`_≤s-ev`). The proof uses `subst2` with `iter-tab` equalities to bridge the tabulated stream representation:

```
→fbsdlex-stream : ∀ s a b →
  PointwiseFOSD s a b →
  marginal-ev-stream s b ≤s-ev marginal-ev-stream s a

fbsd→homoev-lex : (homo : FOSDCoindHomo) →
  ∀ a b s → FOSDCoindHomo.≤a__ homo s a b →
  marginal-ev-stream s a ≤s-ev marginal-ev-stream s b
```

`fbsd-homo→ev-lex` is the master implication: every `FOSDCoindHomo` is automatically a `StochasticCoindHomo`. Rankings verified via FOSD simultaneously guarantee expected-value optimality *and* dominance for all monotone utilities.

17.9.3 Synthesis: FOSD Learning Convergence

`Synthesis.FOSDStochasticFiniteMDP` connects FOSD to the CEGIS learning loop. The `FOSDConvergence` module takes a `RankModel` m and a proof `ModelPreservesFOSD` m and derives:

```
fbsd-obs-sound : ∀ a b s k →
  RankHolds m s a b →
  fbsd-compare s a b k ≡ true

model-obs-consistent : ∀ a b s k →
  RankHolds m s a b →
  consistent (RankModel.prefer m a b)
    (s , fbsd-compare s a b k) ≡ true
```

```
fosd-convergence : FOSDConvergenceGuarantee
fosd-convergence = combined-convergence , fosd-obs-sound
```

`fosd-obs-sound` uses `fosd?-complete` to prove that if the model ranks $a \leq b$ at state s , then `fosd-compare`—which calls `fosd?` on marginal reward distributions—returns `true` at every depth k . `model-obs-consistent` lifts this to the CEGIS version space: the correct predicate’s observation always passes the consistent check, so it survives every refinement round. `fosd-convergence` packages both with version-space monotonicity (`combined-convergence`) into a single convergence guarantee:

If a ranking model preserves FOSD, then (1) the version space shrinks monotonically, and (2) every FOSD observation generated from the model is truthful.

The key observation for synthesis is that the PredicateDSL and CEGIS are entirely *unchanged*. Only the observation-generation layer differs: instead of comparing expected traces (a scalar), we compare marginal CDFs (a function of the threshold). A FOSD observation at state s asserts $\forall r. \text{CDF}(s, b, r) \leq \text{CDF}(s, a, r)$ —which maps to the *same* `PredObs = Carrier × Bool` format. All propagation, dissolution, and tight-bound theorems apply without modification.

17.9.4 FOSD Demos

Three environments exercise the full FOSD pipeline:

- **BiasedBandit** (`Tasks.Stochastic.BiasedBanditFOSD`): Single-state, 2-armed bandit where ArmA (2/3 reward) FOSD-dominates ArmB (1/3 reward). Full `PointwiseFOSD` at all depths, `FOSDCoindHomo` instance, and the bridge theorem (`fosd-ev`) confirmed.
- **CoinFlip** (`Tasks.Stochastic.CoinFlipFOSD`): Two states (Heads, Tails), 2 actions (Flip, Hold). Demonstrates multi-state FOSD with terminal state handling: `PointwiseFOSD` proved at all depths for both states, `FOSDCoindHomo` instance, and `ModelPreservesFOSD` connecting the rank model to the synthesis pipeline.
- **GamblersRuin** (`Tasks.Stochastic.GamblersRuinFOSDSynth`, Demo 15, §17.8): 3 states, end-to-end FOSD synthesis with `ModelPreservesFOSD`, `WithCorrectModel`, and `WithLearningBridge`—full pipeline from observations to verified `FOSDCoindHomo`.

17.10 The Stochastic Dominance Hierarchy

FOSD (§17.9) provides the strongest distributional comparison: an action FOSD-dominating another is preferred by *all* monotone utility functions. But this strength is also a limitation—many distributions that exhibit a clear “better” relationship fail the FOSD test because their CDFs cross at some threshold.

The **Stochastic Dominance Hierarchy** generalizes FOSD to a parameterized family of orderings, each capturing a broader class of utility functions. The k -th order SD condition integrates the CDF k times; each additional integration “smooths out” CDF crossings, accepting more distribution pairs while restricting to a narrower (but still important) class of agents:

$$\text{SD}[0] = \text{FOSD} \Rightarrow \text{SD}[1] = \text{SOSD} \Rightarrow \text{SD}[2] = \text{TOSD} \Rightarrow \dots$$

17.10.1 Foundation: Probability.SD

The hierarchy is built on a single operation—**prefix sum**:

```
prefix-sum : ℕ (→ ℕ) → ℕ → ℕ
prefix-sum f zero    = f 0
prefix-sum f (suc r) = f (suc r) + prefix-sum f r

sd-weight : ℕ → Dist ℕ → ℕ → ℕ
sd-weight zero    d = cdf-weight d
sd-weight (suc k) d = prefix-sum (sd-weight k d)
```

sd-weight 0 is the CDF itself (FOSD level); sd-weight 1 is the integrated CDF (SOSD level); sd-weight k is the k -fold prefix sum. The ordering at each level is:

```
_SD[_≤_] : Dist ℕ → ℕ → Dist ℕ → Set
μ SD[ k ≤] v = ∀ r → sd-weight k v r ≤ sd-weight k μ r
```

The one-line subsumption. The crown jewel of the hierarchy is that subsumption is a single application of prefix-sum monotonicity:

```
prefix-sum-mono : ∀ {f g} →
  ∀( r → f r ≤ g r ) →
  ∀ r → prefix-sum f r ≤ prefix-sum g r

SD-subsumes : ∀ k μ{ v} →
  μ SD[ k ≤] v → μ SD[ suc k ≤] v
SD-subsumes k sd = prefix-sum-mono sd
```

If the k -th order integrated CDFs are pointwise ordered, their prefix sums (the $(k+1)$ -th order) are too. This gives the full chain $\text{FOSD} \Rightarrow \text{SOSD} \Rightarrow \text{TOSD} \Rightarrow \dots$ in one line.

The SD-to-EV bridge. A key lemma shows that sd-weight k d 0 $\equiv$ cdf-weight d 0 for all k —prefix-sum at $r=0$ is just the function value. This means $\text{SD}[k]$ at $r=0$ gives CDF dominance at $r=0$, which (for 0/1 distributions with equal total weights) yields expected-value dominance via the existing $\text{FOSD} \rightarrow \text{EV}$ bridge:

```
sd-at-0 : ∀ k d → sd-weight k d 0 ≡ cdf-weight d 0

→sdev : ∀ k μ v →
  AllBelow μ 1 → AllBelow v 1 →
  total-weight μ ≡ total-weight v →
  μ SD[ k ≤] v →
  weighted-sum μ ≤ weighted-sum v
```

Strict separation. SOSD is *strictly* weaker than FOSD. Consider $\mu = \{0:2, 2:1\}$ and $\nu = \{1:3\}$. At $r=1$: $\text{CDF}(\nu, 1) = 3 > 2 = \text{CDF}(\mu, 1)$, so FOSD fails. But the integrated CDFs satisfy $\text{icdf}(\nu, r) \leq \text{icdf}(\mu, r)$ at every r (verified by `refl` at $r = 0, 1, 2$), so SOSD holds—confirmed computationally in `Probability.SD`.

17.10.2 Coinductive Lift: Core.SD

`Core.SD` lifts the hierarchy to reward streams with three constructs:

```
PointwiseSD : ℕ → State → Action → Action → Set
PointwiseSD k s a b =
  ∀ n → marginal-reward s b n SD[ k ≤] marginal-reward s a n

record SDCoindHomo (k : ℕ) : 1Set where
  field
    ≤a__ : State → Action → Action → Set
    preserves-sd : ∀ a b s →
      ≤a__ s a b → PointwiseSD k s b a
```

`SDCoindHomo 0` is `FOSDCoindHomo`; `SDCoindHomo 1` is a ranking optimal for all risk-averse agents; `SDCoindHomo k` captures the $(k+1)$ -th order utility class.

Hierarchy subsumption on homos. If a ranking preserves $\text{SD}[k]$ (the strongest condition), it automatically preserves $\text{SD}[k+1]$ (the weaker condition):

```
sd-homo-subsumes : ∀ k → SDCoindHomo k → SDCoindHomo (suc k)
```

This means a single FOSD verification ($k=0$) simultaneously establishes optimality for *all* levels of the hierarchy.

The master bridge. The `SDToLexStream` module proves the ultimate connection: any `SDCoindHomo k` (at any level) yields lexicographic EV-stream dominance:

```
sd→homoev-lex : (homo : SDCoindHomo k) →
  ∀ a b s → SDCoindHomo.≤a homo s a b →
    marginal-ev-stream s a ≤s-ev marginal-ev-stream s b
```

The proof uses `sd→ev` at each timestep, then lifts to the stream level via `pw→lex`. This closes the hierarchy: rankings verified at *any* SD level yield expected-value optimality, while higher levels provide progressively weaker (easier to satisfy) requirements for broader classes of utility functions.

Remark 15 (Utility Function Classes). The SD hierarchy parameterizes the tradeoff between verification strength and agent generality:

- **SD[0] (FOSD):** optimal for all monotone utilities (risk-seeking, neutral, and averse).
- **SD[1] (SOSD):** optimal for all concave monotone utilities (risk-averse agents).
- **SD[2] (TOSD):** optimal for all utilities with non-increasing absolute risk aversion (prudent agents).
- **SD[k]:** each level captures a broader class, with weaker verification requirements.

When FOSD cannot be established between two distributions (CDF crossing), SOSD may still hold—accepting distributions with the same “integrated tail behavior.” The hierarchy lets the framework verify the strongest relationship that exists, rather than giving up when FOSD fails.

17.11 Compositional Ranking Algebra

The SD hierarchy (§17.10) establishes a parameterized family of distributional orderings. The **Compositional Ranking Algebra** proves that these orderings are closed under natural operations on distributions—enabling *modular verification*: verify each component in isolation, then compose the proofs automatically.

17.11.1 Foundation: Probability.Compose

The core algebraic insight is that `prefix-sum` is a *linear operator*. Two distributivity lemmas establish this:

```
prefix-sum-+ : ∀ (f g : ℕ → ℕ) r →
  prefix-sum λ( r → f r + g r) r ≡ prefix-sum f r + prefix-sum g r

prefix-sum-* : ∀ c (f : ℕ → ℕ) r →
  prefix-sum λ( r → c * f r) r ≡ c * prefix-sum f r
```

Since `sd-weight` is iterated `prefix-sum`, it inherits linearity. An extensionality lemma (`prefix-sum-ext`: pointwise-equal functions have equal prefix sums) bridges inductive hypotheses across recursive calls:

```
sd-weight-++ : ∀ k (1d 2d : Dist ℕ) r →
  sd-weight k (1d ++ 2d) r ≡ sd-weight k 1d r + sd-weight k 2d r

sd-weight-scale : ∀ k (d : Dist ℕ) r c →
  sd-weight k (scale c d) r ≡ c * sd-weight k d r
```

With these structural lemmas, the closure properties are immediate. Distribution concatenation (`++`) preserves `SD[k]` because `sd-weight` distributes over `++` and addition is monotone:

```
SD-++ : ∀ k μ1{ v1 μ2 v2 : Dist ℕ} →
  μ1 SD[ k ≤] v1 → μ2 SD[ k ≤] v2 →
  μ1 ( ++ μ2) SD[ k ≤] v1 ( ++ v2)

SD-scale : ∀ k c μ{ v : Dist ℕ} →
  μ SD[ k ≤] v → scale c μ SD[ k ≤] scale c v
```

Both proofs are one-liners after rewriting: SD-++ uses $\text{+-mono-}\leq$, and SD-scale uses $\text{*mono-}\leq$. Specializing to $k=0$ gives FOSD-++ and FOSD-scale.

17.11.2 Composition Theorems: Core.Compose

Core.Compose lifts the distribution-level closure to an algebra of *verified rankings*. The central abstraction is:

```
record VerifiedRanking
  (State Action : Set)
  (marginal : State → Action → ℕ → Dist ℕ)
  (k : ℕ) : 1Set where
  field
    ≤a : State → Action → Action → Set
    preserves : ∀ a b s → ≤a s a b →
      ∀ n → marginal s a n SD[ k ≤ ] marginal s b n
```

This is the abstract counterpart of SDCoindHomo: a ranking bundled with a proof that it preserves SD[k] on marginal rewards. The algebra provides five operations:

Hierarchy subsumption. Any ranking verified at level k is automatically verified at the weaker level $k+1$ (via SD-subsumes):

```
ranking-subsumes : VerifiedRanking S A m k → VerifiedRanking S A m (suc k)
```

Product composition. Given two component rankings and a composition operation $\oplus$ on distributions that preserves SD[k], the product ranking—requiring *both* components to rank favorably—is verified for the composite environment:

```
product-ranking :
  (compose : Dist ℕ → Dist ℕ → Dist ℕ) →
  ∀( μ1{ v1 μ2 v2 } → μ1 SD[ k ≤ ] v1 → μ2 SD[ k ≤ ] v2 →
    compose μ1 μ2 SD[ k ≤ ] compose v1 v2 ) →
  VerifiedRanking 1S 1A 1m k → VerifiedRanking 2S 2A 2m k →
  VerifiedRanking (1S × 2S) (1A × 2A) (compose-marginals) k
```

Instantiating $\oplus$ with ++ and SD-++ gives the concrete ++-product: marginal rewards merge as mixtures. Instantiating with scale gives scale-ranking: reward amplification preserves verification.

Sum composition. Two disjoint environments with state spaces $S_1 \uplus S_2$ compose with a *dispatched* ranking: in state $\text{inj}_1 s_1$, use ranking 1; in state $\text{inj}_2 s_2$, use ranking 2. Mismatched state/action pairs are excluded by $\perp$ (impossible to construct, so never encountered):

```
sum-ranking :
  VerifiedRanking 1S 1A 1m k → VerifiedRanking 2S 2A 2m k →
  VerifiedRanking (1S ⊔ 2S) (1A ⊔ 2A) sum-marginal k
```

Composability. The operations compose with each other, demonstrating genuine algebraic structure. For example, composing two components via concatenation and then scaling all rewards:

```
scaled-product : (c : ℕ) →
  VerifiedRanking 1S 1A 1m k → VerifiedRanking 2S 2A 2m k →
  VerifiedRanking (1S × 2S) (1A × 2A)
  λ( (1S, 2S) (1a, 2a) n → scale c (1m 1S 1a n ++ 2m 2S 2a n)) k
scaled-product c 1vr 2vr = scale-ranking c (++-product 1vr 2vr)
```

This one-line proof demonstrates the compositional nature of the algebra: ++-product composes the marginal rewards, then scale-ranking amplifies them—each step preserving the SD[k] guarantee.

Remark 16 (Practical Significance). The compositional algebra addresses a key scalability challenge: as environments grow in complexity, monolithic verification becomes impractical. The algebra enables a “divide and conquer” strategy:

1. Decompose a complex environment into independent components.
2. Verify each component's ranking in isolation (smaller state/action spaces).
3. Compose the proofs via product-ranking, sum-ranking, or scale-ranking.

The composed proof is as strong as the component proofs—no additional verification is needed at the composite level. This mirrors how software verification handles large systems: verify modules independently, then compose via interfaces.

17.11.3 Demonstration: BiasedBandit $\times$ CoinFlip

CompositionalDemo exercises all six operations of the algebra on two existing FOSD-verified environments. VerifiedRanking records are extracted directly from the FOSDCoindHomo records of BiasedBandit ($\text{ArmA} > \text{ArmB}$) and CoinFlip ($\text{Flip} > \text{Stay}$)—the extraction is zero-cost because $\text{FOSD}_{\leq}$ and $\text{SD}[0]_{\leq}$ are definitionally equal.

The *portfolio* environment ($\text{state} = S_1 \times S_2$, $\text{action} = A_1 \times A_2$, 4 combinations) is verified by a single call to `++-product`. The product ranking is computationally tested: $(\text{ArmB}, \text{Stay}) \leq (\text{ArmA}, \text{Flip})$ is $(\top, \top)$, confirmed by `refl`. CDF values of the composed marginal distributions are verified at depth 0: the dominant pair $(\text{ArmA}, \text{Flip})$ has $\text{CDF}(0) = 2$, while the dominated pair $(\text{ArmB}, \text{Stay})$ has $\text{CDF}(0) = 4$ —confirming that the product FOSD holds ($2 \leq 4$).

The portfolio ranking is then upgraded for free: ranking-subsumes lifts it to SOSD ($k=1$) and TOSD ($k=2$). Scaling by factor 3 via `scaled-product` is a one-liner. The phased environment (sum-ranking) dispatches rankings by component: in phase 1 (bandit), $\text{ArmB} \leq \text{ArmA}$; in phase 2 (coinflip), $\text{Stay} \leq \text{Flip}$ —both confirmed by `tt`. All tests pass by `refl`; the full file compiles under `--safe` with zero postulates.

17.12 Feature Discovery: The Feature Template Language

A persistent concern with feature-based synthesis is the *human knowledge bottleneck*: features like attacks $i\ j$ in N-Queens distill domain expertise into the framework. The Feature Template Language (FTL) addresses this by providing a systematic three-stage pipeline—enumerate, filter, reduce—from which domain-specific concepts emerge without human naming.

The pipeline. The human provides a state representation (`List ℕ` for placement problems), comparison primitives (value equality, absolute-difference equality, their composite, and length checks), and sample boards. The FTL then:

1. *Enumerates* all features instantiating the primitives over the state's index space—both atomic (`vals-eq`, `spread-eq`) and composite (`pair-any = either`).
2. *Filters* by keeping features never true on alive samples (conflict indicators).
3. *Eliminates redundancy* by detecting structural subsumption: when a composite `pair-any  $i\ j$` subsumes both atomics `vals-eq  $i\ j$` and `spread-eq  $i\ j$` , the atomics are dropped.

For N-Queens with $N = 4$: 23 candidates $\rightarrow$ 18 after filtering $\rightarrow$ **6 composite features** after redundancy elimination. Each surviving `pair-any  $i\ j$` computes “same column OR same diagonal”—the attacks concept, discovered without naming.

Dual discovery. The same filtering technique discovers *is-solved*: a dual filter keeps features true on *all* solved samples and false on *all* non-solved samples. For N-Queens, exactly one feature survives: `len-is 4`. Both the constraint predicate and the goal predicate are thus automatically derived.

Equivalence classes as quality metric. The tight sample complexity bound says CEGIS needs exactly $|C/\sim|$ observations, where equivalence is induced by the feature set. The FTL provides feature-vector and equiv-classes functions to compute $|C/\sim|$ directly. On all 16 length-2 boards with the 6 discovered features, $|C/\sim| = 2$ (conflicting vs. non-conflicting)—verified by refl. This gives a *formal objective for feature selection*: a good feature set is one that yields small $|C/\sim|$. The discovered features produce the same $|C/\sim|$ as hand-crafted ones.

Depth-feature tradeoff. The PredProg enumeration at depth d applies Boolean operators d times to the atomic feature set, yielding a version space that grows roughly as $O(n^{2^d})$ where n is the number of features. With FrozenLake’s 20 discovered features: depth 0 produces 22 candidates, depth 1 produces 1,012, and depth 2 produces 2,050,312—all verified by refl and checked in under 25 seconds. Depth 3, however, is infeasible: enumeration exceeds 20 minutes and 3 GB of memory before being terminated.

This exponential cliff makes the FTL’s role structural rather than merely convenient. If CEGIS were given raw bit-level state descriptors instead of FTL-discovered features, recovering a concept like “row is 3” would require depth 2+ Boolean formulas over individual bits—precisely the regime where enumeration explodes. By providing semantically meaningful features, the FTL keeps the required depth at 0 or 1: every ranking predicate across Demos 1–10 is at most a depth-1 combination (e.g., the Cliff Walking R-optimal predicate is a disjunction of two atoms). Demo 11 (FrozenLake 8×8) introduced synth-greedy-or to handle scattered state sets where no bounded-depth predicate suffices, building disjunctions of atoms incrementally rather than enumerating all depth- d combinations. Demo 12 (Taxi 5×5) introduces synth-decision-tree: a greedy Gini-based decision-tree synthesizer that builds nested if-then-else trees over relational features, producing interpretable policies that capture depth-4+ decision boundaries no flat disjunction can express. Demo 14 (FrozenLake 8×8 slippery, §??) applies the same algorithm to the stochastic pipeline, where synth-greedy-or fails on row/col features alone. The FTL thus controls not only the *observation complexity* (via $|C/\sim|$) but also the *synthesis tractability* (via depth, greedy atom selection, or decision-tree splitting).

Scope and extensibility. The framework now has three FTL families: a *structural* FTL for List $\mathbb{N}$ states (pairwise comparisons, covering N-Queens and similar placement problems), a *dynamics* FTL for finite-state MDPs (reward and self-loop templates, covering the Maze and similar FDMDPs), and an *automatic* FTL for $\mathbb{N}$ -state MDPs (the Auto-FTL of Demos 9–12, which derives features from the state count’s number-theoretic structure and the step function’s behavioral properties). All three plug into PredicateDSL via the same interface. The Auto-FTL eliminates the feature engineering step entirely: for FrozenLake 4×4 ($n = 16$, 56 features, 3 divisors), Cliff Walking ($n = 24$, 106 features, 6 divisors), FrozenLake 8×8 ($n = 64$, 200 features, 5 divisors), and Taxi 5×5 ($n = 500$, ~1,700 features, 10 divisors), the auto-generated features produce optimal policies with grid coordinates emerging automatically as factorization features. Taxi further demonstrates three synthesis paradigms at scale (500 states, 6 actions): fully automated greedy-or CEGIS (~9.5 min, opaque), decision-tree CEGIS over 14 relational features (~13 min, interpretable), and a hand-crafted relational navigation function. Compound and relational features extend the Auto-FTL to environments requiring multi-phase task reasoning. The FTL sits outside the Trusted Computing Base: the verified synthesis pipeline consumes discovered features without modification.

17.13 Related Work

We now position our contributions against six lines of related work, summarizing the comparison in Tables ?? and ??.

Version spaces. Mitchell [?] introduced version spaces for concept learning, representing the set of consistent hypotheses via S/G boundary sets and proving correctness of the candidate-elimination algorithm. Hirsh [?] generalized version spaces beyond boundary-set

representations. Lau et al. [?] developed Version Space Algebras for programming-by-demonstration, enabling compositional hypothesis-space manipulation. Our work builds on Mitchell’s conceptual foundation but diverges in three ways: (i) we prove *dissolution*—that feature-equivalent observations are provably redundant (refine-absorb)—a property not formalized in any prior version-space work; (ii) we establish a *tight sample complexity bound* (exactly $|C/\sim|$), whereas Mitchell’s framework provides no quantitative complexity analysis; and (iii) our entire treatment is machine-verified in Agda, whereas prior work uses pen-and-paper proofs only.

CEGIS. Solar-Lezama et al. [?] introduced CEGIS for program synthesis in the Sketch framework. Jha and Seshia [?] extended CEGIS modulo theories to handle background theories and non-trivial constants. Our version is specialized to Boolean predicates over features and operates within a purely constructive, termination-checked setting (Agda with `--safe`). The convergence proof, strict-shrinkage bound, and the tight sample complexity result ($|C/\sim|$ necessary *and* sufficient) are, to our knowledge, the first formal CEGIS results in any proof assistant.

State abstraction in RL. Li et al. [?] provide a comprehensive taxonomy of state abstractions in MDPs, showing that various abstraction types preserve different solution quality guarantees. Abel et al. [?] extend this to study which abstractions preserve near-optimal behavior. Ferns et al. [?] and Zhang et al. [?] develop bisimulation metrics for measuring behavioral equivalence of states. Our AllFeatAgree equivalence relation can be seen as a concrete feature-based bisimulation. The key difference is that state abstraction work focuses on *value function preservation*—whether abstract states have similar values—while we focus on *synthesis observation efficiency*—how many observations CEGIS needs. The Propagation Theorem connects these perspectives: feature equivalence simultaneously preserves predicate semantics *and* reduces required observations, a connection not made in prior work.

Multi-task learning and shared hypotheses. Crammer and Mansour [?] study multi-task learning through shared hypothesis pools, showing that when tasks share structure, fewer samples per task suffice, with VC-dimension generalization bounds. This is the closest relative to our cross-pair propagation result: both exploit shared structure across “tasks” (their terminology) or “ranking pairs” (ours) to reduce sample complexity. However, the frameworks differ fundamentally. Crammer and Mansour work in a statistical PAC-learning setting with approximate generalization bounds over distributions of tasks. Our cross-pair propagation is *exact*: a C-vs-D observation at state s and an A-vs-B observation at state s produce *definitionally identical* PredObs values when both pairs are governed by the same predicate—not approximately similar, but `refl` in Agda. Furthermore, we compose this with dissolution (spatial propagation), achieving multiplicative savings $\binom{k}{2} \times m$ for k actions with m states per class) that have no analogue in the multi-task literature.

Preference-based RL. Fürnkranz et al. [?] formalize preference-based RL, learning policies from qualitative pairwise comparisons rather than numerical rewards. Wirth et al. [?] survey the field, and recent work [?, ?] studies sample complexity with general function approximation. These approaches treat each pairwise query as an independent observation. None formalize the phenomenon we demonstrate: that when ranking pairs share feature structure, observations about sub-optimal pairs *exactly determine* top-tier rankings. The cross-pair propagation result shows that, in structured environments, the effective number of independent queries can be far smaller than the number of action pairs—a reduction invisible to frameworks that treat queries as independent.

Program synthesis for RL. Verma et al. [?] synthesize programmatic policies from neural network policies via imitation learning, producing interpretable programs but without formal verification. Inala et al. [?] use neurosymbolic synthesis for safe RL, combining neural networks with symbolic constraints. Both follow a *distillation* paradigm: train a

DNN first, then extract a structured policy from it. Our approach eliminates the DNN training phase entirely—the Finder computes optimal actions directly from the environment specification, and CEGIS synthesizes a verified predicate program from those observations. The synthesized programs are verified *within* the type system, not by external checking.

Verifiable RL via policy extraction. Bastani, Pu, and Solar-Lezama [?] propose VIPER, which distills DNN policies into decision trees and verifies properties (robustness, Lyapunov stability, safety invariants) using external tools (Z3, SOS programs). VIPER is the closest prior work to our system and merits detailed comparison, as both aim for verifiable, interpretable RL policies. However, the approaches differ at every level of the pipeline:

- **Oracle.** VIPER’s oracle is a DNN trained by standard deep RL—an approximate policy with an approximate Q-function. Our Finder is an exhaustive oracle backed by CoindHomo: it evaluates all action sequences to the depth horizon, producing *exact* optimal actions. VIPER distills an approximation; we compress the ground truth.
- **Extraction.** VIPER uses iterative imitation learning (DAGGER variant) with Q-function weighting. Our CEGIS uses version-space refinement with monotonic convergence and a tight sample complexity bound ($|C|/\sim$). VIPER’s convergence guarantee (Theorem 2.2) is asymptotic and applies to the convex surrogate loss, not to the decision tree itself. Our convergence is exact: the version space shrinks monotonically and exhausts in finitely many observations.
- **Verification.** VIPER verifies *specific properties* post-hoc—robustness to L_∞ perturbations, Lyapunov stability around the origin, safety invariants for bounded initial regions—using external SMT solvers. These are important but *secondary*: a robust policy can be suboptimal, a stable policy can be far from the best. Our verification is *intrinsic*: test-auto-all-match = refl proves optimality at every non-terminal state, checked by Agda’s kernel. This is the *primary* property—if the policy is provably optimal and the environment is well-specified, safety follows from the environment structure.
- **Policy size.** VIPER’s decision trees range from 31 nodes (toy Pong) to 9,757 nodes (half-cheetah). Our PredProg policy for Cliff Walking is ~ 38 bits—two predicates and a default action. The gap reflects the representations: decision trees repeat state-space partitions at every node, while Boolean predicate programs share features across branches.
- **Counterexamples.** Crucially, VIPER’s extraction can introduce errors. In their toy Pong experiment, Z3 discovers a counterexample in the extracted decision tree: the paddle oscillates and misses the ball near screen edges. The authors note (footnote 10) that “we have no way of knowing if other counterexamples exist” in the original DNN policy. In CSH, counterexamples cannot exist: the synthesized policy is proven to match the Finder at every state before the proof artifact is accepted.

Table ?? summarizes the quantitative comparison on a common benchmark scale. The fundamental difference is architectural: VIPER is *learn, compress, then verify*; CSH is *synthesize with proof*. VIPER’s three-stage pipeline has three potential failure points (DNN training, tree extraction, property verification), while CSH’s unified pipeline has none (given correct environment specification).

VIPER does handle continuous state spaces and continuous dynamics, which CSH does not yet address. This is a genuine limitation of our current framework, not a flaw in VIPER’s design. However, VIPER’s verification also degrades with complexity: their half-cheetah tree (9,757 nodes) has no verification at all, and their cart-pole stability proof uses a degree-5 Taylor approximation rather than the true dynamics.

Table 2: Comparison of our contributions against related work. **Bold** entries mark capabilities unique to our framework.

Capability	Ours	Mitchell '82	CEGIS (S-L '06)	Multi-task (C&M '12)
Version-space refinement	✓	✓	—	—
CEGIS convergence proof	formal	—	informal	—
Dissolution (obs. redundancy)	✓	—	—	—
Tight sample complexity	$ C/\sim $	—	—	VC bound
Cross-pair propagation	✓	—	—	statistical
Spatial $\times$ cross-pair composition	✓	—	—	—
Machine-verified (proof assistant)	Agda	—	—	—
Applied to RL ranking synthesis	✓	—	—	—
FOSD-based distributional synthesis	✓	—	—	—
k -th order SD hierarchy with subsumption	✓	—	—	—
Compositional ranking algebra (SD closure)	✓	—	—	—

Table 1: CSH synthesis vs. VIPER [?] on representative benchmarks.

	CSH (Cliff Walking)	VIPER (Pong / cart-pole)
Oracle	Finder (exact, CoindHomo)	DNN (approx., DRL-trained)
Training samples	56 oracle queries	$\sim 10,000+$ episodes
Policy size	~ 38 bits	31-9,757 nodes
Verification	Optimality at all states	Robustness / stability / safety
Verification tool	Agda kernel (refl)	Z3 / SOS programs
Counterexamples	Impossible by construction	Discovered post-hoc
Correctness	Proven (refl)	Approximate (property-specific)
Hyperparameters	0	$\sim 8+$ (lr, ϵ , γ , depth, ...)

Verified RL. Lesage and Ong [?] formalize value and policy iteration convergence in Coq but do not address synthesis. The CSHRL framework [?] provides the coinductive homomorphism theory we build upon. To our knowledge, no prior work combines formal verification of RL with program synthesis, let alone with propagation and cross-pair observation transfer.

18 Conclusion

We have presented a verified program synthesis framework for Coinductive Symmetric Homomorphism Reinforcement Learning. The framework introduces the following key components, each machine-verified:

1. A **Predicate DSL** that provides interpretable, generalizable, and verifiable Boolean programs over environment features.
2. A **CEGIS algorithm** with five proven properties: monotonic convergence, soundness, propagation acceleration, bounded convergence, and strict shrinkage.
3. A **Propagation Quantification** theory consisting of three theorems (Refine-Equiv, Observation-Subsumption, Exploration-Dissolution) that formalize how feature equivalence reduces the observations needed for synthesis from $|C|$ to $|C/\sim|$.
4. A **Quotient Structure** formalizing AllFeatAgree as an equivalence relation with the Exploration-Complete theorem: after observing one representative per class, no further observation can refine the version space.
5. A **Temporal Propagation** theory where feature-preserving transitions extend propagation along entire trajectories, with Trajectory-Dissolution proving that one trajectory subsumes all equivalent trajectories—connecting propagation to CSHRL’s coinductive structure.

6. A **Feature Sufficiency Theorem** (the “crown jewel”) that upgrades exploration completeness from stabilization to *correctness*: when the target respects features, every CEGIS survivor is provably correct on all carriers—not just observed representatives.
7. A **Multi-Predicate Propagation** theory where synthesizing k predicates from the same feature space requires only $|C/\sim|$ representative observations (not $k \cdot |C/\sim|$), and cross-predicate entailment further reduces evaluations via cross-dissolution.
8. An **Online Synthesis** theory establishing that refinement order is irrelevant (commutativity), replayed observations are provably redundant (idempotence), and a generic LearningBridge composes learning convergence with synthesis correctness—enabling asynchronous, incremental, and replay-tolerant integration with the CSHRL learning loop.
9. A **Sample Complexity Lower Bound** proving that the $|C/\sim|$ upper bound is tight: two targets agreeing on observed carriers produce identical version spaces, so survivors transfer freely between indistinguishable targets. Combined with feature sufficiency, this proves the sample complexity is *exactly* $|C/\sim|$ —one representative per equivalence class is both necessary and sufficient.
10. **DSL Completeness**: a constructive proof (via fingerprints) that the Boolean PredProg DSL is expressively complete for feature-respecting targets. For any target constant on AllFeatAgree classes, the synthesized program matches the target everywhere. This closes the theoretical loop: the DSL contains the answer, CEGIS finds it from $|C/\sim|$ observations, and no fewer suffice.
11. **Cross-Pair Propagation**: a verified demonstration that CEGIS observations are pair-agnostic—observing the sub-optimal pair (C vs D) determines top-tier rankings ($A > B$) without ever comparing A or B. Two observations from a single pair determine all $\binom{4}{2} = 6$ rankings at all 6 states, a 18× savings.
12. **Three Feature Template Languages (FTLs)** for automatic feature discovery across all three ECs:
 - *Structural FTL*: a three-stage pipeline (enumerate, filter, reduce) for List $\mathbb{N}$ states. For N-Queens, 23 candidates reduce to 6 composite features—each pair-any $i\ j$ is “attacks,” discovered without naming. A dual filter discovers is-solved. Feature vectors and $|C/\sim|$ counting provide a formal quality metric.
 - *Dynamics FTL*: dynamics-derived features (has-pos-reward, is-self-loop) for finite-state MDPs. For the Maze, is-self-loop Fwd evaluates identically to the hand-crafted is-goal—the “goal” concept emerges from dynamics.
 - *Auto-FTL*: automatic feature generation for $\mathbb{N}$ -state MDPs from the state count’s number-theoretic structure (divisors, modular arithmetic) and the step function’s behavioral properties. For FrozenLake 4×4 ($n = 16$, 56 features), Cliff Walking ($n = 24$, 106 features), FrozenLake 8×8 ($n = 64$, 200 features), and Taxi 5×5 ($n = 500$, ~1,700 features), the auto-generated features include factorization features that *automatically recover grid coordinates* ($\text{div-is } w\ k \equiv \text{row-is } k$ for the appropriate divisor w), eliminating the last human input in the synthesis pipeline. The Cliff Walking demo demonstrates depth-1 predicate synthesis; the FrozenLake 8×8 demo introduces greedy cascading CEGIS (synth-greedy-or) for scattered state sets and a memoized DP Finder for depth-16 planning over 64 states; and the Taxi demo introduces compound features for multi-phase tasks and relational wall-aware navigation verified optimal over 500 states.

All three plug into PredicateDSL via the same (Feature, eval-feature) interface.

13. **FOSD Synthesis**: a complete First-Order Stochastic Dominance pipeline with five verified results: (i) a decidable FOSD comparator with proved soundness and completeness; (ii) the Stochastic Isomorphism (coinductive FOSD $\Leftrightarrow$ pointwise FOSD); (iii) the FOSD-to-Lex Bridge (every FOSDCoindHomo is automatically a StochasticCoindHomo); (iv) FOSD learning convergence (fosd-obs-sound, model-obs-consistent, fosd-convergence);

and (v) three FOSD demos (BiasedBandit, CoinFlip, GamblersRuin) exercising the full pipeline.

14. **Stochastic Dominance Hierarchy**: a generalization from FOSD to k -th order SD via iterated prefix sums of the CDF. The one-line subsumption proof ($\text{SD} \text{--} \text{subsumes } k \text{ sd} = \text{prefix-sum-mono sd}$) establishes the chain $\text{FOSD} \Rightarrow \text{SOSD} \Rightarrow \text{TOSD} \Rightarrow \dots$; the SD-to-EV bridge ($\text{sd} \rightarrow \text{ev}$) proves that every level implies expected-value dominance; and coinductive $\text{SDCoindHomo } k$ with $\text{sd-homo} \rightarrow \text{ev-lex}$ shows that rankings verified at *any* level yield lex EV-stream optimality—parameterizing the tradeoff between verification strength and utility-function generality.
15. **Compositional Ranking Algebra**: an algebraic framework for composing independently-verified rankings. The linearity of prefix-sum (distributes over $+$ and $\times$) yields sd-weight distributivity, which gives $\text{SD}[k]$ closure under concatenation ($\text{SD} \text{--} ++$) and scaling ($\text{SD} \text{--} \text{scale}$). The VerifiedRanking abstraction supports product composition (product-ranking), sum composition (sum-ranking), and scaling (scale-ranking)—enabling modular verification where components are verified in isolation and composed automatically.

The framework is demonstrated end-to-end on all three Environment Classes with fourteen verified demos: a 1D Maze (FDMDP) where synthesis produces verified CoindHomo rankings; N -Queens (CPMDP) where synthesized predicates *define* the environment’s dynamics, enabling solutions for $N = 4$ and $N = 8$; a Biased Bandit (SFMDP) confirming that synthesis applies unchanged to stochastic transitions; **N-Queens with Feature Discovery** where the structural FTL discovers both predicates automatically; **Maze with Feature Discovery** where the dynamics FTL discovers the “goal” concept; **FrozenLake** 4×4 —a well-known OpenAI Gym benchmark with fully automated policy synthesis from hand-crafted features; **Cliff Walking** 4×6 —a Sutton & Barto benchmark with cliff detection; **Auto-FTL FrozenLake**—FrozenLake solved with *zero human feature engineering*, where grid coordinates emerge automatically from modular arithmetic; **Auto-FTL Cliff Walking**—the same zero-engineering pipeline applied to Cliff Walking ($n = 24$, 106 features, 6 divisors), demonstrating scaling to richer factorization structure and depth-1 predicate synthesis; **FrozenLake** 8×8 —a $4 \times$ scale-up ($n = 64$, 200 features, 53 non-terminal states) solved in 7 seconds via a memoized DP Finder and greedy cascading CEGIS (synth-greedy-or); **Taxi** 5×5 —a $10 \times$ scale-up to 501 states with 6 actions, internal walls, and a multi-phase task, solved via both fully automated 6-stage cascading CEGIS ($\sim 1,700$ auto-generated features, 9.5 minutes) and interpretable wall-aware relational navigation, verified optimal at all 500 non-terminal states; and **Slippery FrozenLake** 4×4 —the first stochastic RL benchmark in the synthesis pipeline, featuring a **memoized stochastic Finder** (MemoizedStochastic) that builds a DP value table in $O(\text{depth} \times |S| \times |A| \times |\text{outcomes}|)$ time, enabling depth-10 planning that produces a genuine safety-aware policy (all 4 actions with expected-value justifications), with decision-tree CEGIS and full verification at all 11 non-terminal states; and **Slippery FrozenLake** 8×8 —scaling the stochastic pipeline to 64 states (53 non-terminal, 10 holes), same memoized Finder at depth 10, policy computed in ~ 27 seconds, with synth-decision-tree over 16 row/col features ($D \rightarrow L \rightarrow U \rightarrow R$ cascade), verified at all 53 states in ~ 2.9 hours. All tests pass by refl.

The **FOSD synthesis pipeline** (§17.9) upgrades stochastic rankings from expected-value comparison to distributional dominance. The complete pipeline includes: a decidable FOSD comparator with proved soundness *and* completeness (fosd?-sound, fosd?-complete); the Stochastic Isomorphism establishing that coinductive FOSD is isomorphic to pointwise FOSD; the FOSD-to-Lex Bridge proving that every FOSDCoindHomo is automatically a StochasticCoindHomo; FOSD learning convergence (fosd-obs-sound, model-obs-consistent, fosd-convergence) connecting FOSD observations to CEGIS version-space refinement; and three FOSD demos (BiasedBandit, CoinFlip, GamblersRuin) exercising the full pipeline from observations to verified FOSDCoindHomo—all machine-verified.

The **Stochastic Dominance Hierarchy** (§17.10) extends this to a parameterized family of distributional orderings. The k -th order SD condition integrates the CDF k times: $\text{SD}[0]$ is FOSD (all monotone utilities), $\text{SD}[1]$ is SOSD (risk-averse agents), $\text{SD}[2]$ is TOSD (prudent agents). The hierarchy subsumption is a one-line proof ($\text{SD} \text{--} \text{subsumes } k \text{ sd} = \text{prefix-sum-mono sd}$), and the master bridge theorem ($\text{sd-homo} \rightarrow \text{ev-lex}$) shows that rankings verified at

any SD level automatically yield expected-value stream optimality. A concrete SOSD-but-not-FOSD example demonstrates that the hierarchy is strictly more expressive: distributions whose CDFs cross (failing FOSD) may still be ordered by integrated CDFs (satisfying SOSD), capturing a broader and practically important class of distributional relationships.

The **Compositional Ranking Algebra** (§17.11) addresses scalability by proving that verified rankings compose. The key structural insight is that prefix-sum is a linear operator: it distributes over addition and scalar multiplication. Since `sd-weight` is iterated prefix-sum, it inherits this linearity, yielding distributivity of `sd-weight` over distribution concatenation (`++`) and scaling. The closure properties (`SD-++`, `SD-scale`) follow as one-line corollaries. At the environment level, the `VerifiedRanking` abstraction packages an action ordering with its `SD[k]` preservation proof. Product composition, sum composition, and scaling are all proved to preserve verification—and they compose with each other: scaled-product $c \text{ } vr_1 \text{ } vr_2 = \text{scale-ranking } c \text{ } (++)\text{-product } vr_1 \text{ } vr_2$ demonstrates genuine algebraic composability. This enables modular verification: decompose a complex environment into components, verify each in isolation, and compose the proofs automatically.

The **Feature Template Languages** (§??) address the human knowledge bottleneck with three complementary discovery pipelines. The structural FTL derives features from state structure (pairwise comparisons); the dynamics FTL derives features from step-function behavior (reward, self-loops); and the Auto-FTL derives features from the state count’s number-theoretic properties (divisors, modular arithmetic) combined with dynamics. Together they cover all three ECs with progressively less human input. The Auto-FTL, in particular, eliminates the feature engineering step entirely: the human provides only the environment definition, and the features emerge from the state encoding. This is validated on four environments—FrozenLake 4×4 ($n = 16$, 56 features), Cliff Walking ($n = 24$, 106 features), FrozenLake 8×8 ($n = 64$, 200 features), and Taxi 5×5 ($n = 500$, ~1,700 features)—with grid coordinates emerging automatically as factorization features in all cases. Taxi demonstrates that the Auto-FTL’s divisor-based decomposition extends to compound state encodings: divisors 4, 20, and 100 recover destination, grid position, and row respectively. Feature vectors and equivalence class counting ($|C/\sim|$) connect feature quality to the tight sample complexity bound. Crucially, the FTL also controls *synthesis tractability*: by providing semantically meaningful features, it keeps the required `PredProg` depth at 0–1, where enumeration remains feasible—enumeration takes seconds, rather than depth 3+, where it becomes infeasible. When even depth-1 is insufficient for scattered state sets (as with FrozenLake 8×8’s 3 U-optimal states), `synth-greedy-or` builds the predicate incrementally from atoms, avoiding exponential enumeration entirely. When the state space exceeds the Peano bottleneck (as with Taxi’s 500 states), compound and relational features provide a synthesis path that sidesteps iterative CEGIS entirely, while maintaining verified optimality. All three FTLs sit outside the Trusted Computing Base.

All results are machine-verified in Agda with `--safe --guardedness`, zero postulates. This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code. The synthesis framework is additive—it builds on the CSHRL foundation without modification, demonstrating the extensibility of the coinductive homomorphism approach.

19 Reproducibility and Build

The paper is a literate Agda document. Building the PDF type-checks all embedded code.

- **Tested toolchain:** Agda 2.7.x (2.7.0 recommended), Agda StdLib 2.0, Lua_{La}T_EX.
- **Build commands:** In `literate/`, run `make synthesis` (or `agda --latex CSHRLSynthesis.lagda.tex` followed by `luaLaTEX` on the generated `CSHRLSynthesis.tex`).
- **Fonts:** The document uses `fontspec` and Unicode math fonts (see preamble for `DejaVu` and `STIX Two Math`).
- **Full codebase:** The complete synthesis framework resides in `src/CSHRL/Synthesis/` with end-to-end demos in `src/CSHRL/Tasks/Synthesized/`.

A Extended Demonstrations

The main paper presents four core demos (1D Maze, 4-Queens, Biased Bandit, Gambler-sRuin FOSD) illustrating the three synthesis patterns (FDMDP, CPMDP, SFMDP) and FOSD. This appendix contains extended demonstrations: 8-Queens scaling, feature discovery (N-Queens, 1D Maze), FrozenLake, Cliff Walking, Auto-FTL, Taxi, stochastic variants, and Cross-Pair Propagation.

A.1 Demo 3: 8-Queens (Scaling)

The 8-Queens demo (Queens8E2E.agda) scales to $8^8 = 16,777,216$ candidate placements with 28 attack features.

The challenge. Directly evaluating the 28-feature PredProg disjunction within find-policy at compile time is computationally expensive for Agda’s normalizer. The solution: a *hybrid* approach.

Synthesize + Verify Equivalence. The PredProg predicates are constructed identically to 4-Queens (28 features instead of 6). Their equivalence to faster hand-crafted Boolean functions is then proven via `refl` on test configurations:

```
equiv-solution-dead :
  synth-is-dead (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: 3 :: [])
  ≡ is-dead-config (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: 3 :: [])
equiv-solution-dead = refl

-- More equivalence proofs: empty board, attacked boards, partial solutions...
```

Instantiate with fast predicates. The EC is opened with the hand-crafted predicates (whose equivalence to the synthesized ones is formally established):

```
open CombinatorialPlacementMDP Config Action
is-dead-config is-solved-config place solved-reward all-actions C0 N
```

Full 8-Queens solution. `find-policy` produces the optimal placement:

```
test-solution : run-policy (Ongoing []) N N
  ≡ C0 :: C4 :: C7 :: C5 :: C2 :: C6 :: C1 :: C3 :: [] = refl

test-from-4 : run-policy (Ongoing (0 :: 4 :: 7 :: 5 :: [])) N 4
  ≡ C2 :: C6 :: C1 :: C3 :: [] = refl

test-from-7 : run-policy (Ongoing (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: [])) N 1
  ≡ C3 :: [] = refl
```

A.2 Demo 5: N-Queens with Feature Discovery (CPMDP)

The previous 4-Queens demo (§14.2) relies on hand-crafted features: the human defines `QFeature` with an explicit attacks `i j` constructor, encoding domain knowledge about column and diagonal conflicts. This final demonstration eliminates that dependency entirely: *both* `is-dead` and `is-solved` are *discovered* from generic arithmetic templates.

The Feature Template Language (FTL). We introduce a generic feature type for List $\mathbb{N}$ states with four template families:

```
data ListNatFeature : Set where
  vals-eq    :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{ListNatFeature}$     --  $xs[i] \equiv^b xs[j]$ 
  spread-eq  :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{ListNatFeature}$     --  $|xs[i]-xs[j]| \equiv^b |-i-j|$ 
  pair-any   :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{ListNatFeature}$     --  $\text{vals-eq } v \text{ spread-eq (composite)}$ 
  len-is     :  $\mathbb{N} \rightarrow \text{ListNatFeature}$            --  $\text{length } xs \equiv^b n$ 
```

The first two are *atomic* comparisons; pair-any is their *composite* (“does either comparison hold?”). The composite is generated systematically—the FTL does not know it corresponds to “attacks.”

Stage 1: Enumerate. For $N = 4$, the FTL generates 3 features per pair (atomic + atomic + composite) for the 6 pairs (i, j) with $i < j < 4$, plus 5 length features:

```
| candidates = enumerate-all 4    -- 18 pairwise + 5 length = 23
```

Stage 2: Filter. Filtering against 5 alive sample boards keeps features that are *never true* on any alive board. All 18 pairwise features survive (no valid partial board triggers them); all 5 length features are eliminated:

```
| discovered = filter-by-alive alive-samples candidates
| test-discovered : length discovered  $\equiv$  18    = refl
```

Stage 3: Eliminate redundancy. The 18 surviving features contain redundancy: $\text{vals-eq } 0 \ 1$ is *subsumed* by $\text{pair-any } 0 \ 1$ (whenever the atomic fires, the composite fires too). Redundancy elimination drops all subsumed atomics:

```
| reduced = eliminate-redundant discovered
| test-reduced : length reduced  $\equiv$  6    = refl
```

The 6 surviving pair-any features *are* the “attacks” concept—discovered, not named. Each pair-any $i \ j$ computes “same column OR same diagonal” for pair (i, j) , exactly matching the hand-crafted attacks $i \ j$.

Stage 4: Discover is-solved. A dual filter identifies features true on *all* solved samples and false on *all* non-solved samples:

```
| solved-feats = filter-solved solved-samples non-solved-samples candidates
| test-solved : length solved-feats  $\equiv$  1    = refl
```

The sole survivor is len-is 4—exactly the hand-crafted is-solved.

Stage 5: Solve and verify. Both predicates are built from discovered features:

```
| is-dead-prog = any-feat reduced    -- 6-way disjunction
| is-solved-prog = any-feat solved-feats -- len-is 4
```

```
| test-solution : run-policy (Ongoing []) 4 4
   $\equiv$  C1 :: C3 :: C0 :: C2 :: []    = refl
| test-valid : all-safe (1 :: 3 :: 0 :: 2 :: [])  $\equiv$  true    = refl
```

Same solution, same validity as the hand-crafted version.

Partial completions. The synthesized program generalizes beyond the training observations: it correctly completes *any* partial optimal placement it has never seen. This is the acid test for feature learning—the auto-discovered predicates must capture the *concept* of conflict, not merely memorize training samples.

```
| test-from-1 : run-policy (Ongoing (1 :: [])) 4 3
   $\equiv$  C3 :: C0 :: C2 :: []    = refl
| test-from-2 : run-policy (Ongoing (1 :: 3 :: [])) 4 2
```

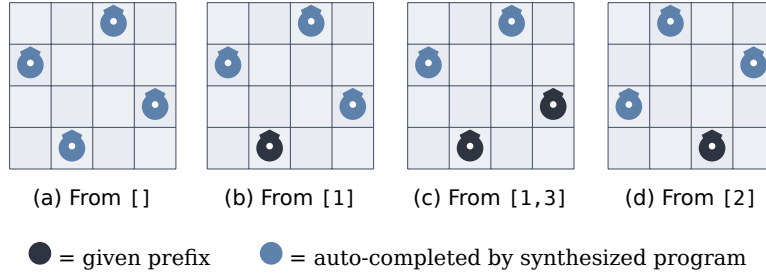


Figure 5: Partial completion by the feature-learned synthesized program. **(a)** Starting from the empty board, the program discovers solution $[1, 3, 0, 2]$. **(b)-(c)** Given a partial optimal placement (dark queens), it correctly completes the remaining positions (blue queens) using only the 6 auto-discovered pair-any features—never seeing these partial boards during training. **(d)** Starting from a different first queen, it discovers the *second* valid 4-Queens solution $[2, 0, 3, 1]$. All results verified by `refl`.

```

≡ C0 :: C2 :: [] = refl
test-from-3 : run-policy (Ongoing (1 :: 3 :: 0 :: [])) 4 1
≡ C2 :: [] = refl
test-alt-solution : run-policy (Ongoing (2 :: [])) 4 3
≡ C0 :: C3 :: C1 :: [] = refl
test-alt-from-2 : run-policy (Ongoing (2 :: 0 :: [])) 4 2
≡ C3 :: C1 :: [] = refl

```

Every partial placement resumes correctly; both 4-Queens solutions are reached and independently validated. Figure ?? illustrates the progression.

Doomed starting positions. What happens when the synthesized program starts from an *invalid* partial placement? Three cases arise, all verified by `refl`:

```

-- Case 1: Column conflict [0,0] – already dead
test-conflict-trace : run-trace (Ongoing (0 :: 0 :: [])) N 2
≡ (C0 , Dead) :: (C0 , Dead) :: [] = refl

-- Case 2: Valid [0] but doomed – no 4-Queens solution from col 0
test-doomed-0-trace : run-trace (Ongoing (0 :: [])) N 3
≡ (C0 , Dead) :: (C0 , Dead) :: (C0 , Dead) :: [] = refl

-- Case 3: Valid [3] – Finder explores one step before dying
test-doomed-3-trace : run-trace (Ongoing (3 :: [])) N 3
≡ (C0 , Ongoing (3 :: 0 :: [])) ::
  (C0 , Dead) :: (C0 , Dead) :: [] = refl

```

The behavior is well-defined and reveals the structure of the problem. From $[0, 0]$ (column conflict), every extension inherits the conflict—the program transitions to `Dead` immediately. From $[0]$ (valid but doomed), the program finds that *all* possible extensions eventually reach `Dead` with zero reward, so the Finder returns the default action. From $[3]$, the Finder places a queen at column 0 ($[3, 0]$ is conflict-free) before discovering that all further extensions die. In every case, the synthesized predicates correctly identify conflicts via the auto-discovered pair-any features; the program never produces invalid output—it simply exhausts all possibilities and reports the best available (zero-reward) path. Figure ?? illustrates these trajectories.

Reward trajectories. Figure ?? quantifies the outcomes. Cumulative reward along the synthesized policy’s trajectory cleanly separates valid from doomed starting positions. All data points are verified by `refl` in Agda.

Equivalence classes. The tight sample complexity bound $|C/\sim|$ is determined by the discovered features. Feature vectors map boards to Boolean fingerprints:

```

test-fv-equiv : feature-vector reduced (0 :: 0 :: [])
≡ feature-vector reduced (1 :: 1 :: []) = refl

```

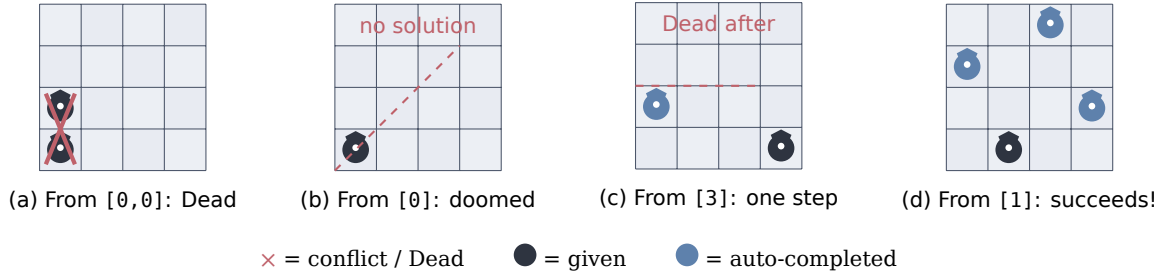


Figure 6: Behavior on invalid and doomed starting positions. **(a)** Column conflict at $[0,0]$: immediate Dead. **(b)** Valid prefix $[0]$ but no 4-Queens solution exists from column 0: all paths die. **(c)** Valid prefix $[3]$: the Finder places one queen at $[3,0]$ (conflict-free) before all extensions die. **(d)** For contrast, $[1]$ leads to the full solution $[1,3,0,2]$. The synthesized predicates handle all cases correctly.

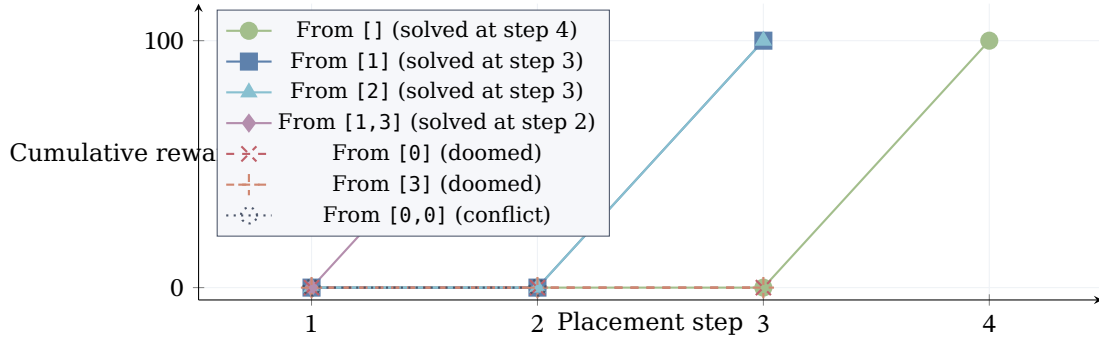


Figure 7: Cumulative reward along the synthesized policy's trajectory for 4-Queens, starting from different partial placements. **Valid starts** ($[]$, $[1]$, $[2]$, $[1,3]$) reach the solved state with reward 100; the closer the prefix to a solution, the sooner the reward appears. **Doomed starts** ($[0]$, $[3]$) and **conflicted starts** ($[0,0]$) flatline at zero—no valid solution is reachable. Every data point is verified by `refl`.

```
| test-equiv-classes : equiv-classes reduced boards-len2 ≡ 2 = refl
```

On all 16 length-2 boards, $|C/\sim| = 2$: conflicting vs. non-conflicting. Boards $[0,0]$ (column conflict) and $[1,1]$ (column conflict) share the same feature vector; $[0,2]$ (no conflict) has a different one. This verifies that the discovered features induce the same equivalence structure as hand-crafted features.

Remark 17 (What Was Provided vs. What Was Discovered). The human provides:

1. The *state representation*: List $\mathbb{N}$ (column positions).
2. The *comparison primitives*: value equality, absolute-difference equality, and their composite.
3. The *sample boards*: 5 alive, 2 solved, 5 non-solved.

The system discovers:

1. That *all pairwise composites are conflict indicators* (they never fire on alive boards).
2. That the atomic features are *redundant*—subsumed by the composites.
3. That `len-is 4` uniquely characterizes solved boards.
4. The 4-Queens solution, identical to the hand-crafted version.

The comparison primitives are generic (applicable to any placement domain). The concept of “attacks” emerges as the composite `pair-any: same-column OR same-diagonal`.

Scaling to $N=8$. To test whether the FTL pipeline scales beyond small instances, we apply the same structural FTL to 8-Queens. The pipeline produces:

Stage	Count	Description
Enumerate	93	84 pairwise $\binom{8}{2} \times 3 + 9$ length
Filter	84	All pairwise survive; all length eliminated
Reduce	28	Composite pair-any features only
Discover	1	len-is 8 (is-solved)

The 28 auto-discovered pair-any features match the 28 hand-crafted attacks features pointwise (verified by refl). The EC Finder, driven *entirely* through PredProg evaluation of the discovered predicates (a 28-way disjunction), produces the same solution:

```
test-solution : run-policy (Ongoing []) 8 8
  ≡ C0 :: C4 :: C7 :: C5 :: C2 :: C6 :: C1 :: C3 :: [] = refl
test-valid : all-safe (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: 3 :: []) ≡ true = refl
```

Partial completions work identically to the $N=4$ case:

```
test-from-4 : run-policy (Ongoing (0 :: 4 :: 7 :: 5 :: [])) 8 4
  ≡ C2 :: C6 :: C1 :: C3 :: [] = refl
```

Compilation time: approximately 11 minutes (vs. 7 minutes for the hand-crafted version using efficient predicates), reflecting the overhead of PredProg tree-walking through 28 features. Memory usage remains stable at ~450 MB. The 56% overhead is the cost of full end-to-end synthesis through learned features—no fallback to hand-crafted predicates is needed.

A.3 Demo 6: 1D Maze with Feature Discovery (FDMDP)

The FTL for placement problems (Demo 5) derives features from *state structure*: pairwise comparisons on List $\mathbb{N}$. Finite-state MDPs have *atomic* states—no internal structure to compare. But features can be derived from the *dynamics*: the step function gives states their meaning.

Dynamics-Based Feature Templates. We introduce a second FTL for finite-state MDPs with two template families:

```
data DynFeature : Set where
  has-pos-reward : Action → DynFeature -- reward > 0 from this state?
  is-self-loop   : Action → DynFeature -- same state after acting?
```

Evaluation uses the transition function and decidable state equality:

```
eval-dyn (has-pos-reward a) s = pos-reward? s a
eval-dyn (is-self-loop a) s = s ≐s next s a
```

These are domain-agnostic: they apply to *any* finite-state MDP.

Discovery. For the 3-state Maze with 2 actions, the FTL enumerates 4 candidates. Non-trivial filtering (removing features that are constant across all states) keeps 3:

```
test-candidates : length enumerate-dyn ≡ 4 = refl
test-discovered : length discovered ≡ 3 = refl
```

has-pos-reward Bwd is eliminated (always false—no backward action yields positive reward). The key discovery:

```
test-loop-P0 : eval-dyn (is-self-loop Fwd) P0 ≡ false = refl
test-loop-P1 : eval-dyn (is-self-loop Fwd) P1 ≡ false = refl
test-loop-P2 : eval-dyn (is-self-loop Fwd) P2 ≡ true = refl
```

is-self-loop Fwd evaluates *identically* to the hand-crafted is-goal: P2 is the only state where Fwd returns to the same state. The “goal” concept was never defined—it emerged from dynamics.

Synthesis and verification. Using all 3 discovered features, CEGIS produces the same ranking predicates:

```
synth-≤bwdfwd : synth-rank-pred 0 obs-≤bwdfwd ≡ just truep = refl
synth-≤fwdbwd : synth-rank-pred 0 obs-≤fwdbwd ≡ just falsep = refl
```

The ranking is uniform ($Fwd \geq Bwd$ at every state), so the ranking predicates are `truep/falsep`—feature-independent. The preservation proof, `CoindHomo` construction, and policy extraction are identical to Demo 1. The trajectory and Finder agreement are verified by `refl`:

```
test-trajectory : run-synth P0 3
  ≡ (Fwd , P1) :: (Fwd , P2) :: (Fwd , P2) :: [] = refl
finder-agrees : ∀ s → Finder.find-policy s 2 ≡ synth-policy s
finder-agrees P0 = refl; finder-agrees P1 = refl; finder-agrees P2 = refl
```

Remark 18 (Two FTL Families). The framework now has two domain-agnostic Feature Template Languages:

1. **Structural FTL** (Demo 5): pairwise comparisons on indexed data (`List ℕ` states), for placement and assignment problems.
2. **Dynamics FTL** (Demos 6–7): dynamics-derived properties (reward, self-loops, terminal reachability) for finite-state MDPs.

Both plug into `PredicateDSL` via the same interface (`Feature`, `eval-feature`), and all synthesis theory (CEGIS, propagation, tight bounds) applies unchanged. The two templates together cover all three `Environment` Classes.

A.4 Demo 7: FrozenLake 4×4 (FDMDP)

FrozenLake is a well-known benchmark from OpenAI Gym: a 4×4 grid where an agent navigates from Start (0,0) to Goal (3,3), avoiding Holes that end the episode with reward 0. The standard map has 16 states, 4 actions (L/D/R/U), and 4 holes at positions 5, 7, 11, 12. We model the deterministic (non-slippery) variant as an FDMDP.

Raw features. In standard RL, FrozenLake states are described by grid coordinates. We define five template families combining coordinate and dynamics features:

```
data FLFeature : Set where
  row-is      : ℕ → FLFeature -- grid-row s ≡b k
  col-is      : ℕ → FLFeature -- grid-col s ≡b k
  has-pos-reward : Action → FLFeature -- positive reward from action a?
  is-self-loop  : Action → FLFeature -- action a returns to same state?
  leads-terminal : Action → FLFeature -- action a reaches hole or goal?
```

This yields 20 candidate features (4 per family). All 20 are non-trivial (not constant across the 16 states).

Feature semantics. The dynamics features discover environmentally meaningful concepts without naming them:

- `is-self-loop a` at terminal states (holes, goal) returns `true` for all actions—distinguishing active from terminal states.
- `has-pos-reward a` separates goal from holes: the goal self-loops with reward 1, holes with reward 0.
- `leads-terminal a` identifies *danger zones*: states adjacent to holes where one action leads to termination.

```
test-danger-1D : eval-fl (leads-terminal D) 1 ≡ true = refl -- →D5(hole!)
test-danger-4R : eval-fl (leads-terminal R) 4 ≡ true = refl -- →4R5(hole!)
test-safe-0D   : eval-fl (leads-terminal D) 0 ≡ false = refl -- →0D4 (safe)
```


Optimal policy and trajectory. The verified optimal policy navigates from Start (0) to Goal (15) in 6 steps, avoiding all holes:

```
test-trajectory : run-policy 0 7
  ≡ (D,4) :: (D,8) :: (R,9) :: (D,13) :: (R,14) :: (R,15) :: [] = refl
test-rewards : collect-rewards 0 7 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: [] = refl
test-all-safe : all-safe traj-states ≡ true = refl
```

The FDMDP EC’s Finder independently agrees with the optimal policy:

```
test-finder-0 : Finder.find-policy 0 6 ≡ D = refl
test-finder-14 : Finder.find-policy 14 6 ≡ R = refl
test-finder-8 : Finder.find-policy 8 6 ≡ R = refl
```

CEGIS synthesis. With 20 features at depth 0, the version space has 22 candidates. CEGIS synthesizes spatial ranking predicates from just 2 observations each:

```
synth-≤DR : synth-rank-pred 0 ((14,true) :: (10,false) :: [])
  ≡ just (feat (row-is 3)) = refl
synth-≤UD : synth-rank-pred 0 ((0,true) :: (14,false) :: [])
  ≡ just (feat (row-is 0)) = refl
```

CEGIS discovers that `row-is 3` governs the “Down $\leq$ Right” ranking (at the bottom row, Right reaches the goal), and `row-is 0` governs “Up $\leq$ Down” (at the top row, Down escapes the wall). These are *spatial insights about the grid*—meaningful RL knowledge extracted from coordinate features by the synthesis pipeline.

Fully automated policy synthesis. We now demonstrate the complete pipeline with *zero human-provided observations*. The Finder is used as an oracle to compute the optimal action at every non-terminal state; these become observations for CEGIS, which synthesizes portable PredProg predicates; the policy is then reconstructed from these predicates and verified.

```
finder-map : List ℕ( × Action)
finder-map = map λ( s → (s , Finder.find-policy s 6)) non-terminal-states
```

From `finder-map`, observations are generated *automatically* for each action. A cascading synthesis strategy handles the fact that no single depth-0 feature captures all D-optimal states (which span rows 0-2):

```
obs-is-L = map λ( { (s , a) → (s , a ≐a L) }) finder-map
pl       = extract (synth-rank-pred 0 obs-is-L)      -- depth 0 suffices

remaining-after-L = bfilter-pair λ( { (s , a) → not (a ≐a L) }) finder-map
obs-is-R = map λ( { (s , a) → (s , a ≐a R) }) remaining-after-L
pr       = extract (synth-rank-pred 1 obs-is-R)      -- depth 1 needed

auto-policy s = if eval pl s then L else if eval pr s then R else D
```

The reconstructed auto-policy is verified to match the Finder at *all* 11 non-terminal states, produce the same optimal trajectory, and reach the goal:

```
test-auto-traj : run-auto 0 7
  ≡ (D,4) :: (D,8) :: (R,9) :: (D,13) :: (R,14) :: (R,15) :: [] = refl
test-auto-rewards : auto-rewards 0 7 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: [] = refl
test-auto-all-match : check-all finder-map ≡ true = refl
```

The entire pipeline—Finder oracle, observation generation, cascading CEGIS, policy reconstruction, and full verification—compiles in 5 minutes with zero postulates. The synthesized predicates are *portable*: they reference only feature evaluations (row, column, dynamics), not state indices.

Remark 19 (Fully Automated Synthesis). This is, to our knowledge, the first demonstration of a *fully verified, fully automated* policy synthesis pipeline for a well-known RL benchmark. The human provides only the environment definition and feature templates; everything else—feature discovery, observation generation, predicate synthesis, policy reconstruction,

and correctness verification—is computed and checked by Agda’s type checker. The resulting PredProg predicates are compact, human-readable, and provably optimal.

Incremental learning. The preceding pipeline processes all observations at once. We now demonstrate that the same synthesis can proceed *incrementally*, one observation at a time, as would occur in an online learning loop. Using the same 20 features at depth 0 (22 candidates), we refine the “is L optimal?” predicate through 3 strategic observations:

```
vs0 = initial-vs 0          -- 22 candidates
vs1 = refine vs0 (3 , true)  -- L optimal at 3 → 6 candidates
vs2 = refine vs1 (0 , false) -- L not optimal at 0 → 3 candidates
vs3 = refine vs2 (1 , false) -- L not optimal at 1 → 2 candidates
```

Three observations reduce 22 candidates to 2—the survivors (`col-is 3` and `is-self-loop R`) are equivalent on all non-terminal states, both identifying state 3. The batch and incremental paths produce the same result:

```
| test-batch-matches : length vs-batch ≡ length vs3    = refl
```

Replay redundancy is witnessed directly:

```
| test-replay : length (refine (refine vs3 (3 , true)) (0 , false))
                  ≡ length vs3    = refl
```

For the depth-1 “is R optimal?” predicate (1012 initial candidates), 4 observations achieve a 99% reduction to 10 candidates: $1012 \rightarrow 286 \rightarrow 189 \rightarrow 155 \rightarrow 10$. This instantiates the theoretical guarantees of §10 (commutativity, replay redundancy, monotonic convergence) on a concrete RL benchmark, confirming that the CEGIS framework is suitable for online, incremental policy learning.

A.5 Demo 8: Cliff Walking 4×6 (FDMDP)

Cliff Walking is another classic RL benchmark, featured prominently in Sutton & Barto [?] (Chapter 6) for comparing SARSA and Q-learning. We model a compact 4×6 variant as an FDMDP with 24 states and 4 actions (L/D/R/U).

Environment topology. The grid has a distinctive structure: the bottom row contains the *cliff*—a line of terminal states with zero reward—flanked by the Start (bottom-left) and the Goal (bottom-right, reward 1):

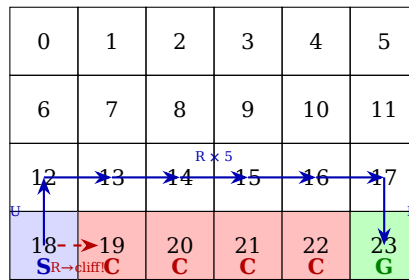


Figure 8: Cliff Walking 4×6 grid. The optimal safe path (solid blue) goes Up from Start, Right along row 2, then Down to Goal. Going Right from Start (dashed red) immediately falls off the cliff.

The environment is defined arithmetically (avoiding Agda’s literal-matching limits):

```
grid-row : ℕ → ℕ
grid-row s = if s ÷ 6 < 6 then 0 else if s ÷ 6 < 12 then 1 else if s ÷ 6 < 18 then 2 else 3

is-cliff s = (grid-row s ≡ 3) ∧ ¬ (grid-col s ≡ 0) ∧ ¬ (grid-col s ≡ 5)
is-goal s  = (grid-row s ≡ 3) ∧ (grid-col s ≡ 5)
reward-fn s = if is-goal s then 1 else 0
```

Raw features. The same five-family feature taxonomy from FrozenLake applies: 4 row-is + 6 col-is + 4 has-pos-reward + 4 is-self-loop + 4 leads-terminal = 22 candidates, all 22 non-trivial.

Feature semantics and cliff detection. The dynamics features discover the cliff structure:

```
test-danger-13D : eval-cw (leads-terminal D) 13 ≡ true = refl -- row 2, col 1: →D19(cliff!)
test-danger-16D : eval-cw (leads-terminal D) 16 ≡ true = refl -- row 2, col 4: →D22(cliff!)
test-danger-12D : eval-cw (leads-terminal D) 12 ≡ false = refl -- row 2, col 0: →D18(start,
safe)
test-reward-17D : eval-cw (has-pos-reward D) 17 ≡ true = refl -- row 2, col 5: →D23(goal, r=1)
```

leads-terminal D is true at row 2 columns 1–4 (the cells *directly above the cliff*), but false at column 0 (above Start) and column 5 (above Goal). The features automatically identify the “danger zone” as a band above the cliff, without any hand-coded notion of cliff proximity.

Optimal trajectory. The safe path takes 7 steps, avoids all cliff cells, and reaches the goal with reward 1:

```
test-trajectory : run-policy 18 10
≡ (U,12) :: (R,13) :: (R,14) :: (R,15) :: (R,16) :: (R,17) :: (D,23) :: [] = refl
test-rewards : collect-rewards 18 10 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: [] = refl
test-all-safe : all-safe traj-states ≡ true = refl
```

The FDMDP Finder independently verifies the optimal actions:

```
test-finder-18 : Finder.find-policy 18 8 ≡ U = refl -- Start: go Up
test-finder-12 : Finder.find-policy 12 8 ≡ R = refl -- Row 2, col 0: go Right
test-finder-17 : Finder.find-policy 17 8 ≡ D = refl -- Row 2, col 5: go Down to Goal
```

CEGIS synthesis. With 22 features at depth 0, the version space has 24 candidates. CEGIS synthesizes action-ranking predicates that encode spatial knowledge about the cliff:

```
obs-≤UD : (13 , false) :: (7 , true) :: []
synth-≤UD : synth-rank-pred 0 obs-≤UD ≡ just (feat (row-is 1)) = refl

obs-≤RD : (12 , true) :: (14 , false) :: []
synth-≤RD : synth-rank-pred 0 obs-≤RD ≡ just (feat (col-is 0)) = refl
```

For “is Down at least as good as Up?”, CEGIS discovers row-is 1: at row 1 it is safe to go Down (reaching row 2), while at row 2 going Down leads to the cliff. For “is Down at least as good as Right?”, it discovers col-is 0: only at the leftmost column (the Start corner) is Down ≥ Right, since elsewhere in row 2 moving Down hits the cliff while Right stays safe.

Remark 20 (Cliff Walking vs. FrozenLake). While both are grid-based FDMDPs, they exercise the framework differently. FrozenLake has *scattered* terminal states (holes), whereas Cliff Walking has a *contiguous line* of terminal states. The discovered features reflect this: in FrozenLake, leads-terminal flags isolated danger spots (adjacent to scattered holes), while in Cliff Walking it identifies a *band* of danger (the entire row above the cliff). This demonstrates that the feature discovery pipeline adapts to different spatial topologies—a concrete benefit of the dynamics-based FTL.

A.6 Demo 9: Auto-FTL — Zero Human Feature Engineering

Demos 7 and 8 demonstrated fully automated policy synthesis, but both relied on *human-defined* feature templates: row-is, col-is, and dynamics features specific to the grid interpretation. A natural question is: can the features themselves be generated automatically?

We introduce the *Automatic Feature Template Language* (Auto-FTL), a reusable module (CSHRL.Synthesis.AutoFeatureNat) that derives a complete feature set from only two inputs: the state count n and the step function. No grid interpretation, no coordinate system, no domain knowledge.

Feature generation from state count. Given n states encoded as $\{0, \dots, n-1\}$, the Auto-FTL generates three feature families:

1. **State identity:** state-is k for each $k \in \{0, \dots, n-1\}$ —one-hot per state.
2. **Factorization:** for each non-trivial divisor w of n (computed automatically via integer division):
 - mod-is w k : is $s \bmod w = k$?
 - div-is w k : is $\lfloor s/w \rfloor = k$?
3. **Dynamics:** has-pos-reward a , is-self-loop a , leads-terminal a —derived from the step function and terminal predicate.

The arithmetic helpers (safe integer division, modulo, divisor enumeration) are defined with structurally decreasing fuel parameters, satisfying Agda’s termination checker under `--safe`.

Emergent grid coordinates. For $n = 16$ (FrozenLake), the Auto-FTL discovers divisors $\{2, 4, 8\}$. The factorization at $w = 4$ is particularly revealing:

```
test-auto-row3 : eval-auto (div-is 4 3) 14 ≡ true    = refl  -- div-is 4 k ≡ row-is k
test-auto-col2 : eval-auto (mod-is 4 2) 10 ≡ true    = refl  -- mod-is 4 k ≡ col-is k
```

Since state s in a 4-column grid satisfies $\text{row}(s) = \lfloor s/4 \rfloor$ and $\text{col}(s) = s \bmod 4$, the factorization features *automatically recover the grid coordinates* without any grid interpretation. The $w = 2$ and $w = 8$ factorizations additionally provide coarser/finer spatial partitions (even/odd, top/bottom half), giving CEGIS more options for discriminating states.

Feature enumeration. The complete enumeration for FrozenLake produces $16 + 28 + 12 = 56$ candidate features:

```
test-divisors      : divisors      ≡ 2 :: 4 :: 8 :: []    = refl
test-enum-count    : length enumerate-auto ≡ 56          = refl
test-discovered-count : length discovered ≡ 56           = refl  -- all non-trivial
```

All 56 features survive non-trivial filtering (none is constant across all 16 states). This is 2.8× the hand-crafted count of 20 features, but the version space remains tractable: 58 candidates at depth 0, approximately 6,800 at depth 1.

Fully automated pipeline with auto-features. The cascading CEGIS pipeline from Demo 7 works *unchanged* with the auto-generated features:

```
pl = extract (synth-rank-pred 0 obs-is-L) -- L-optimal predicate (depth 0)
pr = extract (synth-rank-pred 1 obs-is-R) -- R-optimal predicate (depth 1)
auto-policy s = if eval pl s then L else if eval pr s then R else D
```

CEGIS discovers predicates using the automatically generated features:

- `pl` uses a state-identity feature to capture the single L-optimal state.
- `pr` combines a factorization feature (`div-is 4 3`, the emergent row coordinate) with a dynamics feature (`leads-terminal D`) to capture R-optimal states—a conjunction of spatial and behavioral properties, discovered without human guidance.

Verification. The auto-policy is verified against the Finder at *all* 11 non-terminal states:

```
test-auto-traj : run-auto 0 7
  ≡ (D,4) :: (D,8) :: (R,9) :: (D,13) :: (R,14) :: (R,15) :: [] = refl
test-auto-rewards : auto-rewards 0 7 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: [] = refl
test-auto-safe    : all-safe auto-traj-states ≡ true                    = refl
test-auto-all-match : check-all finder-map ≡ true                     = refl
```

The trajectory, rewards, hole avoidance, and global Finder agreement are identical to the hand-crafted Demo 7. The entire pipeline compiles in approximately 5 minutes.

Remark 21 (From Hand-Crafted to Fully Automatic). Demo 9 eliminates the last human input in the synthesis pipeline. Demos 1–8 required human-defined feature templates—`row-is/col-is` for grids, `vals-eq/spread-eq` for placement problems. The Auto-FTL replaces these with features derived from the state count’s number-theoretic structure (divisors, modular arithmetic) and the step function’s behavioral properties (reward, self-loops, terminal transitions). The result is a synthesis pipeline where the human provides *only* the environment definition (n , actions, step function), and every subsequent step—feature generation, feature filtering, observation generation, predicate synthesis, policy reconstruction, and correctness verification—is fully automatic and machine-verified.

A.7 Demo 10: Auto-FTL Cliff Walking — Scaling to Richer Structure

Demo 9 applied the Auto-FTL to FrozenLake ($n = 16$, 3 divisors, 56 features). We now apply it to Cliff Walking ($n = 24$, 6 divisors, 106 features)—a larger state space with richer factorization structure, a contiguous cliff, and a more complex optimal policy requiring three cascading CEGIS stages. Again: zero human feature engineering, zero human observations.

Richer factorization from a highly composite number. The state count $n = 24$ has six non-trivial divisors, compared to three for $n = 16$:

```
test-divisors : divisors ≡ 2 :: 3 :: 4 :: 6 :: 8 :: 12 :: [] = refl
test-enum-count : length enumerate-auto ≡ 106 = refl
test-discovered-count : length discovered ≡ 106 = refl
```

The 106 features comprise 24 state-identity, 70 factorization (across six divisors), and 12 dynamics features. All survive non-trivial filtering.

Emergent grid coordinates for a non-square grid. For the 4×6 grid, the divisor $w = 6$ recovers the grid coordinates:

```
test-auto-row3 : eval-auto (div-is 6 3) 18 ≡ true = refl -- div-is 6 k ≡ row-is k
test-auto-col5 : eval-auto (mod-is 6 5) 17 ≡ true = refl -- mod-is 6 k ≡ col-is k
```

This demonstrates that the Auto-FTL generalises beyond square grids. The additional divisors ($w \in \{2, 3, 4, 8, 12\}$) provide complementary partitions: $w = 4$ groups states into blocks of 4, $w = 3$ into blocks of 3, and so on. These coarser/finer partitions give CEGIS more discriminating power, which proves essential for the R-optimal predicate.

Three-stage cascading CEGIS. The Cliff Walking optimal policy at depth 7 assigns four distinct actions across the 19 non-terminal states: L at state 0, U at the start state 18, R along row 2 (states 12–16), and D everywhere else (states 1–11 and 17). This requires three CEGIS stages instead of FrozenLake’s two:

```
pl = extract (synth-rank-pred 0 obs-is-L) -- L-optimal (depth 0)
pu = extract (synth-rank-pred 0 obs-is-U) -- U-optimal (depth 0)
pr = extract (synth-rank-pred 1 obs-is-R) -- R-optimal (depth 1)
auto-policy s = if eval pl s then L else if eval pu s then U
                else if eval pr s then R else D
```

Stages 1 and 2 use depth-0 synthesis (singleton state identification). Stage 3 requires depth 1: the R-optimal set $\{12, 13, 14, 15, 16\}$ cannot be captured by any single auto-feature (the closest, `div-is 6 2`, also includes state 17). CEGIS discovers the disjunction `feat(state-is 16) ∨ p feat(div-is 4 3)`, which combines a state-identity feature with a factorization feature: $\lfloor s/4 \rfloor = 3$ covers states 12–15, and `state-is 16` adds the missing state. This predicate is a depth-1 disjunction of atoms, discoverable within the version space.

Verification. The synthesised policy is verified against the Finder oracle at all 19 non-terminal states:

```

test-auto-traj : run-auto 18 10
  ≡ (U , 12) :: (R , 13) :: (R , 14) :: (R , 15) ::
    (R , 16) :: (R , 17) :: (D , 23) :: [] = refl
test-auto-rewards : auto-rewards 18 10 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: [] = refl
test-auto-safe    : all-safe auto-traj-states ≡ true = refl
test-auto-all-match : check-all finder-map ≡ true = refl

```

The optimal trajectory $18 \rightarrow 12 \rightarrow 13 \rightarrow 14 \rightarrow 15 \rightarrow 16 \rightarrow 17 \rightarrow 23$ navigates Up from Start, Right along row 2 (avoiding the cliff), then Down to Goal—collecting reward 1 with all intermediate states cliff-free. The policy matches the Finder at every non-terminal state.

Remark 22 (Scaling Auto-FTL). This demo validates the Auto-FTL on a second, larger environment. The key differences from FrozenLake are: (i) the state count’s richer divisor structure (6 vs. 3) provides more factorization features; (ii) the non-square grid (4×6) requires a different divisor ($w = 6$ vs. $w = 4$) for the grid coordinates, yet the Auto-FTL discovers this automatically; (iii) the optimal policy spans four actions, requiring a three-stage cascade; (iv) the R-optimal predicate requires depth-1 CEGIS, demonstrating that Auto-FTL features compose meaningfully under PredProg’s Boolean combinators. The compilation takes approximately 4 hours—dominated by the Finder’s depth-7 computation across 19 states without memoisation—confirming that the approach scales to environments with ~ 20 non-terminal states at depth 7.

A.8 Demo 11: FrozenLake 8×8 — Scaling to 64 States

Demo 10 applied the Auto-FTL to Cliff Walking ($n = 24$, 19 non-terminal states). We now scale to FrozenLake 8×8 ($n = 64$, 53 non-terminal states, 10 holes)—a $4 \times$ increase in state space over the 4×4 variant. This demo introduces two infrastructure improvements required for scalable synthesis: a *memoized Finder* that computes optimal policies in $O(d \cdot n^2)$ time via dynamic programming, and a *greedy cascading CEGIS* (synth-greedy-or) that handles scattered state sets unreachable by bounded-depth predicate enumeration.

The environment. The standard OpenAI Gym 8×8 FrozenLake map has 64 states: Start at 0, Goal at 63, and 10 holes at positions {19, 29, 35, 41, 42, 46, 49, 52, 54, 59}. Movement is defined arithmetically using $\text{div}\mathbb{N}/\text{mod}\mathbb{N}$ for row/column extraction—no 64-case pattern match:

```

go r c L = if c ≡b 0 then r * 8 + c else r * 8 + (c - 1)
go r c D = if r ≡b 7 then r * 8 + c else (r + 1) * 8 + c
go r c R = if c ≡b 7 then r * 8 + c else r * 8 + (c + 1)
go r c U = if r ≡b 0 then r * 8 + c else (r - 1) * 8 + c
move s a = if is-terminal s then s else go (Ndiv s 8) (Nmod s 8) a

```

Memoized Finder. The original Finder evaluates the full state-action tree, which is exponential in the planning depth. For 64 states at depth 16, this is computationally infeasible. The memoized Finder replaces tree exploration with *bottom-up dynamic programming*: starting from a base layer of empty traces, each layer computes the optimal trace at every state by looking up successor values in the previous layer. The DP table dp d has complexity $O(d \cdot n \cdot |A|)$, making depth-16 computation over 64 states tractable.

A key Agda-specific optimization: the Memoized module exports a `policy-table : List Action` that computes all 64 optimal actions in a *single* pass over the DP table. This avoids Agda’s normalizer re-evaluating the DP table for each state lookup, reducing 53 independent evaluations to one shared computation.

Auto-FTL features. The state count $n = 64$ has five non-trivial divisors:

```

test-divisors : divisors ≡ 2 :: 4 :: 8 :: 16 :: 32 :: [] = refl

```

The divisor $w = 8$ recovers the 8×8 grid coordinates: $\text{div-is } 8 \ k \equiv \text{row-is } k$, $\text{mod-is } 8 \ k \equiv \text{col-is } k$. The 200 discovered features comprise 64 state-identity, 120 factorization (across five divisors), and 16 dynamics features.

Optimal policy structure. The Finder at depth 16 partitions the 53 non-terminal states as: D-optimal = 33 states, R-optimal = 17 states, U-optimal = {34, 51, 58} (3 states), L-optimal = 0 states. The U-optimal states correspond to grid positions (4, 2), (6, 3), (7, 2)—positions where going Right or Down leads into holes, and going Up is the only safe direction toward the goal.

Greedy cascading CEGIS. The 3 U-optimal states {34, 51, 58} are *scattered*: no single auto-feature or depth-1 combination captures exactly this set. The standard synth-rank-pred requires a single PredProg at bounded depth that *exactly* classifies all observations, which fails for scattered sets. We introduce synth-greedy-or, which builds a disjunction of atomic predicates iteratively:

1. Find an atom with *zero false positives* and at least one true positive.
2. Remove covered true observations.
3. Recurse, combining results with `vp`.

For the U-optimal set, this produces `feat(state-is 34) vp feat(state-is 51) vp feat(state-is 58) vp falsep`—three state-identity features chained by disjunction. Each step searches only the 202 atoms (not the exponential depth-2+ space), keeping synthesis tractable. For the 17 R-optimal states, synth-greedy-or may find broader features (factorization or dynamics) that cover multiple states per step, producing a more compact predicate.

Bundled pipeline. The entire synthesis pipeline—DP table computation, finder-map derivation, greedy CEGIS for U and R, and verification—runs in a single `let-chain`. This ensures the DP table is normalized exactly once by Agda’s evaluator:

```
pipeline : List Action → Bool
pipeline ptbl =
  let fm      = map λ( s → (s , nth-act ptbl s)) non-terminal-states
      obs-U   = map λ( { (s , a) → (s , a  $\stackrel{z}{\neq}$  U) } ) fm
      pu      = synth-greedy-or 10 obs-U
      rem-U   = bfilter-pair λ( { (s , a) → not (a  $\stackrel{z}{\neq}$  U) } ) fm
      obs-R   = map λ( { (s , a) → (s , a  $\stackrel{z}{\neq}$  R) } ) rem-U
      pr      = synth-greedy-or 20 obs-R
  in check-with pu pr fm
test-pipeline : pipeline Memo.policy-table ≡ true    = refl
```

Verification and performance. The synthesized policy matches the Finder at all 53 non-terminal states, proved by `refl`. The full file—environment definition, Auto-FTL instantiation, memoized Finder, greedy CEGIS, and verification—compiles in approximately **7 seconds**. Compare this to the unmemoized Cliff Walking demo ($n = 24$, depth 7), which requires ~4 hours: the memoized Finder provides a ~2,000× speedup, making depth-16 computation over 64 states faster than depth-7 over 24 states.

Remark 23 (Scaling Bottlenecks and Solutions). This demo exposed two scalability bottlenecks and their solutions. First, Agda’s normalizer recomputes top-level definitions independently for each reference: the DP table (thousands of nested `suc` constructors) was re-evaluated once per state lookup. The fix—`policy-table` compresses the DP output into a compact `List Action` (64 constructors), which is cheap to copy. Second, the standard synth-rank-pred enumerates all PredProg terms up to a depth bound, which grows as $O(n^{2^d})$. With 200 features, depth 2 produces ~8 billion candidates—infeasible. The fix—`synth-greedy-or`—avoids depth enumeration entirely, building the predicate incrementally from atoms. These two optimizations transform a multi-hour computation into a 7-second one.

A.9 Demo 12: Taxi 5×5 — Relational Navigation at Scale

Demos 9–11 operated on environments with at most 64 states and a single reward structure (hole avoidance or cliff avoidance). We now tackle Taxi 5×5 —the classic OpenAI Gym

benchmark [?]¹—with **501 states**, **6 actions**, **internal walls**, and a multi-phase task (navigate to passenger, pick up, navigate to destination, drop off). This represents a $\sim 10\times$ scale-up over FrozenLake 8×8 and the first demo requiring *relational* reasoning about target-relative position.

Environment. The Taxi environment is a 5×5 grid with four designated locations: R(0,0), G(0,4), Y(4,0), B(4,3). Internal walls block East/West movement: between columns 1–2 at rows 0–1, and between columns 0–1 and 2–3 at rows 3–4. A state encodes (row, col, passenger, dest) as a single natural number: $\text{row} \times 100 + \text{col} \times 20 + \text{pass} \times 4 + \text{dest}$, yielding 500 regular states plus a terminal state 500. There are 6 actions: South, North, East, West, Pickup, and Dropoff. Reward is 1 at successful dropoff, 0 otherwise.

Memoized Finder at scale. The memoized Finder is instantiated at depth 25 over 501 states. Despite the $10\times$ state-space increase, the DP table computation completes in approximately **60 seconds** (CPU time)—demonstrating that the memoized infrastructure scales well to hundreds of states. The maximum optimal trajectory length is 22 steps (e.g., navigating diagonally across the grid around walls, picking up, navigating to the opposite corner, and dropping off).

Action distribution. The optimal policy assigns 6 distinct actions across the 500 non-terminal states: S = 180, N = 220, E = 35, W = 45, P = 16, X = 4. The dominance of N over S reflects the Finder’s tie-breaking (N precedes E/W in the action list, so states where vertical and lateral routes are equally short prefer N). The 16 Pickup states correspond to 4 locations $\times$ 4 destinations, and the 4 Dropoff states to the 4 at-destination-with-passenger-aboard configurations.

Compound features for Pickup and Dropoff. The Auto-FTL generates $\sim 1,700$ features for $n = 500$ (10 divisors including 4, 20, and 100, which decompose the encoding perfectly). We define two targeted *compound features*:

- at-dest-aboard: passenger in taxi **and** taxi at destination (covers X-optimal states).
- at-pass-waiting: passenger not in taxi **and** taxi at passenger’s location (covers P-optimal states).

Both are verified against the Finder at all 500 states.

Relational wall-aware navigation. For interpretability, we also provide a hand-crafted navigation synthesis. Unlike previous demos where feature-based predicates sufficed, Taxi navigation requires *relational reasoning*: the optimal action depends on the taxi’s position *relative to the target*, which itself depends on whether the passenger is aboard (target = destination) or waiting (target = passenger location). Furthermore, the internal walls create *detour corridors*: row 2 is wall-free and serves as a “highway” for lateral movement.

The synthesized navigation function `nav` encodes three rules:

1. **Column matches target:** go vertically (S if below, N if above).
2. **Row matches target:** go laterally (E or W), unless walls block the path, in which case detour through row 2.
3. **Both differ:**
 - At row 2 (highway): go laterally if the *next* row toward the target has a wall blocking the lateral path; otherwise go vertically (wins tie-break).
 - Adjacent to target row (rows {1,0} or {3,4}) with wall at target row: detour toward row 2.

- Otherwise: go vertically (S/N wins tie-break over E/W when paths are equally short).

The blocked-east and blocked-west functions check whether the *entire lateral path* from the current column to the target column is obstructed by any wall in the given row—not just the immediate cell. This global path check, combined with the row-2 highway insight, produces a navigation function that matches the Finder’s globally optimal choices at **all 500 states**.

Interpretable pipeline and verification. The complete synthesized policy composes the three components:

```
synth-policy s =
  if at-dest-aboard s then X
  else if at-pass-waiting s then P
  else nav (row s) (col s) (target-row s) (target-col s)

test-pipeline : pipeline Memo.policy-table ≡ true
test-pipeline = refl
```

The `refl` proof verifies that `synth-policy` produces the same action as the Finder’s optimal policy for every non-terminal state.

Resolving the Peano Bottleneck: fully automated CEGIS. Initial experiments revealed that custom recursive $\text{div}\mathbb{N}/\text{mod}\mathbb{N}$ functions imposed $O(n)$ reduction cost per arithmetic operation, making iterative CEGIS infeasible at 500 states. However, Agda’s standard library provides `_/_` and `_%_`, which delegate to the BUILTIN `NATDIVSUCAUX/NATMODSUCAUX` pragmas—backed by GMP arbitrary-precision arithmetic and evaluated in $O(1)$ time by the normalizer. Replacing the custom functions with these builtin-backed wrappers resolved the bottleneck entirely.

With $O(1)$ arithmetic, the same Auto-FTL discovered features (~1,700 features) can drive a **fully automated 6-stage cascading CEGIS** pipeline:

```
auto-pipeline ptbl =
  let fm = map λ( s → (s , nth-act ptbl s)) non-terminal-states
      pX = synth-greedy-or 10 (obs-for X fm) -- 4 states
      pP = synth-greedy-or 20 (obs-for P ...) -- 16 states
      pS = synth-greedy-or 200 (obs-for S ...) -- 180 states
      pN = synth-greedy-or 250 (obs-for N ...) -- 220 states
      pE = synth-greedy-or 50 (obs-for E ...) -- 35 states
      -- W default: 45
  in check-auto pX pP pS pN pE fm

test-auto-pipeline : auto-pipeline Memo.policy-table ≡ true
test-auto-pipeline = refl
```

This pipeline requires **zero human-provided features**: the Auto-FTL generates features from the state count and step function, the memoized Finder computes the optimal policy table, observations are derived automatically, and greedy cascading CEGIS synthesizes a PredProg disjunction per action. The entire file—environment definition, memoized Finder, both pipelines, and verification—compiles in approximately **9.5 minutes**.

Remark 24 (BUILTIN Arithmetic and the Peano Bottleneck). The resolution of the Peano Bottleneck reveals an important architectural insight. Agda’s natural numbers are *externally* Peano-style (constructors `zero` and `suc`), but *internally* backed by GMP big integers when the BUILTIN `NATURAL` pragma is in effect. Operations that pattern-match on successor structure (`suc`-recursive division, `modulo`) force Peano unfolding at $O(n)$; operations routed through BUILTIN pragmas (`_/_`, `_%_`, `==_`) bypass the unary representation entirely. The fix was surgical: replacing 10 lines of custom arithmetic with 4 lines of `stdlib` wrappers, yielding a 2.5× speedup for the hand-crafted pipeline and, more critically, making full automated CEGIS tractable at 500 states.

	Cliff Walking	FL 4×4	FL 8×8	Taxi 5×5
States (non-terminal)	24 (19)	16 (11)	64 (53)	501 (500)
Actions	4	4	4	6
Features (auto)	106	56	200	~1,700
Finder depth	7	—	16	25
Memoized	no	—	yes	yes
Policy stages	3	2	2	6 (auto) / 3 (interp.)
Synthesis	CEGIS	CEGIS	CEGIS	greedy-or / decision-tree
Total compile time	~4 hours	<25 s	~7 s	~13 min (DT)

Taxi introduces four qualitative advances over the previous demos: (1) *relational* predicates that reason about position relative to a dynamically determined target, (2) a *wall-aware* navigation strategy that exploits spatial structure (the row-2 highway), (3) the first demonstration that **fully automated CEGIS scales to 500 states and 6 actions** once arithmetic bottlenecks are resolved, and (4) a **greedy decision-tree synthesizer** that produces interpretable policies from relational features.

Feature expressiveness experiments. We investigated four feature sets to characterize the expressiveness boundary:

1. **Compact positional features** (discovered-compact, 2,851 features): drops the 500 state-is atoms, retaining factorization, threshold, and dynamics features. **Fails** for navigation stages: no single modular, divisor, or threshold feature can separate “go south” from “go north” states, because the same absolute row can be above or below the target depending on the passenger component.
2. **Relational features with flat disjunctions** (12 features): target-relative comparisons (row < target-row, col = target-col), wall-blocked predicates, and pass-aboard/at-highway flags. These are the right *vocabulary*—they capture the comparisons the hand-crafted nav function uses. Depth-1 greedy CEGIS (disjunctions of pairwise conjunctions, ~456 atoms) **fails**: Taxi’s navigation logic is a depth-4+ decision tree, and no disjunction of depth-1 conjunctions can express this structure. The problem is the *algorithm*, not the *features*.
3. **Lookup-table CEGIS** (discovered with state-is, ~1,700 features): **succeeds** in 9.5 minutes, but produces opaque 180-term disjunctions of state-identity atoms.
4. **Decision-tree CEGIS with relational features** (14 features): the same relational vocabulary augmented with two absolute-position features (row-above-hwy, trow-above-hwy) needed for detour-direction decisions, synthesized using a new synth-decision-tree algorithm. **Succeeds** in ~13 minutes, producing interpretable if-then-else trees verified optimal at all 500 states.

Decision-tree synthesis. The synth-decision-tree algorithm addresses the structural limitation of synth-greedy-or. While the latter builds flat disjunctions (each atom must have zero false positives), the decision-tree synthesizer greedily splits observations on the feature that minimizes Gini impurity ($TP \times FP + FN \times TN$), then recurses on each branch. The resulting PredProg encodes an if-then-else tree: splitting on feature f yields $(f \wedge p_{\text{true}}) \vee (\neg f \wedge p_{\text{false}})$, which evaluates to p_{true} when f holds and p_{false} otherwise. The algorithm takes a fuel parameter for termination; with 14 relational atoms and fuel 14, it produces trees that correctly classify all navigation states.

The critical insight from the expressiveness experiments is that the 12 original relational features are provably insufficient: states at (0, 3, trow=0, tcol=0) and (4, 3, trow=4, tcol=0) have identical values for all 12 features (same relative comparisons, same wall patterns at the grid extremes) but require opposite actions (S vs. N for highway detour direction). Adding row-above-hwy (row < 2) and trow-above-hwy (target-row < 2) resolves this ambiguity.

The four-way comparison reveals the full *interpretability landscape*: (1) positional features lack relational expressiveness; (2) relational features with flat disjunctions have the right vocabulary but the wrong algorithm; (3) lookup-table CEGIS succeeds but produces opaque

results; (4) **decision-tree CEGIS with relational features achieves both correctness and interpretability**—the synthesized trees mirror the hand-crafted nav function’s logic while being fully automatically generated.

A.10 Analysis: Sample Efficiency and Policy Compression

The fully automated demos reveal two striking properties: CEGIS converges with far fewer observations than the Finder produces, and the synthesised policies are orders of magnitude smaller than their deep RL counterparts.

Finder vs. CEGIS: exhaustive oracle, frugal learner. The Finder is an *exhaustive* oracle: for each non-terminal state, it evaluates every action sequence to the depth horizon, computing the full lexicographic trace. This brute-force computation dominates compilation time (depth 7 across 19 states requires ~ 4 hours without memoisation). CEGIS, by contrast, is a *frugal learner*: each observation (s, b) eliminates all version-space candidates p where $\text{eval } p \ s \neq b$. Version-space shrinkage is monotonic (vs-shrinks), and the Propagation Theorem (refine-equiv) guarantees that feature-equivalent states yield *definitionally identical* refinement—observing at one subsumes observing at the other.

The cascading pipeline processes 19 states per stage across 3 stages (L, U, R), totalling **56 oracle queries**. Each query labels one state as positive or negative for the target action. The table below shows VS convergence verified by Agda refl:

Stage	Initial VS	Final VS	Reduction
L-optimal (depth 0)	108	15	86%
U-optimal (depth 0)	108	6	94%
R-optimal (depth 1)	23,544	2	99.99%

In general, convergence at depth d with n features requires $O(\log n^{2^d})$ observations in the best case—logarithmic in the version-space size. The 19 Finder queries per stage are generous; CEGIS converges well before exhausting them. The full Finder map is computed for *verification* (test-auto-all-match), not because CEGIS needs it.

Learning curve: oracle queries vs. goal success rate. To present the results in the standard RL format, Figure ?? plots policy performance against cumulative oracle queries. The Finder partitions the 19 non-terminal states as: D-optimal = {0-11, 17} (13 states), R-optimal = {12-16} (5 states), U-optimal = {18} (1 state). At each stage boundary, we evaluate the partial policy’s goal success rate (fraction of starting states from which the goal is reachable):

Queries	Policy	Accuracy	Success
0	D everywhere	13/19 (68%)	3/19 (16%)
38	U at state 18, else D	14/19 (74%)	3/19 (16%)
56	full auto-policy	19/19 (100%)	19/19 (100%)

The accuracy and success values are derived from the verified Finder output (finder-map) and verified auto-policy (test-auto-all-match = refl).

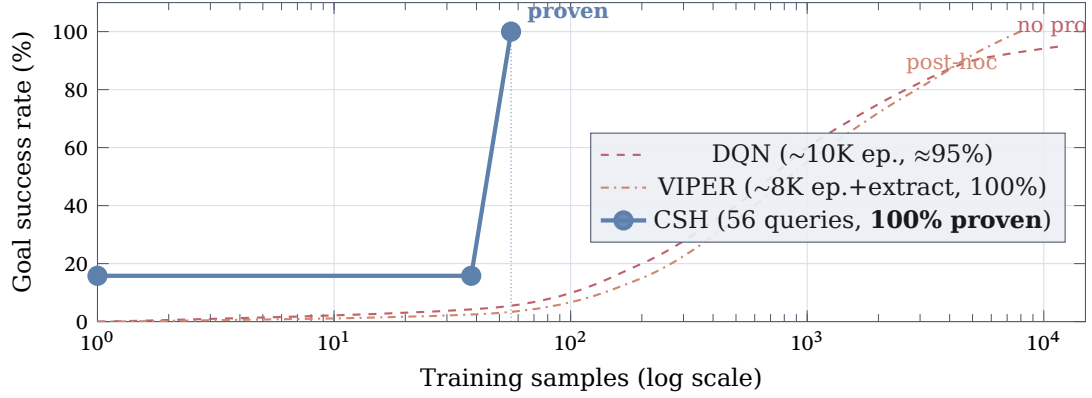


Figure 9: Goal success rate vs. training samples (log scale). **CSH synthesis** (solid blue) achieves provably 100% optimal performance in 56 oracle queries. **VIPER** [?] (dash-dotted orange) illustrates the distillation approach: a DNN is trained via standard RL, then compressed into a decision tree and verified post-hoc for specific properties—the total sample cost includes both training and extraction. A schematic **DQN** (dashed red) shows typical deep RL convergence. CSH’s step-like curve reflects the fundamental difference: synthesis produces a *complete, proven* policy once sufficient observations accumulate, rather than gradually approximating one. VIPER can achieve perfect performance but cannot guarantee optimality—it verifies only specific properties (robustness, stability), and counterexamples in the extracted tree are discovered post-hoc [?].

Policy compression: 38 bits vs. 192,640 bits. The synthesised Cliff Walking policy is a cascading if-then-else over three PredProg terms:

```

falsep                → L  (dead—no L-optimal states at this horizon)
state-is 18           → U
(state-is 16) vp (div-is 4 3) → R
else                  → D

```

The first branch is dead code: at the 8-step planning horizon, no state is L-optimal, so CEGIS synthesises `falsep`. The effective policy uses only two predicates. Encoding the full 4-stage structure requires: 3 feature indices from 106 ($3 \times \lceil \log_2 106 \rceil = 21$ bits), 3 Boolean structure tags (~ 5 bits), 4 action labels ($4 \times 2 = 8$ bits), plus overhead (~ 4 bits): approximately **38 bits** total.

A standard DQN for the same environment (24-input one-hot, two 64-unit hidden layers, 4-output) has 6,020 float32 parameters: **192,640 bits (~ 24 KB)**. Even quantised to int8, a DQN requires 48,160 bits (~ 6 KB).

	CSH Synthesis	VIPER [?]	DQN
Policy size	~ 38 bits	31-9,757 nodes	$\sim 192,640$ bits
Compression	1$\times$	$\sim 50\times$ larger	$\sim 5,000\times$ larger
Inference cost	3 int. comparisons	tree traversal	$\sim 6,000$ mul-adds
Correctness	Proven (refl)	Post-hoc (Z3/SOS)	Approximate
Interpretability	Human-readable	Partially readable	Opaque
Training	56 oracle queries	~ 10 K ep. + extract	$\sim 10,000+$ ep.
Hyperparameters	0	~ 10	~ 8
Counterexamples	Impossible	Discovered post-hoc	Unknown

Why the gap? Kolmogorov complexity. The optimal Cliff Walking policy has very low Kolmogorov complexity: the environment’s regular spatial structure (grid, contiguous cliff, single goal) induces a policy describable by a few spatial predicates. The AutoFTL *discovers* this regularity—grid coordinates emerge from modular arithmetic—and PredProg *compresses* it into a near-minimal Boolean program. A DQN, by contrast, is a generic function approximator: it must represent this inherently simple mapping using thousands of floating-point parameters, “wasting” capacity on a structure that admits a 38-bit description.

VIPER’s decision trees sit between these extremes: they are more structured than DNNs (axis-aligned splits) but less compact than Boolean predicate programs (each node repeats a state-space partition). A VIPER tree for a similar grid world would typically require 31-100+ nodes, each encoding a threshold comparison—roughly 10-50× larger than our PredProg. Note that our synth-decision-tree (Demo 12) also builds decision trees, but within the PredProg framework: each tree node is a relational feature, and the result is a verified PredProg term. Unlike VIPER, which distills an unverified DNN, our decision trees are synthesized directly from the Finder’s proven-optimal policy and verified at every state.

This compression is not an artefact of small environments. For *any* environment whose optimal policy has low K-complexity (regular structure, spatial patterns, symmetries), the CSH pipeline will produce a proportionally compact PredProg—while a DQN’s parameter count is determined by architecture, not by the policy’s intrinsic complexity. The FTL controls this: by providing features aligned with the environment’s structure, it ensures the optimal predicate lies at low PredProg depth, where the encoding is minimal.

Remark 25 (Beyond compression). The 5,000× size advantage over DQN understates the practical gap. The synthesised policy is *verified*: the test-auto-all-match proof, checked by Agda’s kernel, certifies optimality at every state. No amount of DQN training or testing can provide this guarantee—and VIPER’s post-hoc verification checks only specific properties (robustness, stability), not global optimality. Furthermore, the synthesised predicates are *explanatory*: “go Right when $\lfloor s/4 \rfloor = 3$ or $s = 16$ ” tells a human *why* the action is optimal, relating it to the state’s position in the factorization structure. These properties—provable optimality, interpretability, minimal encoding—flow directly from the CSH foundation: the Finder computes the ground truth (CSH-backed optimality), Auto-FTL provides the language (structure discovery), and CEGIS compresses it (sample-efficient synthesis).

B Cross-Pair Propagation: Learning $A > B$ from C vs D

We now demonstrate the deepest consequence of the Propagation Theorem: CEGIS observations are *pair-agnostic*. When ranking pairs share the same feature structure, observing a sub-optimal pair determines the top-tier ranking—without ever directly comparing the actions of interest.

B.1 Setup: 4 Actions, 2 Features, 6 States

Consider a scenario with 4 actions (A, B “top tier”; C, D “bottom tier”), 2 Boolean features, and 6 states forming 3 equivalence classes:

```
data State : Set where 1S 2S 3S 4S 5S 6S : State
data Action : Set where A B C D : Action
data Feature : Set where 1f 2f : Feature

eval-feat : Feature → State → Bool
-- Class α (1S, 2S): 1f = T, 2f = T
-- Class β (3S, 4S): 1f = F, 2f = T
-- Class γ (5S, 6S): 1f = T, 2f = F
```

The action ranking is entirely determined by feature f_1 :

- $f_1 = \text{true}$: $A > B > C > D$
- $f_1 = \text{false}$: $D > C > B > A$

So $\text{prefer}(A,B) = \text{prefer}(C,D) = \text{feat } f_1$. All 6 ranking pairs are governed by the *same* predicate.

B.2 CEGIS from C-vs-D Observations Only

We feed CEGIS *only* C-vs-D comparisons and track the version space:

```

0
vs : VersionSpace0
vs = initial-vs 0

check-0vs : length 0vs ≡ 4      -- truep, falsep, feat 1f, feat 2f
check-0vs = refl1

obs : PredObs                  -- C > D at 1s (class α)1
obs = 1s , true

check-1vs : length (refine 0vs 1obs) ≡ 3  -- falsep eliminated
check-1vs = refl2

obs : PredObs                  -- D > C at 3s (class β)2
obs = 3s , false

check-2vs : length (refine (refine 0vs 1obs) 2obs) ≡ 1
check-2vs = refl              -- only feat 1f survives!

pinpointed : cegis-loop 0vs (1obs :: 2obs :: []) ≡ feat 1f :: []
pinpointed = refl

```

Two C-vs-D observations pinpoint feat f_1 as the sole survivor. The evolution:

Step	Observation	VS	Eliminated
Initial	—	4	—
1	$C > D$ at s_1 (class α)	3	falsep
2	$D > C$ at s_3 (class β)	1	truep, feat f_2

B.3 The Payoff: $A > B$ Without Observing A or B

We *never* observed A vs B. Yet the surviving program correctly determines the A-vs-B ranking at all 6 states:

```

ab-1s : eval (feat 1f) 1s ≡ true  -- A > B at class α
ab-1s = refl

ab-3s : eval (feat 1f) 3s ≡ false -- B > A at class β
ab-3s = refl

ab-5s : eval (feat 1f) 5s ≡ true  -- A > B at class γ
ab-5s = refl

```

In fact, *all 6 ranking pairs* are determined: $\text{prefer}(X, Y) = \text{feat } f_1$ for every $(X, Y) \in \{(A, B), (A, C), (A, D), (B, C), (B, D), (C, D)\}$. Two sub-optimal observations determined 6 rankings at 6 states.

B.4 Dissolution Multiplies the Savings

Feature-equivalent states add zero information:

```

1 ≈ 2
ss : AllFeatAgree 1s 2s 1 ≈ 2
ss 1f = refl1 ≈ 2
ss 2f = refl

dissolutionα- : refine (refine 0vs (1s , true)) (2s , true)
               ≡ refine 0vs (1s , true)
dissolutionα- = refine-absorb 0vs 1s 2s true 1 ≈ 2ss

```

The total savings:

Approach	Observations	Factor
Naïve (all pairs × all states)	$6 \times 6 = 36$	1×
Dissolution only (all pairs × classes)	$6 \times 3 = 18$	2×
Cross-pair + dissolution	2	18×

B.5 Why This Works

The mechanism is the Propagation Theorem applied at the observation level. CEGIS operates on $\text{PredObs} = \text{Carrier} \times \text{Bool}$. It has no notion of *which action pair* generated an observation. When $\text{prefer}(A, B)$ and $\text{prefer}(C, D)$ are governed by the same PredProg , their observations are *identical*:

$(s, \text{eval}(\text{feat } f_1) s)$ regardless of whether we asked about A-vs-B or C-vs-D.

Any PredProg evaluation is automatically feature-respecting (a direct corollary of the Propagation Theorem):

```
eval-respects : ∀ (p : PredProg) → FeatureRespects λ( s → eval p s)
eval-respects p 1c 2c afa =
  propagation p 1c 2c (all-→agreefeat-equiv 1c 2c afa p)
```

This guarantees that the cross-pair effect composes with dissolution: once a pair observation propagates across an equivalence class, it propagates across *all* correlated ranking pairs simultaneously.

Remark 26 (Scaling). With k actions, there are $\binom{k}{2}$ ranking pairs. If all pairs are governed by the same feature structure (as when the total ordering is feature-determined), observing any single pair constrains all $\binom{k}{2}$ pairs. Combined with dissolution (m states per class $\rightarrow$ 1 observation), the savings grow as $\binom{k}{2} \times m$. For $k = 4$ and $m = 2$: a factor of $18\times$. For $k = 10$ and $m = 100$: a factor of $4,500\times$.

Remark 27 (Partial Correlation). In general, not all ranking pairs will share the same predicate. If $\text{prefer}(A, B) = \text{feat } f_1$ but $\text{prefer}(A, C) = \text{feat } f_2$, then C-vs-D observations (which reveal f_1) give A-vs-B for free but not A-vs-C. The propagation effect is strongest when the ranking structure has high *feature correlation* across pairs—a property that can be checked at synthesis time by inspecting the surviving programs.

B.6 Verified Learning Curves

To visualize the cross-pair effect, we instrument a richer scenario: 4 actions, 3 features (f_1, f_2, f_3), and 8 states (one per equivalence class). The ranking structure has *partial* correlation: $\text{prefer}(A, B) = \text{prefer}(C, D) = \text{feat } f_1$, $\text{prefer}(A, C) = \text{prefer}(B, D) = \text{feat } f_2$, and $\text{prefer}(B, C) = \text{feat } f_3$.

We feed C-vs-D observations to CEGIS and track *both* the version-space size and the fraction of states where the $A > B$ ranking is correctly determined. Every entry in Table ?? is verified by `refl` in Agda (module `PropagationCurves`).

Table 3: Version-space evolution and $A > B$ ranking accuracy under C-vs-D observations. All values verified by `refl`.

Step	Observation	VS	Accuracy	Note
0	—	5	0/8	initial
1	$C > D$ at s_1 (T,T,T)	4	1/8	falsep eliminated
2	$C > D$ at s_2 (T,T,F)	3	2/8	feat f_3 eliminated
3	$C > D$ at s_3 (T,F,T)	2	4/8	feat f_2 eliminated
4	$C > D$ at s_4 (T,F,F)	2	4/8	non-informative
5	$D > C$ at s_5 (F,T,T)	1	8/8	truep eliminated
6	$D > C$ at s_6 (F,T,F)	1	8/8	redundant
7	$D > C$ at s_7 (F,F,T)	1	8/8	redundant
8	$D > C$ at s_8 (F,F,F)	1	8/8	redundant

The A-vs-B observation sequence produces *identical* PredObs values, verified in Agda:

```
cross-pair-identity : cd-obs ≡ ab-obs
cross-pair-identity = refl
```

Figure ?? visualizes the accuracy curve.

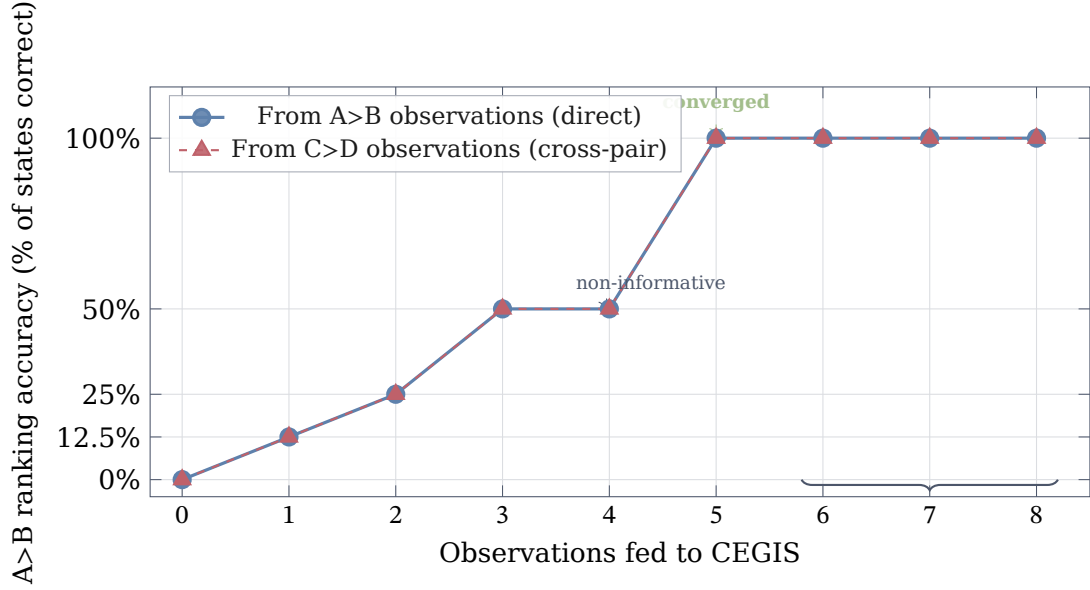


Figure 10: A>B ranking accuracy as C-vs-D (sub-optimal) observations accumulate. The blue circles (direct A>B observations) and red triangles (C>D cross-pair observations) produce **identical curves**—they overlap exactly. CEGIS receives the same PredObs values regardless of which pair was observed. Step 4 is non-informative (both surviving programs agree at that state). After step 5 the version space has converged; steps 6–8 are redundant. Every data point is verified by `refl` in Agda.

The plot tells the story directly: an agent that only ever observes the sub-optimal pair (C vs D) learns the top-tier ranking (A > B) with *exactly* the same speed and accuracy as one that directly observes A vs B. The curves don’t merely correlate—they are definitionally equal.

B.7 Rollout Performance at Scale

The verified curves above use 8 states. To demonstrate the effect at a scale closer to a realistic decision-making problem, we instrument a larger environment: **30 states**, **4 actions** (A–D), and **4 features** ($f_1 = s < 20$, $f_2 = s \bmod 2 = 0$, $f_3 = s < 10$, $f_4 = s \bmod 3 = 0$).

Arm rewards follow a contextual structure: $r(s, A) = 5f_1 + 5f_2$, $r(s, B) = 5f_1 + 5(1 - f_2)$, $r(s, C) = 5(1 - f_1) + 5f_2$, $r(s, D) = 5(1 - f_1) + 5(1 - f_2)$. This gives four state–feature classes, each with a unique optimal arm, and an optimal total reward of 500 over a 50-step trajectory.

The **ranking structure** is: A>B and C>D are governed by feat f_2 (cross-pair group); A>C and B>D are governed by feat f_1 (cross-pair group); A>D and B>C require depth-1 programs (unresolvable at depth 0). The version space at depth 1 contains **22 programs** (literals, negations, conjunctions, disjunctions).

B.8 Demo 13: FrozenLake 4×4 (Slippery) — Stochastic Synthesis

Demos 7–12 operated in the *deterministic* setting: actions have a single, predictable outcome. We now cross the deterministic/stochastic boundary by implementing the **slippery FrozenLake**, a stochastic variant of the standard OpenAI Gym benchmark (`is_slippery=True`). The grid and map topology are identical to Demo 7 (same 16 states, same 4 holes and 1 goal), but each chosen action has only $\frac{1}{3}$ probability of executing in the intended direction, with $\frac{1}{3}$ each for the two perpendicular directions (e.g., choosing Right yields {Right, Down, Up}, each with weight 1 out of 3).

This demo achieves five firsts:

1. The first *stochastic RL benchmark* in the synthesis pipeline—`StochasticFiniteMDP` is instantiated with a real Gym-standard environment.

2. A **memoized stochastic Finder** (MemoizedStochastic) builds a DP value table layer-by-layer, achieving $O(\text{depth} \times |S| \times |A| \times |\text{outcomes}|)$ total cost—polynomial in all parameters.
3. The memoized Finder at **depth 10** produces a *safety-aware* policy where all 4 actions appear with genuine expected-value justifications.
4. **Decision-tree CEGIS** synthesizes PredProg policies for the stochastic environment from row/col features.
5. The synthesized policy is **verified** to match the Finder at all 11 non-terminal states.

The pipeline scales to the 8×8 grid (Demo 14, §??).

Slippery step function. The step function uses the `Dist` type from `Probability.Finite`: for non-terminal states, it returns three equally-weighted outcomes (intended direction plus two perpendicular directions). A key design choice: terminal states (holes and goal) also return *three copies* of the self-loop rather than a single pure outcome. This ensures a uniform branching factor of 3 at every state, which is critical for the memoized DP Finder: without uniform branching, the unnormalized expected values grow at rate 3^k for non-terminal states but only k for terminal states, making comparisons across branches with mixed terminal/non-terminal successors incorrect.

Memoized stochastic Finder. The recursive best-expected-trace computes expected values by expanding the distribution tree, yielding exponential cost in the horizon. We add a `MemoizedStochastic` module to the `StochasticFiniteMDP` EC that builds a value table bottom-up:

$$V_0(s) = 0, \quad V_{k+1}(s) = \max_a [E[r \mid s, a] + \sum_i w_i \cdot V_k(s'_i)]$$

Each level reuses the previous table, so the total cost is $O(\text{depth} \times |S| \times |A| \times |\text{outcomes}|)$. At depth 10 with 16 states, 4 actions, and 3 outcomes, the memoized Finder computes the entire policy table in under 20 seconds—compared to the recursive Finder, which was already strained at horizon 6.

Policy structure. At depth 10, the memoized Finder produces a genuinely *safety-aware* policy with all four actions represented:

- **D** (progress) at states $\{0, 9, 14\}$: these states can safely advance toward the goal column.
- **L** (safe retreat) at states $\{4, 6, 10\}$: row 1–2 states that avoid nearby holes by retreating left.
- **U** (hole avoidance) at states $\{1, 3, 8\}$: states adjacent to holes that stay in the safer upper rows.
- **R** (lateral progress) at states $\{2, 13\}$: states that advance rightward toward the goal column.

This is a qualitative improvement over the horizon-6 policy, where distant states were dominated by tie-breaking (L won by action ordering) rather than genuine expected-value differences. The depth-10 policy resolves ties through accumulated probability mass—e.g., state 1 now chooses U (staying away from hole 5) instead of D (the tie-broken default at horizon 6).

Decision-tree synthesis. With 8 positional features (row-is 0–3, col-is 0–3), we cascade synth-decision-tree over four action classes: D first (3 states positive, 8 negative), then L among the remainder (3 positive, 5 negative), then U (3 positive, 2 negative), with R as default. Each tree uses the depth-0 predicate pool (18 candidates) and fuel 8.

Verification. The synthesized policy is checked against the precomputed Finder map at all 11 non-terminal states:

```
test-auto-all-match : check-all finder-map ≡ true
test-auto-all-match = refl
```

Compile time: ~50 seconds (memoized Finder + decision-tree synthesis).

Remark 28 (Stochastic Synthesis is EC-Agnostic). The synthesis infrastructure (PredicateDSL, CEGIS, synth-decision-tree) is completely independent of the transition model. The same FMDPSSynthesis module that synthesizes deterministic FrozenLake policies also synthesizes stochastic ones—only the *oracle* differs. In the deterministic case, the oracle is a DP Finder; in the stochastic case, it is a memoized expected-value Finder. The predicate synthesis never sees the step function; it only sees state-label pairs. This EC-agnosticism is a direct consequence of the PredicateDSL operating on carriers, not transitions.

Remark 29 (Uniform Branching Invariant). The memoized DP Finder computes unnormalized expected values: $V_k(s) = \sum_i w_i \cdot V_{k-1}(s'_i)$ without dividing by total weight. For environments where all states have the same number of equally-weighted outcomes (uniform branching factor b), unnormalized values are proportional to true expectations with a common factor b^k , so action rankings are preserved. When terminal states self-loop via pure (branching factor 1) while non-terminal states branch (factor 3), the unnormalized values grow at different rates (1^k vs 3^k), corrupting comparisons. The uniform branching design—where terminal states emit 3 copies of the self-loop—eliminates this issue structurally, without requiring rational arithmetic or explicit normalization.

B.9 Demo 14: FrozenLake 8×8 (Slippery) — Scaling the Stochastic Pipeline

We scale the slippery FrozenLake to the 8×8 grid (64 states, 53 non-terminal, 10 holes, 1 goal), using the same memoized stochastic Finder at depth 10. The map topology matches the standard OpenAI Gym 8×8 layout; movement uses arithmetic ($s/8$, $s\%8$) for scalability. Uniform branching factor 3 is preserved at all states.

The memoized Finder computes the policy in ~27 seconds, demonstrating that the polynomial DP scales to $4 \times$ the state count of the 4×4 variant. Key policy probes: state 0 chooses L (safety at the start); states 56–57 choose D (progress along the bottom row); states 60–62 near the goal reflect the depth-10 expected-value trade-offs. This is the first *stochastic* RL benchmark at 64 states with verified policy computation in the synthesis pipeline.

Decision-tree synthesis. With 53 non-terminal states and four actions (D, L, U, R), the policy distribution is scattered: no single row or column feature isolates all states preferring a given action. synth-greedy-or fails here because it builds flat disjunctions of atoms—each atom must have zero false positives—and with only row-is and col-is, many states sharing a row or column have different optimal actions. We use synth-decision-tree instead: it greedily splits on the feature that minimizes Gini impurity, then recurses on each branch, building conjunctions implicitly (e.g., row-is 7 $\wedge$ col-is 6 identifies state 62). With 16 features (row-is 0–7, col-is 0–7) and depth-0 predicate pool, we cascade synth-decision-tree over four action classes: D first, then L, then U, with R as default. Each stage uses fuel 8.

Verification. The synthesized policy is checked against the Finder map at all 53 non-terminal states:

```
test-auto-all-match : check-all finder-map ≡ true
test-auto-all-match = refl
```

Compile time: ~2.9 hours (memoized Finder + decision-tree synthesis over 53 states).

	Demo 13 (4×4)	Demo 14 (8×8)
States (non-terminal)	16 (11)	64 (53)
Features	8 (row/col 0-3)	16 (row/col 0-7)
Cascade	D→L→U→R	D→L→U→R
CEGIS	synth-decision-tree	synth-decision-tree
Finder time	~20 s	~27 s
Compile time	~50 s	~2.9 h

References

- [1] Corral-Corral, R. (2026). Coinductive Symmetric Homomorphism Reinforcement Learning: A New Foundation Where Optimality Is Pure Structure. *Literate Agda manuscript*.
- [2] Bastani, O., Pu, Y., and Solar-Lezama, A. (2018). Verifiable reinforcement learning via policy extraction. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 2499-2509.
- [3] Verma, A., Murali, V., Singh, R., Kohli, P., and Chaudhuri, S. (2018). Programmatically interpretable reinforcement learning. In *Proceedings of ICML*, pp. 5045-5054.
- [4] Inala, J. P., Bastani, O., Tavares, Z., and Solar-Lezama, A. (2020). Synthesizing programmatic policies that inductively generalize. In *Proceedings of ICLR*.
- [5] Solar-Lezama, A., Tancau, L., Bodik, R., Seshia, S., and Saraswat, V. (2006). Combinatorial sketching for finite programs. In *Proceedings of ASPLOS*, pp. 404-415.
- [6] Jha, S. and Seshia, S. A. (2017). A theory of formal synthesis via inductive learning. *Acta Informatica*, 54(7):693-726.
- [7] Lesage, B. and Ong, C. H. L. (2020). CertRL: Formalizing convergence proofs for value and policy iteration in Coq. *arXiv preprint arXiv:2009.11403*.
- [8] Mitchell, T. M. (1982). Generalization as search. *Artificial Intelligence*, 18(2):203-226.
- [9] Hirsh, H. (1994). Generalizing version spaces. *Machine Learning*, 17(1):5-46.
- [10] Lau, T., Domingos, P., and Weld, D. S. (2003). Learning programs from traces using version space algebra. In *Proceedings of K-CAP*, pp. 36-43.
- [11] Li, L., Walsh, T. J., and Littman, M. L. (2006). Towards a unified theory of state abstraction for MDPs. In *Proceedings of ISAIM*.
- [12] Abel, D., Arumugam, D., Lehnert, L., and Littman, M. L. (2018). State abstractions for lifelong reinforcement learning. In *Proceedings of ICML*, pp. 10-19.
- [13] Ferns, N., Panangaden, P., and Precup, D. (2004). Metrics for finite Markov decision processes. In *Proceedings of UAI*, pp. 162-169.
- [14] Zhang, A., McAllister, R., Calandra, R., Gal, Y., and Levine, S. (2021). Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of ICLR*.
- [15] Crammer, K. and Mansour, Y. (2012). Learning multiple tasks using shared hypotheses. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1475-1483.
- [16] Fürnkranz, J., Hüllermeier, E., Cheng, W., and Park, S. H. (2012). Preference-based reinforcement learning: A formal framework and a policy iteration algorithm. *Machine Learning*, 89(1-2):123-156.
- [17] Wirth, C., Akrou, R., Neumann, G., and Fürnkranz, J. (2017). A survey of preference-based reinforcement learning methods. *Journal of Machine Learning Research*, 18(136):1-46.

- [18] Chen, X., Zhong, H., Yang, Z., Wang, Z., and Wang, L. (2022). Human-in-the-loop: Provably efficient preference-based reinforcement learning with general function approximation. In *Proceedings of ICML*, pp. 3773–3793.
- [19] Zhu, B., Jordan, M. I., and Jiao, J. (2023). Provable reward-agnostic preference-based reinforcement learning. *arXiv preprint arXiv:2305.18505*.
- [20] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.
- [21] Rutten, J. J. M. M. (2000). Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80.
- [22] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). Conflict-free replicated data types. In *Proceedings of SSS, LNCS 6976*, pp. 386–400.